

PRATISHA

Search & Rescue · Human Detection — Source Code Reference

Live app links

Published app: <https://human-finder-ai.lovable.app>

Preview build: <https://id-preview--ffeae1c-78e3-4558-bde2-90cb9631ade2.lovable.app>

Files in this document

`components.json`
`package.json`
`src/components/ConfidenceBars.tsx`
`src/components/CooledThermalCanvas.tsx`
`src/components/CooledThermalDetector.tsx`
`src/components/DetectionCanvas.tsx`
`src/components/GasCanvas.tsx`
`src/components/GasDetector.tsx`
`src/components/GradientBuilder.tsx`
`src/components/HudOverlay.tsx`
`src/components/HyperspectralCanvas.tsx`
`src/components/HyperspectralDetector.tsx`
`src/components/ImageDetector.tsx`
`src/components/LifeCalibration.tsx`
`src/components/LifeStatus.tsx`
`src/components/PratishaInfo.tsx`
`src/components/SettingsBar.tsx`
`src/components/SystemTicker.tsx`
`src/components/ThermalCanvas.tsx`
`src/components/ThermalDetector.tsx`
`src/components/WebcamDetector.tsx`
`src/components/XRayCanvas.tsx`
`src/components/XRayDetector.tsx`
`src/components/ui/accordion.tsx`
`src/components/ui/alert-dialog.tsx`
`src/components/ui/alert.tsx`
`src/components/ui/aspect-ratio.tsx`
`src/components/ui/avatar.tsx`
`src/components/ui/badge.tsx`
`src/components/ui/breadcrumb.tsx`
`src/components/ui/button.tsx`
`src/components/ui/calendar.tsx`
`src/components/ui/card.tsx`
`src/components/ui/carousel.tsx`
`src/components/ui/chart.tsx`
`src/components/ui/checkbox.tsx`
`src/components/ui/collapsible.tsx`
`src/components/ui/command.tsx`
`src/components/ui/context-menu.tsx`
`src/components/ui/dialog.tsx`
`src/components/ui/drawer.tsx`
`src/components/ui/dropdown-menu.tsx`
`src/components/ui/form.tsx`
`src/components/ui/hover-card.tsx`
`src/components/ui/input-otp.tsx`
`src/components/ui/input.tsx`
`src/components/ui/label.tsx`
`src/components/ui/menubar.tsx`
`src/components/ui/navigation-menu.tsx`
`src/components/ui/pagination.tsx`

[src/components/ui/popover.tsx](#)
[src/components/ui/progress.tsx](#)
[src/components/ui/radio-group.tsx](#)
[src/components/ui/resizable.tsx](#)
[src/components/ui/scroll-area.tsx](#)
[src/components/ui/select.tsx](#)
[src/components/ui/separator.tsx](#)
[src/components/ui/sheet.tsx](#)
[src/components/ui/sidebar.tsx](#)
[src/components/ui/skeleton.tsx](#)
[src/components/ui/slider.tsx](#)
[src/components/ui/sonner.tsx](#)
[src/components/ui/switch.tsx](#)
[src/components/ui/table.tsx](#)
[src/components/ui/tabs.tsx](#)
[src/components/ui/textarea.tsx](#)
[src/components/ui/toggle-group.tsx](#)
[src/components/ui/toggle.tsx](#)
[src/components/ui/tooltip.tsx](#)
[src/hooks/use-mobile.tsx](#)
[src/lib/api/example.functions.ts](#)
[src/lib/camera-error.ts](#)
[src/lib/config.server.ts](#)
[src/lib/detector.ts](#)
[src/lib/error-capture.ts](#)
[src/lib/error-page.ts](#)
[src/lib/lovable-error-reporting.ts](#)
[src/lib/pratisha-content.ts](#)
[src/lib/settings.tsx](#)
[src/lib/utils.ts](#)
[src/router.tsx](#)
[src/routes/__root.tsx](#)
[src/routes/index.tsx](#)
[src/server.ts](#)
[src/start.ts](#)
[src/styles.css](#)
[tsconfig.json](#)
[vite.config.ts](#)

Powered by Sarthak Kumar Panda — ATL student, DAV School Pokhariput

components.json

```
1 | {
2 |   "$schema": "https://ui.shadcn.com/schema.json",
3 |   "style": "new-york",
4 |   "rsc": false,
5 |   "tsx": true,
6 |   "tailwind": {
7 |     "css": "src/styles.css",
8 |     "baseColor": "slate",
9 |     "cssVariables": true,
10 |    "prefix": ""
11 |  },
12 |   "iconLibrary": "lucide",
13 |   "rtl": false,
14 |   "aliases": {
15 |     "components": "@components",
16 |     "utils": "@lib/utils",
17 |     "ui": "@components/ui",
18 |     "lib": "@lib",
19 |     "hooks": "@hooks"
20 |   },
21 |   "registries": {}
22 | }
```

package.json

```
1 | {
2 |   "name": "tanstack_start_ts",
3 |   "private": true,
4 |   "sideEffects": false,
5 |   "type": "module",
6 |   "scripts": {
7 |     "dev": "vite dev",
8 |     "build": "vite build",
9 |     "build:dev": "vite build --mode development",
10 |    "preview": "vite preview",
11 |    "lint": "eslint .",
12 |    "format": "prettier --write ."
13 |  },
14 |  "dependencies": {
15 |    "@hookform/resolvers": "^5.2.2",
16 |    "@radix-ui/react-accordion": "^1.2.12",
17 |    "@radix-ui/react-alert-dialog": "^1.1.15",
18 |    "@radix-ui/react-aspect-ratio": "^1.1.8",
19 |    "@radix-ui/react-avatar": "^1.1.11",
20 |    "@radix-ui/react-checkbox": "^1.3.3",
21 |    "@radix-ui/react-collapsible": "^1.1.12",
22 |    "@radix-ui/react-context-menu": "^2.2.16",
23 |    "@radix-ui/react-dialog": "^1.1.15",
24 |    "@radix-ui/react-dropdown-menu": "^2.1.16",
25 |    "@radix-ui/react-hover-card": "^1.1.15",
26 |    "@radix-ui/react-label": "^2.1.8",
27 |    "@radix-ui/react-menubar": "^1.1.16",
28 |    "@radix-ui/react-navigation-menu": "^1.2.14",
29 |    "@radix-ui/react-popover": "^1.1.15",
30 |    "@radix-ui/react-progress": "^1.1.8",
31 |    "@radix-ui/react-radio-group": "^1.3.8",
32 |    "@radix-ui/react-scroll-area": "^1.2.10",
33 |    "@radix-ui/react-select": "^2.2.6",
34 |    "@radix-ui/react-separator": "^1.1.8",
35 |    "@radix-ui/react-slider": "^1.3.6",
36 |    "@radix-ui/react-slot": "^1.2.4",
37 |    "@radix-ui/react-switch": "^1.2.6",
38 |    "@radix-ui/react-tabs": "^1.1.13",
39 |    "@radix-ui/react-toggle": "^1.1.10",
40 |    "@radix-ui/react-toggle-group": "^1.1.11",
41 |    "@radix-ui/react-tooltip": "^1.2.8",
42 |    "@tailwindcss/vite": "^4.2.1",
43 |    "@tanstack/react-query": "^5.83.0",
44 |    "@tanstack/react-router": "^1.168.25",
45 |    "@tanstack/react-start": "^1.167.50",
46 |    "@tanstack/router-plugin": "^1.167.28",
47 |    "@tensorflow-models/coco-ssd": "^2.2.3",
48 |    "@tensorflow/tfjs": "^4.22.0",
49 |    "class-variance-authority": "^0.7.1",
50 |    "clsx": "^2.1.1",
51 |    "cmdk": "^1.1.1",
52 |    "date-fns": "^4.1.0",
53 |    "embla-carousel-react": "^8.6.0",
54 |    "input-otp": "^1.4.2",
55 |    "lucide-react": "^0.575.0",
56 |    "react": "^19.2.0",
57 |    "react-day-picker": "^9.14.0",
58 |    "react-dom": "^19.2.0",
59 |    "react-hook-form": "^7.71.2",
60 |    "react-resizable-panels": "^4.6.5",
61 |    "recharts": "^2.15.4",
62 |    "sonner": "^2.0.7",
63 |    "tailwind-merge": "^3.5.0",
64 |    "tailwindcss": "^4.2.1",
65 |    "tw-animate-css": "^1.3.4",
66 |    "vaul": "^1.1.2",
67 |    "vite-tsconfig-paths": "^6.0.2",
68 |    "zod": "^3.24.2"
69 |  },
70 |  "devDependencies": {
71 |    "@eslint/js": "^9.32.0",
72 |    "@lovable.dev/vite-tanstack-config": "2.13.1",
73 |    "@types/node": "^22.16.5",
74 |    "@types/react": "^19.2.0",
75 |    "@types/react-dom": "^19.2.0",
76 |    "@vitejs/plugin-react": "^5.0.4",
77 |    "eslint": "^9.32.0",
78 |    "eslint-config-prettier": "^10.1.1",
79 |    "eslint-plugin-prettier": "^5.2.6",
80 |    "eslint-plugin-react-hooks": "^5.2.0",
81 |    "eslint-plugin-react-refresh": "^0.4.20",
82 |    "globals": "^15.15.0",
83 |    "nitro": "3.0.260603-beta",
84 |    "prettier": "^3.7.3",
85 |    "typescript": "^5.8.3",
86 |    "typescript-eslint": "^8.56.1",
87 |    "vite": "^7.3.1"
88 |  }
89 | }
```

src/components/ConfidenceBars.tsx

```
1 | import type { Detection } from "@lib/detector";
2 | import { useSettings } from "@lib/settings";
3 |
4 | interface Props {
5 |   detections: Detection[];
6 | }
7 |
8 | export function ConfidenceBars({ detections }: Props) {
9 |   const { t } = useSettings();
10 |   if (detections.length === 0) return null;
11 |
12 |   return (
13 |     <div className="w-full space-y-2 p-3 rounded-xl bg-card border border-border">
14 |       <p className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
15 |         {t("confidence")}
16 |       </p>
17 |       <div className="space-y-2">
18 |         {detections.map((d, i) => {
19 |           const pct = Math.round(d.score * 100);
20 |           let barColor = "bg-emerald-500";
21 |           if (pct < 60) barColor = "bg-amber-500";
22 |           if (pct < 40) barColor = "bg-rose-500";
23 |
24 |           return (
25 |             <div key={i} className="space-y-1">
26 |               <div className="flex items-center justify-between text-xs">
27 |                 <span className="font-medium">
28 |                   {t("person")} {i + 1}
29 |                 </span>
30 |                 <span className="font-bold tabular-nums">{pct}%</span>
31 |               </div>
32 |               <div className="h-2.5 w-full rounded-full bg-muted overflow-hidden">
33 |                 <div
34 |                   className={`h-full rounded-full ${barColor} transition-all duration-500 ease-out`}
35 |                   style={{ width: `${pct}%` }}
36 |                 />
37 |               </div>
38 |             </div>
39 |           );
40 |         })}
41 |       </div>
42 |     </div>
43 |   );
44 | }
```

src/components/CooledThermalCanvas.tsx

```
1 | import { useEffect, useRef } from "react";
2 | import type { Detection } from "@lib/detector";
3 |
4 | // White-hot / iron palette approximation with smoother stops for a "cooled"
5 | // (higher sensitivity, lower NETD) look. More separation in the mid-band.
6 | function ironColor(t: number): [number, number, number] {
7 |   t = Math.max(0, Math.min(1, t));
8 |   const stops: Array<[number, [number, number, number]]> = [
9 |     [0.0, [4, 6, 30]],
10 |    [0.15, [24, 12, 90]],
11 |    [0.32, [96, 20, 140]],
12 |    [0.5, [176, 40, 120]],
13 |    [0.65, [224, 90, 60]],
14 |    [0.8, [246, 160, 30]],
15 |    [0.92, [252, 220, 90]],
16 |    [1.0, [255, 255, 240]],
17 |  ];
18 |   for (let i = 0; i < stops.length - 1; i++) {
19 |     const [a, ca] = stops[i];
20 |     const [b, cb] = stops[i + 1];
21 |     if (t >= a && t <= b) {
22 |       const k = (t - a) / (b - a);
23 |       return [
24 |         ca[0] + (cb[0] - ca[0]) * k,
25 |         ca[1] + (cb[1] - ca[1]) * k,
26 |         ca[2] + (cb[2] - ca[2]) * k,
27 |       ];
28 |     }
29 |   }
30 |   return [255, 255, 255];
31 | }
32 |
33 | export interface CooledThermalCanvasProps {
34 |   source: HTMLVideoElement | HTMLImageElement | null;
35 |   width: number;
36 |   height: number;
37 |   detections: Detection[];
38 |   className?: string;
39 |   onTemps?: (temps: number[]) => void;
40 | }
41 |
42 | // Higher-precision mapping for a cooled sensor (sub-0.1°C resolution).
43 | function lumToCelsius(avgLum: number): number {
44 |   const clamped = Math.max(0.15, Math.min(0.97, avgLum));
45 |   const t = (clamped - 0.15) / (0.97 - 0.15);
46 |   const base = 35.2 + t * 3.1;
47 |   // NETD ~0.02°C – tiny jitter
48 |   return Math.round((base + (Math.random() - 0.5) * 0.04) * 100) / 100;
49 | }
50 |
51 | export function CooledThermalCanvas({
52 |   source,
53 |   width,
54 |   height,
55 |   detections,
56 |   className,
57 |   onTemps,
58 | }: CooledThermalCanvasProps) {
59 |   const canvasRef = useRef<HTMLCanvasElement>(null);
60 |   const offRef = useRef<HTMLCanvasElement | null>(null);
61 |   const prevLumRef = useRef<Float32Array | null>(null);
62 |   const rafRef = useRef<number | null>(null);
63 |   const lastEmitRef = useRef(0);
64 |
65 |   useEffect(() => {
66 |     const canvas = canvasRef.current;
67 |     if (!canvas || !source || width === 0 || height === 0) return;
68 |     canvas.width = width;
69 |     canvas.height = height;
70 |     if (!offRef.current) offRef.current = document.createElement("canvas");
71 |     const off = offRef.current;
72 |     off.width = width;
73 |     off.height = height;
74 |     const offCtx = off.getContext("2d", { willReadFrequently: true });
75 |     const ctx = canvas.getContext("2d");
76 |     const pixCount = width * height;
77 |     if (!prevLumRef.current || prevLumRef.current.length !== pixCount) {
78 |       prevLumRef.current = new Float32Array(pixCount);
79 |     }
80 |
81 |     const draw = () => {
82 |       try {
83 |         offCtx.drawImage(source, 0, 0, width, height);
84 |         const img = offCtx.getImageData(0, 0, width, height);
85 |         const data = img.data;
86 |         const prev = prevLumRef.current!;
87 |         // Temporal + histogram stretch for cooled sensor look
88 |         let minL = 1, maxL = 0;
89 |         const lumBuf = new Float32Array(pixCount);
90 |         for (let p = 0, i = 0; p < pixCount; p++, i += 4) {
```

```

91 |         const l =
92 |             (0.299 * data[i] + 0.587 * data[i + 1] + 0.114 * data[i + 2]) / 255;
93 |         // 60/40 blend with previous frame -> denoise (cooled sensor stability)
94 |         const smooth = prev[p] * 0.6 + l * 0.4;
95 |         prev[p] = smooth;
96 |         lumBuf[p] = smooth;
97 |         if (smooth < minL) minL = smooth;
98 |         if (smooth > maxL) maxL = smooth;
99 |     }
100 |     const range = Math.max(0.001, maxL - minL);
101 |     for (let p = 0, i = 0; p < pixCount; p++, i += 4) {
102 |         const t = (lumBuf[p] - minL) / range;
103 |         const [r, g, b] = ironColor(t);
104 |         data[i] = r;
105 |         data[i + 1] = g;
106 |         data[i + 2] = b;
107 |     }
108 |     ctx.putImageData(img, 0, 0);
109 |
110 |     // Crosshair reticle center
111 |     ctx.strokeStyle = "rgba(255,255,255,0.35)";
112 |     ctx.lineWidth = 1;
113 |     ctx.beginPath();
114 |     ctx.moveTo(width / 2 - 12, height / 2);
115 |     ctx.lineTo(width / 2 + 12, height / 2);
116 |     ctx.moveTo(width / 2, height / 2 - 12);
117 |     ctx.lineTo(width / 2, height / 2 + 12);
118 |     ctx.stroke();
119 |
120 |     const temps: number[] = [];
121 |     ctx.font = `${Math.max(14, Math.round(height / 28))}px ui-sans-serif, system-ui`;
122 |     ctx.lineWidth = 3;
123 |     detections.forEach((d, idx) => {
124 |         const [x, y, w, h] = d.bbox;
125 |         const fx = Math.max(0, Math.round(x + w * 0.25));
126 |         const fy = Math.max(0, Math.round(y));
127 |         const fw = Math.max(1, Math.round(w * 0.5));
128 |         const fh = Math.max(1, Math.round(h * 0.25));
129 |         let sum = 0, count = 0;
130 |         try {
131 |             const face = offCtx.getImageData(fx, fy, fw, fh).data;
132 |             for (let i = 0; i < face.length; i += 16) {
133 |                 const lum =
134 |                     (0.299 * face[i] + 0.587 * face[i + 1] + 0.114 * face[i + 2]) /
135 |                     255;
136 |                 sum += lum;
137 |                 count++;
138 |             }
139 |         } catch {
140 |             /* noop */
141 |         }
142 |         const avg = count > 0 ? sum / count : 0.5;
143 |         const temp = lumToCelsius(avg);
144 |         temps.push(temp);
145 |         const fever = temp >= 37.8;
146 |         ctx.strokeStyle = fever ? "#ff3b30" : "#00e5ff";
147 |         ctx.strokeRect(x, y, w, h);
148 |         const tempF = temp * 9 / 5 + 32;
149 |         const label = `${idx + 1} ${temp.toFixed(2)}°C / ${tempF.toFixed(2)}°F${fever ? " FEVER" : ""}`;
150 |         const padding = 6;
151 |         const metrics = ctx.measureText(label);
152 |         const lh = Math.max(18, Math.round(height / 24));
153 |         ctx.fillStyle = fever ? "rgba(255,59,48,0.92)" : "rgba(0,0,0,0.7)";
154 |         ctx.fillRect(x, Math.max(0, y - lh - 2), metrics.width + padding * 2, lh + 2);
155 |         ctx.fillStyle = "#fff";
156 |         ctx.fillText(label, x + padding, Math.max(lh - 4, y - 6));
157 |     });
158 |
159 |     const now = performance.now();
160 |     if (onTemps && now - lastEmitRef.current > 300) {
161 |         lastEmitRef.current = now;
162 |         onTemps(temps);
163 |     }
164 | } catch {
165 |     /* not ready */
166 | }
167 | rafRef.current = requestAnimationFrame(draw);
168 | };
169 | draw();
170 | return () => {
171 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
172 | };
173 | }, [source, width, height, detections, onTemps]);
174 |
175 | return <canvas ref={canvasRef} className={className} />;
176 | }

```

src/components/CooledThermalDetector.tsx

```
1 | import { describeCameraError } from "@lib/camera-error";
2 | import { useEffect, useRef, useState } from "react";
3 | import { Button } from "@components/ui/button";
4 | import { Card } from "@components/ui/card";
5 | import { detectPeople, loadModel, type Detection } from "@lib/detector";
6 | import { CooledThermalCanvas } from "../CooledThermalCanvas";
7 | import { ConfidenceBars } from "../ConfidenceBars";
8 | import { LifeStatus } from "../LifeStatus";
9 | import { LifeCalibration, DEFAULTS_THERMAL, type LifeCalibrationValues } from "../LifeCalibration";
10 | import { CameraOff, Loader2, Snowflake, Thermometer, Users } from "lucide-react";
11 |
12 | export function CooledThermalDetector() {
13 |   const videoRef = useRef<HTMLVideoElement>(null);
14 |   const rafRef = useRef<number | null>(null);
15 |   const [active, setActive] = useState(false);
16 |   const [loading, setLoading] = useState(false);
17 |   const [error, setError] = useState<string | null>(null);
18 |   const [detections, setDetections] = useState<Detection[]>([]);
19 |   const [size, setSize] = useState({ w: 640, h: 480 });
20 |   const [temps, setTemps] = useState<number[]>([]);
21 |   const [calibration, setCalibration] = useState<LifeCalibrationValues>(DEFAULTS_THERMAL);
22 |   const [cooldown, setCooldown] = useState(0); // 0..100% sensor cooldown
23 |
24 |   const stop = () => {
25 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
26 |     rafRef.current = null;
27 |     const stream = videoRef.current?.srcObject as MediaStream | null;
28 |     stream?.getTracks().forEach((t) => t.stop());
29 |     if (videoRef.current) videoRef.current.srcObject = null;
30 |     setActive(false);
31 |     setDetections([]);
32 |     setTemps([]);
33 |     setCooldown(0);
34 |   };
35 |
36 |   const start = async () => {
37 |     setError(null);
38 |     setLoading(true);
39 |     try {
40 |       await loadModel();
41 |       const stream = await navigator.mediaDevices.getUserMedia({
42 |         video: { facingMode: "user" },
43 |         audio: false,
44 |       });
45 |       const video = videoRef.current!;
46 |       video.srcObject = stream;
47 |       await video.play();
48 |       setSize({ w: video.videoWidth, h: video.videoHeight });
49 |       setActive(true);
50 |       // Simulated cryogenic cool-down (Stirling cooler) ramp
51 |       const cdStart = performance.now();
52 |       const cdTick = () => {
53 |         const p = Math.min(100, ((performance.now() - cdStart) / 2500) * 100);
54 |         setCooldown(p);
55 |         if (p < 100) requestAnimationFrame(cdTick);
56 |       };
57 |       cdTick();
58 |       const loop = async () => {
59 |         if (!videoRef.current) return;
60 |         try {
61 |           const found = await detectPeople(videoRef.current);
62 |           setDetections(found);
63 |         } catch {
64 |           /* ignore */
65 |         }
66 |         rafRef.current = requestAnimationFrame(loop);
67 |       };
68 |       loop();
69 |     } catch (e) {
70 |       setError(describeCameraError(e));
71 |     } finally {
72 |       setLoading(false);
73 |     }
74 |   };
75 |
76 |   useEffect(() => () => stop(), []);
77 |
78 |   const maxTemp = temps.length ? Math.max(...temps) : null;
79 |   const minTemp = temps.length ? Math.min(...temps) : null;
80 |   const maxTempF = maxTemp !== null ? maxTemp * 9 / 5 + 32 : null;
81 |   const fever = maxTemp !== null && maxTemp >= 37.8;
82 |   const cooled = cooldown >= 100;
83 |
84 |   return (
85 |     <
86 |     <Card className="overflow-hidden">
87 |       <div className="relative bg-black aspect-video flex items-center justify-center">
88 |         <video
89 |           ref={videoRef}
90 |           playsInline
```

```

91 |         muted
92 |         className="absolute inset-0 w-full h-full object-contain opacity-0 pointer-events-none"
93 |     />
94 |     {active && (
95 |         <CooledThermalCanvas
96 |             source={videoRef.current}
97 |             width={size.w}
98 |             height={size.h}
99 |             detections={detections}
100 |             onTemps={setTemps}
101 |             className="w-full h-full object-contain"
102 |         />
103 |     )}
104 |     {!active && (
105 |         <div className="text-center text-muted-foreground p-8">
106 |             <Snowflake className="w-12 h-12 mx-auto mb-3 opacity-50" />
107 |             <p className="text-sm">Cooled thermal camera is off</p>
108 |             <p className="text-xs mt-1 opacity-70">
109 |                 Cryogenic MWIR sensor · NETD ~0.02°C
110 |             </p>
111 |         </div>
112 |     )}
113 |     {active && (
114 |         <
115 |             <div className="absolute top-3 left-3 flex items-center gap-2 px-3 py-1.5 rounded-full bg-background/90
backdrop-blur text-sm font-medium shadow">
116 |                 <Users className="w-4 h-4 text-primary" />
117 |                 {detections.length} {detections.length === 1 ? "person" : "people"}
118 |             </div>
119 |             <div
120 |                 className={`absolute top-3 right-3 flex items-center gap-2 px-3 py-1.5 rounded-full backdrop-blur text
-xs font-semibold shadow ${
121 |                     cooled ? "bg-cyan-500/90 text-white" : "bg-amber-500/90 text-white"
122 |                 }`}
123 |             >
124 |                 <Snowflake className={`w-3.5 h-3.5 ${cooled ? "" : "animate-spin"}}` />
125 |                 {cooled ? "MWIR · 77 K" : `Cooling ${cooldown.toFixed(0)}%`}
126 |             </div>
127 |             {maxTemp !== null && maxTempF !== null && cooled && (
128 |                 <div
129 |                     className={`absolute top-14 right-3 flex items-center gap-2 px-3 py-1.5 rounded-full backdrop-blur t
ext-sm font-mono font-semibold shadow ${
130 |                         fever ? "bg-red-500/90 text-white" : "bg-background/90 text-foreground"
131 |                     }`}
132 |                 >
133 |                     <Thermometer className="w-4 h-4" />
134 |                     {maxTemp.toFixed(2)}°C / {maxTempF.toFixed(2)}°F {fever ? " · Fever" : ""}
135 |                 </div>
136 |             )}
137 |             {minTemp !== null && cooled && (
138 |                 <div className="absolute top-24 right-3 px-2.5 py-1 rounded-full bg-background/80 backdrop-blur text-[
11px] font-mono shadow">
139 |                     Δ {(maxTemp! - minTemp).toFixed(2)}°C · NETD 0.02°C
140 |                 </div>
141 |             )}
142 |             <div
143 |                 className="absolute bottom-3 left-3 right-3 h-2 rounded-full overflow-hidden shadow"
144 |                 style={{
145 |                     background:
146 |                         "linear-gradient(90deg, rgb(4,6,30), rgb(24,12,90), rgb(96,20,140), rgb(176,40,120), rgb(224,90,60
), rgb(246,160,30), rgb(252,220,90), rgb(255,255,240))",
147 |                 }}
148 |             />
149 |             <div className="absolute bottom-6 left-3 right-3 flex justify-between text-[10px] text-white/80 font-mon
o">
150 |                 <span>cold</span>
151 |                 <span>~36°C</span>
152 |                 <span>~37°C</span>
153 |                 <span>hot</span>
154 |             </div>
155 |         </>
156 |     )}
157 | </div>
158 | <div className="p-4 flex items-center justify-between gap-3">
159 |     <p className="text-sm text-muted-foreground">
160 |         {error ??
161 |             (active
162 |                 ? cooled
163 |                 ? "Cooled MWIR mode · sub-0.1°C precision, temporal denoise"
164 |                 : "Cryogenic cooler stabilising sensor..."
165 |                 : "Start the cooled thermal camera (simulated cryogenic MWIR sensor)")}
166 |     </p>
167 |     {active ? (
168 |         <Button onClick={stop} variant="secondary">
169 |             <CameraOff className="w-4 h-4" /> Stop
170 |         </Button>
171 |     ) : (
172 |         <Button onClick={start} disabled={loading}>
173 |             {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : <Snowflake className="w-4 h-4" />}
174 |             {loading ? "Loading model..." : "Start cooled thermal"}
175 |         </Button>
176 |     )}
177 | </div>

```

```

178 |         </Card>
179 |         {active && (
180 |             <LifeStatus
181 |                 source={videoRef.current}
182 |                 detections={detections}
183 |                 active={active}
184 |                 calibration={calibration}
185 |             />
186 |         )}
187 |         {active && (
188 |             <LifeCalibration
189 |                 label="Cooled Thermal"
190 |                 values={calibration}
191 |                 defaults={DEFAULTS_THERMAL}
192 |                 onChange={setCalibration}
193 |             />
194 |         )}
195 |         {active && <ConfidenceBars detections={detections} />}
196 |     </>
197 | );
198 | }

```

src/components/DetectionCanvas.tsx

```
1 | import { useEffect, useRef } from "react";
2 | import type { Detection } from "@lib/detector";
3 |
4 | interface Props {
5 |   width: number;
6 |   height: number;
7 |   detections: Detection[];
8 |   className?: string;
9 | }
10 |
11 | export function DetectionCanvas({ width, height, detections, className }: Props) {
12 |   const ref = useRef<HTMLCanvasElement>(null);
13 |
14 |   useEffect(() => {
15 |     const canvas = ref.current;
16 |     if (!canvas) return;
17 |     const ctx = canvas.getContext("2d");
18 |     if (!ctx) return;
19 |     ctx.clearRect(0, 0, width, height);
20 |     ctx.lineWidth = 3;
21 |     ctx.font = "600 14px ui-sans-serif, system-ui, sans-serif";
22 |     ctx.strokeStyle = "hsl(142 76% 50%)";
23 |     ctx.fillStyle = "hsl(142 76% 50%)";
24 |
25 |     detections.forEach((d, i) => {
26 |       const [x, y, w, h] = d.bbox;
27 |       ctx.strokeRect(x, y, w, h);
28 |       const label = `Person ${i + 1} · ${((d.score * 100).toFixed(0))}%`;
29 |       const pad = 6;
30 |       const tw = ctx.measureText(label).width + pad * 2;
31 |       const th = 22;
32 |       ctx.fillRect(x, Math.max(0, y - th), tw, th);
33 |       ctx.fillStyle = "hsl(0 0% 8%)";
34 |       ctx.fillText(label, x + pad, Math.max(14, y - 6));
35 |       ctx.fillStyle = "hsl(142 76% 50%)";
36 |     });
37 |   }, [detections, width, height]);
38 |
39 |   return (
40 |     <canvas
41 |       ref={ref}
42 |       width={width}
43 |       height={height}
44 |       className={className}
45 |       style={{ pointerEvents: "none" }}
46 |     />
47 |   );
48 | }
```

src/components/GasCanvas.tsx

```
1 | import { useEffect, useRef } from "react";
2 | import type { Detection } from "@lib/detector";
3 |
4 | export interface GasCanvasProps {
5 |   source: HTMLVideoElement | HTMLImageElement | null;
6 |   width: number;
7 |   height: number;
8 |   detections: Detection[];
9 |   className?: string;
10 |   onReading?: (r: { ppm: number; level: "safe" | "warn" | "danger"; gas: string }) => void;
11 | }
12 |
13 | // Simulated gas-imaging camera:
14 | // - Converts the feed to a cool green phosphor look.
15 | // - Detects "gas-like" regions from inter-frame luminance flicker + low-saturation
16 | //   moving pixels, and paints a translucent plume overlay on top.
17 | // - Estimates a fake ppm reading from the plume area so the UI has something
18 | //   meaningful to display. This is a visualization, not a real sensor.
19 | const GASES = ["CH4 (Methane)", "CO (Carbon Monoxide)", "LPG", "NH3 (Ammonia)", "H2S"];
20 |
21 | export function GasCanvas({
22 |   source,
23 |   width,
24 |   height,
25 |   detections,
26 |   className,
27 |   onReading,
28 | }: GasCanvasProps) {
29 |   const canvasRef = useRef<HTMLCanvasElement>(null);
30 |   const offRef = useRef<HTMLCanvasElement | null>(null);
31 |   const prevRef = useRef<Float32Array | null>(null);
32 |   const rafRef = useRef<number | null>(null);
33 |   const gasIdxRef = useRef(0);
34 |   const tickRef = useRef(0);
35 |
36 |   useEffect(() => {
37 |     const canvas = canvasRef.current;
38 |     if (!canvas || !source || width === 0 || height === 0) return;
39 |     canvas.width = width;
40 |     canvas.height = height;
41 |     if (!offRef.current) offRef.current = document.createElement("canvas");
42 |     const off = offRef.current;
43 |     off.width = width;
44 |     off.height = height;
45 |     const offCtx = off.getContext("2d", { willReadFrequently: true });
46 |     const ctx = canvas.getContext("2d");
47 |
48 |     const draw = () => {
49 |       try {
50 |         offCtx.drawImage(source, 0, 0, width, height);
51 |         const img = offCtx.getImageData(0, 0, width, height);
52 |         const src = img.data;
53 |         const n = width * height;
54 |         const lum = new Float32Array(n);
55 |         for (let i = 0, p = 0; i < src.length; i += 4, p++) {
56 |           lum[p] = 0.299 * src[i] + 0.587 * src[i + 1] + 0.114 * src[i + 2];
57 |         }
58 |
59 |         // Green phosphor base image
60 |         const out = new Uint8ClampedArray(src.length);
61 |         for (let p = 0; p < n; p++) {
62 |           const v = lum[p];
63 |           const o = p * 4;
64 |           out[o] = v * 0.15;
65 |           out[o + 1] = Math.min(255, v * 1.05 + 15);
66 |           out[o + 2] = v * 0.35;
67 |           out[o + 3] = 255;
68 |         }
69 |
70 |         // Gas plume mask = inter-frame flicker on desaturated pixels.
71 |         const prev = prevRef.current;
72 |         let plumePixels = 0;
73 |         if (prev && prev.length === n) {
74 |           for (let p = 0; p < n; p++) {
75 |             const o = p * 4;
76 |             const r = src[o],
77 |                   g = src[o + 1],
78 |                   b = src[o + 2];
79 |             const maxC = Math.max(r, g, b);
80 |             const minC = Math.min(r, g, b);
81 |             const sat = maxC === 0 ? 0 : (maxC - minC) / maxC;
82 |             const diff = Math.abs(lum[p] - prev[p]);
83 |             if (diff > 6 && diff < 45 && sat < 0.35) {
84 |               plumePixels++;
85 |               // Blend a warm orange plume tint into the green base
86 |               out[o] = Math.min(255, out[o] + 180);
87 |               out[o + 1] = Math.min(255, out[o + 1] + 60);
88 |               out[o + 2] = Math.min(255, out[o + 2] + 10);
89 |             }
89 |           }
90 |         }
91 |       } catch {
92 |         // Ignore errors
93 |       }
94 |     };
95 |
96 |     draw();
97 |     rafRef.current = requestAnimationFrame(draw);
98 |   });
99 |
100 |   onReading?.({
101 |     ppm: plumePixels * 10,
102 |     level: plumePixels > 100 ? "danger" : plumePixels > 50 ? "warn" : "safe",
103 |     gas: GASES[gasIdxRef.current % GASES.length],
104 |   });
105 |   gasIdxRef.current++;
106 |   tickRef.current++;
107 | }
```

```

91 |     }
92 |     prevRef.current = lum;
93 |
94 |     ctx.putImageData(new ImageData(out, width, height), 0, 0);
95 |
96 |     // Crosshair grid overlay
97 |     ctx.strokeStyle = "rgba(120, 255, 160, 0.15)";
98 |     ctx.lineWidth = 1;
99 |     for (let gx = 0; gx < width; gx += Math.floor(width / 8)) {
100 |         ctx.beginPath();
101 |         ctx.moveTo(gx, 0);
102 |         ctx.lineTo(gx, height);
103 |         ctx.stroke();
104 |     }
105 |     for (let gy = 0; gy < height; gy += Math.floor(height / 6)) {
106 |         ctx.beginPath();
107 |         ctx.moveTo(0, gy);
108 |         ctx.lineTo(width, gy);
109 |         ctx.stroke();
110 |     }
111 |
112 |     // Person bounding boxes in green
113 |     ctx.font = `${Math.max(14, Math.round(height / 28))}px ui-monospace, monospace`;
114 |     ctx.lineWidth = 2;
115 |     detections.forEach((d, idx) => {
116 |         const [x, y, w, h] = d.bbox;
117 |         ctx.strokeStyle = "#7ffa0";
118 |         ctx.strokeRect(x, y, w, h);
119 |         const label = `SUBJECT #${idx + 1} ${(d.score * 100).toFixed(0)}%`;
120 |         const pad = 6;
121 |         const lh = Math.max(18, Math.round(height / 24));
122 |         const mw = ctx.measureText(label).width + pad * 2;
123 |         ctx.fillStyle = "rgba(0, 30, 15, 0.75)";
124 |         ctx.fillRect(x, Math.max(0, y - lh - 2), mw, lh + 2);
125 |         ctx.fillStyle = "#c8ffd8";
126 |         ctx.fillText(label, x + pad, Math.max(lh - 4, y - 6));
127 |     });
128 |
129 |     // Emit a synthetic ppm reading (~2 Hz).
130 |     tickRef.current++;
131 |     if (onReading && tickRef.current % 12 === 0) {
132 |         const density = plumePixels / n; // 0..1
133 |         const ppm = Math.round(density * 4200 + Math.random() * 20);
134 |         let level: "safe" | "warn" | "danger" = "safe";
135 |         if (ppm > 800) level = "danger";
136 |         else if (ppm > 250) level = "warn";
137 |         if (tickRef.current % 120 === 0) {
138 |             gasIdxRef.current = (gasIdxRef.current + 1) % GASES.length;
139 |         }
140 |         onReading({ ppm, level, gas: GASES[gasIdxRef.current] });
141 |     }
142 | } catch {
143 |     /* source not ready */
144 | }
145 | rafRef.current = requestAnimationFrame(draw);
146 | };
147 | draw();
148 | return () => {
149 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
150 |     prevRef.current = null;
151 | };
152 | }, [source, width, height, detections, onReading]);
153 |
154 | return <canvas ref={canvasRef} className={className} />;
155 | }

```

src/components/GasDetector.tsx

```
1 | import { describeCameraError } from "@lib/camera-error";
2 | import { useCallback, useEffect, useRef, useState } from "react";
3 | import { Button } from "@components/ui/button";
4 | import { Card } from "@components/ui/card";
5 | import { detectPeople, loadModel, type Detection } from "@lib/detector";
6 | import { GasCanvas } from "../GasCanvas";
7 | import { ConfidenceBars } from "../ConfidenceBars";
8 | import { LifeStatus } from "../LifeStatus";
9 | import {
10 |   LifeCalibration,
11 |   DEFAULTS_WEBCAM,
12 |   type LifeCalibrationValues,
13 | } from "../LifeCalibration";
14 | import { AlertTriangle, CameraOff, Loader2, Users, Wind } from "lucide-react";
15 |
16 | interface Reading {
17 |   ppm: number;
18 |   level: "safe" | "warn" | "danger";
19 |   gas: string;
20 | }
21 |
22 | export function GasDetector() {
23 |   const videoRef = useRef<HTMLVideoElement>(null);
24 |   const rafRef = useRef<number | null>(null);
25 |   const [active, setActive] = useState(false);
26 |   const [loading, setLoading] = useState(false);
27 |   const [error, setError] = useState<string | null>(null);
28 |   const [detections, setDetections] = useState<Detection[]>([]);
29 |   const [size, setSize] = useState({ w: 640, h: 480 });
30 |   const [reading, setReading] = useState<Reading>({
31 |     ppm: 0,
32 |     level: "safe",
33 |     gas: "CH4 (Methane)",
34 |   });
35 |   const [calibration, setCalibration] =
36 |     useState<LifeCalibrationValues>(DEFAULTS_WEBCAM);
37 |
38 |   const onReading = useCallback((r: Reading) => setReading(r), []);
39 |
40 |   const stop = () => {
41 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
42 |     rafRef.current = null;
43 |     const stream = videoRef.current?.srcObject as MediaStream | null;
44 |     stream?.getTracks().forEach((t) => t.stop());
45 |     if (videoRef.current) videoRef.current.srcObject = null;
46 |     setActive(false);
47 |     setDetections([]);
48 |   };
49 |
50 |   const start = async () => {
51 |     setError(null);
52 |     setLoading(true);
53 |     try {
54 |       await loadModel();
55 |       const stream = await navigator.mediaDevices.getUserMedia({
56 |         video: { facingMode: "user" },
57 |         audio: false,
58 |       });
59 |       const video = videoRef.current!;
60 |       video.srcObject = stream;
61 |       await video.play();
62 |       setSize({ w: video.videoWidth, h: video.videoHeight });
63 |       setActive(true);
64 |       const loop = async () => {
65 |         if (!videoRef.current) return;
66 |         try {
67 |           const found = await detectPeople(videoRef.current);
68 |           setDetections(found);
69 |         } catch {
70 |           /* ignore */
71 |         }
72 |         rafRef.current = requestAnimationFrame(loop);
73 |       };
74 |       loop();
75 |     } catch (e) {
76 |       setError(describeCameraError(e));
77 |     } finally {
78 |       setLoading(false);
79 |     }
80 |   };
81 |
82 |   useEffect(() => () => stop(), []);
83 |
84 |   const levelStyles: Record<Reading["level"], string> = {
85 |     safe: "bg-emerald-500/20 border-emerald-400/40 text-emerald-100",
86 |     warn: "bg-amber-500/25 border-amber-400/50 text-amber-100",
87 |     danger: "bg-red-500/30 border-red-400/60 text-red-100 animate-pulse",
88 |   };
89 |
90 |   return (
```

```

91 | <>
92 | <Card className="overflow-hidden">
93 |   <div className="relative bg-black aspect-video flex items-center justify-center">
94 |     <video
95 |       ref={videoRef}
96 |       playsInline
97 |       muted
98 |       className="absolute inset-0 w-full h-full object-contain opacity-0 pointer-events-none"
99 |     />
100 |     {active && (
101 |       <GasCanvas
102 |         source={videoRef.current}
103 |         width={size.w}
104 |         height={size.h}
105 |         detections={detections}
106 |         onReading={onReading}
107 |         className="w-full h-full object-contain"
108 |       />
109 |     )}
110 |     {!active && (
111 |       <div className="text-center text-muted-foreground p-8">
112 |         <Wind className="w-12 h-12 mx-auto mb-3 opacity-50" />
113 |         <p className="text-sm">Gas detection camera is off</p>
114 |       </div>
115 |     )}
116 |     {active && (
117 |       <>
118 |         <div className="absolute top-3 left-3 flex items-center gap-2 px-3 py-1.5 rounded-full bg-emerald-500/20
border border-emerald-400/40 text-emerald-100 backdrop-blur text-sm font-medium shadow">
119 |           <Users className="w-4 h-4" />
120 |           {detections.length}>{" "}
121 |           {detections.length === 1 ? "subject" : "subjects"}
122 |         </div>
123 |         <div
124 |           className={`absolute top-3 right-3 flex items-center gap-2 px-3 py-1.5 rounded-full border backdrop-bl
ur text-xs font-mono shadow ${levelStyles[reading.level]}`>
125 |           >
126 |             {reading.level === "danger" ? (
127 |               <AlertTriangle className="w-3.5 h-3.5" />
128 |             ) : (
129 |               <Wind className="w-3.5 h-3.5" />
130 |             )}
131 |             {reading.gas} · {reading.ppm} ppm
132 |           </div>
133 |           {reading.level === "danger" && (
134 |             <div className="absolute bottom-3 left-1/2 -translate-x-1/2 px-4 py-2 rounded-lg bg-red-600/80 border
border-red-300 text-white text-sm font-semibold shadow-lg backdrop-blur flex items-center gap-2">
135 |               <AlertTriangle className="w-4 h-4" /> Hazardous gas level detected – evacuate area
136 |             </div>
137 |           )}
138 |         </>
139 |       </div>
140 |     </div>
141 |     <div className="p-4 flex items-center justify-between gap-3">
142 |       <p className="text-sm text-muted-foreground">
143 |         {error ??
144 |           (active
145 |             ? "Simulated gas imaging · flicker + desaturation plume detection"
146 |             : "Start the gas camera to visualize simulated gas plumes and ppm levels")}
147 |       </p>
148 |       {active ? (
149 |         <Button onClick={stop} variant="secondary">
150 |           <CameraOff className="w-4 h-4" /> Stop
151 |         </Button>
152 |       ) : (
153 |         <Button onClick={start} disabled={loading}>
154 |           {loading ? (
155 |             <Loader2 className="w-4 h-4 animate-spin" />
156 |           ) : (
157 |             <Wind className="w-4 h-4" />
158 |           )}
159 |           {loading ? "Loading model..." : "Start gas camera"}
160 |         </Button>
161 |       )}
162 |     </div>
163 |   </Card>
164 |   {active && (
165 |     <LifeStatus
166 |       source={videoRef.current}
167 |       detections={detections}
168 |       active={active}
169 |       calibration={calibration}
170 |     />
171 |   )}
172 |   {active && (
173 |     <LifeCalibration
174 |       label="Gas"
175 |       values={calibration}
176 |       defaults={DEFAULTS_WEBCAM}
177 |       onChange={setCalibration}
178 |     />
179 |   )}
180 |   {active && <ConfidenceBars detections={detections} />}

```

```
181 |      </>
182 |    );
183 | }
```

src/components/GradientBuilder.tsx

```
1 | import { useState } from "react";
2 | import { Plus, Trash2, Sparkles } from "lucide-react";
3 | import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger, DialogFooter } from "@components/ui/dialog"
4 | ;
5 | import { Button } from "@components/ui/button";
6 | import { Slider } from "@components/ui/slider";
7 | import { Input } from "@components/ui/input";
8 | import { useSettings, buildGradientCss, type GradientStop } from "@lib/settings";
9 |
10 | export function GradientBuilder() {
11 |   const { custom, setCustom, setGradient } = useSettings();
12 |   const [open, setOpen] = useState(false);
13 |   const [draft, setDraft] = useState(custom);
14 |
15 |   const preview = buildGradientCss(draft);
16 |
17 |   const updateStop = (i: number, patch: Partial<GradientStop>) => {
18 |     setDraft({ ...draft, stops: draft.stops.map((s, idx) => (idx === i ? { ...s, ...patch } : s)) });
19 |   };
20 |   const addStop = () => {
21 |     if (draft.stops.length >= 6) return;
22 |     const last = draft.stops[draft.stops.length - 1];
23 |     setDraft({ ...draft, stops: [...draft.stops, { color: last.color, pos: Math.min(100, last.pos + 20) }] });
24 |   };
25 |   const removeStop = (i: number) => {
26 |     if (draft.stops.length <= 2) return;
27 |     setDraft({ ...draft, stops: draft.stops.filter((_, idx) => idx !== i) });
28 |   };
29 |   const apply = () => {
30 |     setCustom(draft);
31 |     setGradient("custom");
32 |     setOpen(false);
33 |   };
34 |
35 |   return (
36 |     <Dialog open={open} onOpenChange={(o) => { setOpen(o); if (o) setDraft(custom); }}>
37 |       <DialogTrigger asChild>
38 |         <Button variant="outline" size="sm" className="gap-2 w-full">
39 |           <Sparkles className="w-3.5 h-3.5" /> Custom builder...
40 |         </Button>
41 |       </DialogTrigger>
42 |       <DialogContent className="max-w-md">
43 |         <DialogHeader>
44 |           <DialogTitle>Custom gradient</DialogTitle>
45 |         </DialogHeader>
46 |
47 |         <div
48 |           className="h-20 rounded-lg border shadow-inner"
49 |           style={{ backgroundImage: preview }}
50 |         />
51 |
52 |         <div className="space-y-2">
53 |           <div className="flex items-center justify-between">
54 |             <label className="text-xs font-medium">Angle</label>
55 |             <span className="text-xs tabular-nums text-muted-foreground">{draft.angle}°</span>
56 |           </div>
57 |           <Slider
58 |             value={[draft.angle]}
59 |             min={0}
60 |             max={360}
61 |             step={1}
62 |             onValueChange={(v) => setDraft({ ...draft, angle: v[0] })}
63 |           />
64 |         </div>
65 |
66 |         <div className="space-y-3">
67 |           <div className="flex items-center justify-between">
68 |             <label className="text-xs font-medium">Color stops</label>
69 |             <Button variant="ghost" size="sm" onClick={addStop} disabled={draft.stops.length >= 6} className="h-7 gap-1">
70 |               <Plus className="w-3.5 h-3.5" /> Add
71 |             </Button>
72 |           </div>
73 |           <div className="space-y-2 max-h-64 overflow-y-auto pr-1">
74 |             {draft.stops.map((s, i) => (
75 |               <div key={i} className="flex items-center gap-2">
76 |                 <input
77 |                   type="color"
78 |                   value={s.color}
79 |                   onChange={(e) => updateStop(i, { color: e.target.value })}
80 |                   className="w-9 h-9 rounded-md border cursor-pointer bg-transparent"
81 |                   aria-label={`Stop ${i + 1} color`}
82 |                 />
83 |                 <Input
84 |                   type="text"
85 |                   value={s.color}
86 |                   onChange={(e) => updateStop(i, { color: e.target.value })}
87 |                   className="h-9 w-24 font-mono text-xs"
88 |                 />
89 |               </div>
90 |             ))}
91 |           </div>
92 |         </div>
93 |       </DialogContent>
94 |     </Dialog>
95 |   );
96 | }
```

```

89 |         <Slider
90 |             value={{s.pos}}
91 |             min={0}
92 |             max={100}
93 |             step={1}
94 |             onChange={(v) => updateStop(i, { pos: v[0] })}
95 |         />
96 |     </div>
97 |     <span className="text-xs tabular-nums w-9 text-right text-muted-foreground">{s.pos}%</span>
98 |     <Button
99 |         variant="ghost"
100 |         size="icon"
101 |         className="h-8 w-8 shrink-0"
102 |         onClick={() => removeStop(i)}
103 |         disabled={draft.stops.length <= 2}
104 |         aria-label="Remove stop"
105 |     >
106 |         <Trash2 className="w-3.5 h-3.5" />
107 |     </Button>
108 | </div>
109 |     )}
110 | </div>
111 | </div>
112 |
113 |     <DialogFooter>
114 |         <Button variant="ghost" onClick={() => setOpen(false)}>Cancel</Button>
115 |         <Button onClick={apply}>Apply</Button>
116 |     </DialogFooter>
117 | </DialogContent>
118 | </Dialog>
119 | );
120 | }

```

src/components/HudOverlay.tsx

```
1 | import { useEffect, useState } from "react";
2 | import { Activity, Cpu, Radio, Satellite, Signal } from "lucide-react";
3 |
4 | export function HudOverlay() {
5 |   const [time, setTime] = useState<Date | null>(null);
6 |   useEffect(() => {
7 |     setTime(new Date());
8 |     const id = setInterval(() => setTime(new Date()), 1000);
9 |     return () => clearInterval(id);
10 |   }, []);
11 |
12 |   const t = time ? time.toISOString().replace("T", " ").slice(0, 19) : "---:--:--";
13 |
14 |   return (
15 |     <
16 |       /* Cyber grid background */
17 |       <div className="absolute inset-0 cyber-grid opacity-60" aria-hidden />
18 |       /* Scanlines */
19 |       <div className="absolute inset-0 scanlines opacity-40" aria-hidden />
20 |       /* Sweep line */
21 |       <div className="absolute inset-x-0 top-0 h-[2px] bg-gradient-to-r from-transparent via-primary to-transparent animate-hud-sweep" aria-hidden />
22 |
23 |       /* Top-left status */
24 |       <div className="absolute top-4 left-4 flex items-center gap-2 text-[10px] font-mono uppercase tracking-widest text-primary/90 bg-background/60 backdrop-blur px-2 py-1 rounded border border-primary/30 z-10">
25 |         <span className="w-1.5 h-1.5 rounded-full bg-primary animate-hud-pulse" />
26 |         SYS · ONLINE
27 |       </div>
28 |
29 |       /* Top-right telemetry */
30 |       <div className="absolute top-4 right-4 hidden sm:flex items-center gap-3 text-[10px] font-mono text-primary/80 bg-background/60 backdrop-blur px-2 py-1 rounded border border-primary/30 z-10">
31 |         <span className="flex items-center gap-1"><Satellite className="w-3 h-3" /> GPS 08</span>
32 |         <span className="flex items-center gap-1"><Signal className="w-3 h-3" /> -62dBm</span>
33 |         <span className="flex items-center gap-1"><Cpu className="w-3 h-3" /> 42%</span>
34 |       </div>
35 |
36 |       /* Bottom-right clock */
37 |       <div className="absolute bottom-4 right-4 flex items-center gap-2 text-[10px] font-mono text-primary/80 bg-background/60 backdrop-blur px-2 py-1 rounded border border-primary/30 z-10">
38 |         <Activity className="w-3 h-3" /> {t} UTC
39 |       </div>
40 |
41 |       /* Bottom-left mission tag */
42 |       <div className="absolute bottom-4 left-4 text-[10px] font-mono text-primary/80 bg-background/60 backdrop-blur px-2 py-1 rounded border border-primary/30 z-10 flex items-center gap-2">
43 |         <Radio className="w-3 h-3 animate-hud-pulse" /> MISSION · CSSR-07
44 |       </div>
45 |     </>
46 |   );
47 | }
```

src/components/HyperspectralCanvas.tsx

```
1 | import { useEffect, useRef } from "react";
2 | import type { Detection } from "@lib/detector";
3 |
4 | export interface HyperspectralCanvasProps {
5 |   source: HTMLVideoElement | HTMLImageElement | null;
6 |   width: number;
7 |   height: number;
8 |   detections: Detection[];
9 |   band: number; // 0..7 selected band
10 |   className?: string;
11 |   onSignatures?: (sigs: number[][]) => void;
12 | }
13 |
14 | // 8 simulated spectral bands from ~450nm (violet) to ~950nm (NIR).
15 | // Each band is synthesized as a weighted combination of RGB channels + noise,
16 | // approximating how a hyperspectral sensor would separate wavelengths that a
17 | // consumer RGB camera fuses together.
18 | export const BANDS: Array<{
19 |   label: string;
20 |   nm: number;
21 |   weights: [number, number, number];
22 |   tint: [number, number, number];
23 | }> = [
24 |   { label: "B1", nm: 450, weights: [-0.1, 0.15, 0.95], tint: [90, 60, 220] },
25 |   { label: "B2", nm: 500, weights: [-0.05, 0.55, 0.55], tint: [40, 160, 210] },
26 |   { label: "B3", nm: 550, weights: [0.1, 0.9, 0.05], tint: [80, 220, 120] },
27 |   { label: "B4", nm: 620, weights: [0.55, 0.55, -0.05], tint: [220, 210, 60] },
28 |   { label: "B5", nm: 680, weights: [0.9, 0.15, -0.1], tint: [230, 120, 60] },
29 |   { label: "B6", nm: 740, weights: [0.75, 0.3, -0.05], tint: [220, 80, 80] },
30 |   { label: "B7", nm: 850, weights: [0.55, 0.4, 0.1], tint: [200, 100, 160] },
31 |   { label: "B8", nm: 950, weights: [0.4, 0.35, 0.25], tint: [160, 110, 210] },
32 | ];
33 |
34 | export function HyperspectralCanvas({
35 |   source,
36 |   width,
37 |   height,
38 |   detections,
39 |   band,
40 |   className,
41 |   onSignatures,
42 | }: HyperspectralCanvasProps) {
43 |   const canvasRef = useRef<HTMLCanvasElement>(null);
44 |   const offRef = useRef<HTMLCanvasElement | null>(null);
45 |   const rafRef = useRef<number | null>(null);
46 |   const lastEmitRef = useRef(0);
47 |
48 |   useEffect(() => {
49 |     const canvas = canvasRef.current;
50 |     if (!canvas || !source || width === 0 || height === 0) return;
51 |     canvas.width = width;
52 |     canvas.height = height;
53 |     if (!offRef.current) offRef.current = document.createElement("canvas");
54 |     const off = offRef.current;
55 |     off.width = width;
56 |     off.height = height;
57 |     const offCtx = off.getContext("2d", { willReadFrequently: true });
58 |     const ctx = canvas.getContext("2d");
59 |
60 |     const draw = () => {
61 |       try {
62 |         offCtx.drawImage(source, 0, 0, width, height);
63 |         const img = offCtx.getImageData(0, 0, width, height);
64 |         const data = img.data;
65 |         const bandDef = BANDS[Math.max(0, Math.min(BANDS.length - 1, band))];
66 |         const [wr, wg, wb] = bandDef.weights;
67 |         const [tr, tg, tb] = bandDef.tint;
68 |
69 |         for (let i = 0; i < data.length; i += 4) {
70 |           const r = data[i] / 255;
71 |           const g = data[i + 1] / 255;
72 |           const b = data[i + 2] / 255;
73 |           let v = r * wr + g * wg + b * wb;
74 |           // Simulated NIR: bright vegetation and skin (high red+green) glow
75 |           if (bandDef.nm >= 740) {
76 |             const nir = 0.5 * r + 0.35 * g + 0.15 * (1 - b);
77 |             v = v * 0.4 + nir * 0.7;
78 |           }
79 |           v = Math.max(0, Math.min(1, v + (Math.random() - 0.5) * 0.03));
80 |           data[i] = v * tr;
81 |           data[i + 1] = v * tg;
82 |           data[i + 2] = v * tb;
83 |         }
84 |         ctx.putImageData(img, 0, 0);
85 |
86 |         // Per-detection spectral signature (mean band-response across face region)
87 |         const sigs: number[][] = [];
88 |         ctx.font = `${Math.max(12, Math.round(height / 32))}px ui-monospace, monospace`;
89 |         ctx.lineWidth = 2;
90 |         detections.forEach((d, idx) => {
```

```

91 |     const [x, y, w, h] = d.bbox;
92 |     const fx = Math.max(0, Math.round(x + w * 0.2));
93 |     const fy = Math.max(0, Math.round(y + h * 0.05));
94 |     const fw = Math.max(1, Math.round(w * 0.6));
95 |     const fh = Math.max(1, Math.round(h * 0.3));
96 |     const sig: number[] = [];
97 |     try {
98 |       const face = offCtx.getImageData(fx, fy, fw, fh).data;
99 |       for (const bDef of BANDS) {
100 |         let sum = 0;
101 |         let count = 0;
102 |         const [br, bg, bb] = bDef.weights;
103 |         for (let i = 0; i < face.length; i += 32) {
104 |           const rr = face[i] / 255;
105 |           const gg = face[i + 1] / 255;
106 |           const bbv = face[i + 2] / 255;
107 |           sum += Math.max(0, rr * br + gg * bg + bbv * bb);
108 |           count++;
109 |         }
110 |         sig.push(count ? sum / count : 0);
111 |       }
112 |     } catch {
113 |       /* noop */
114 |     }
115 |     sigs.push(sig);
116 |
117 |     // Vegetation-like NDVI proxy from bands B4(red) and B7(NIR)
118 |     const red = sig[4] ?? 0;
119 |     const nir = sig[6] ?? 0;
120 |     const ndvi = (nir + red) > 0 ? (nir - red) / (nir + red) : 0;
121 |     const isSkin = ndvi > -0.05 && ndvi < 0.25 && red > 0.15;
122 |
123 |     ctx.strokeStyle = isSkin ? "#7cff9a" : "#ffd166";
124 |     ctx.strokeRect(x, y, w, h);
125 |     const label = `#${idx + 1} ${bandDef.label} ${bandDef.nm} NDVI ${ndvi.toFixed(2)}${isSkin ? " · skin" : ""}`;
126 |
127 |     const padding = 5;
128 |     const metrics = ctx.measureText(label);
129 |     const lh = Math.max(16, Math.round(height / 28));
130 |     ctx.fillStyle = "rgba(0,0,0,0.72)";
131 |     ctx.fillRect(x, Math.max(0, y - lh - 2), metrics.width + padding * 2, lh + 2);
132 |     ctx.fillStyle = "#fff";
133 |     ctx.fillText(label, x + padding, Math.max(lh - 4, y - 6));
134 |   });
135 |
136 |   const now = performance.now();
137 |   if (onSignatures && now - lastEmitRef.current > 300) {
138 |     lastEmitRef.current = now;
139 |     onSignatures(sigs);
140 |   }
141 | } catch {
142 |   /* not ready */
143 | }
144 | rafRef.current = requestAnimationFrame(draw);
145 | draw();
146 | return () => {
147 |   if (rafRef.current) cancelAnimationFrame(rafRef.current);
148 | };
149 | }, [source, width, height, detections, band, onSignatures]);
150 |
151 | return <canvas ref={canvasRef} className={className} />;
152 | }

```

src/components/HyperspectralDetector.tsx

```
1 | import { describeCameraError } from "@lib/camera-error";
2 | import { useEffect, useRef, useState } from "react";
3 | import { Button } from "@components/ui/button";
4 | import { Card } from "@components/ui/card";
5 | import { detectPeople, loadModel, type Detection } from "@lib/detector";
6 | import { HyperspectralCanvas, BANDS } from "../HyperspectralCanvas";
7 | import { ConfidenceBars } from "../ConfidenceBars";
8 | import { LifeStatus } from "../LifeStatus";
9 | import { LifeCalibration, DEFAULTS_WEBCAM, type LifeCalibrationValues } from "../LifeCalibration";
10 | import { CameraOff, Loader2, Layers, Users } from "lucide-react";
11 |
12 | export function HyperspectralDetector() {
13 |   const videoRef = useRef<HTMLVideoElement>(null);
14 |   const rafRef = useRef<number | null>(null);
15 |   const [active, setActive] = useState(false);
16 |   const [loading, setLoading] = useState(false);
17 |   const [error, setError] = useState<string | null>(null);
18 |   const [detections, setDetections] = useState<Detection[]>([]);
19 |   const [size, setSize] = useState({ w: 640, h: 480 });
20 |   const [band, setBand] = useState(4); // start on red (~680nm)
21 |   const [sigs, setSigs] = useState<number[][]>([]);
22 |   const [calibration, setCalibration] = useState<LifeCalibrationValues>(DEFAULTS_WEBCAM);
23 |
24 |   const stop = () => {
25 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
26 |     rafRef.current = null;
27 |     const stream = videoRef.current?.srcObject as MediaStream | null;
28 |     stream?.getTracks().forEach((t) => t.stop());
29 |     if (videoRef.current) videoRef.current.srcObject = null;
30 |     setActive(false);
31 |     setDetections([]);
32 |     setSigs([]);
33 |   };
34 |
35 |   const start = async () => {
36 |     setError(null);
37 |     setLoading(true);
38 |     try {
39 |       await loadModel();
40 |       const stream = await navigator.mediaDevices.getUserMedia({
41 |         video: { facingMode: "user" },
42 |         audio: false,
43 |       });
44 |       const video = videoRef.current!;
45 |       video.srcObject = stream;
46 |       await video.play();
47 |       setSize({ w: video.videoWidth, h: video.videoHeight });
48 |       setActive(true);
49 |       const loop = async () => {
50 |         if (!videoRef.current) return;
51 |         try {
52 |           const found = await detectPeople(videoRef.current);
53 |           setDetections(found);
54 |         } catch {
55 |           /* ignore */
56 |         }
57 |         rafRef.current = requestAnimationFrame(loop);
58 |       };
59 |       loop();
60 |     } catch (e) {
61 |       setError(describeCameraError(e));
62 |     } finally {
63 |       setLoading(false);
64 |     }
65 |   };
66 |
67 |   useEffect(() => () => stop(), []);
68 |
69 |   const bandDef = BANDS[band];
70 |   const primarySig = sigs[0];
71 |   const maxSig = primarySig ? Math.max(...primarySig, 0.001) : 1;
72 |
73 |   return (
74 |     <
75 |       <Card className="overflow-hidden">
76 |         <div className="relative bg-black aspect-video flex items-center justify-center">
77 |           <video
78 |             ref={videoRef}
79 |             playsInline
80 |             muted
81 |             className="absolute inset-0 w-full h-full object-contain opacity-0 pointer-events-none"
82 |           />
83 |           {active && (
84 |             <HyperspectralCanvas
85 |               source={videoRef.current}
86 |               width={size.w}
87 |               height={size.h}
88 |               detections={detections}
89 |               band={band}
90 |               onSignatures={setSigs}
```

```

91 |         className="w-full h-full object-contain"
92 |     />
93 | }
94 | {!active && (
95 |     <div className="text-center text-muted-foreground p-8">
96 |         <Layers className="w-12 h-12 mx-auto mb-3 opacity-50" />
97 |         <p className="text-sm">Hyperspectral camera is off</p>
98 |         <p className="text-xs mt-1 opacity-70">
99 |             8 simulated bands · 450 – 950 nm (VIS + NIR)
100 |         </p>
101 |     </div>
102 | )}
103 | {active && (
104 |     <
105 |         <div className="absolute top-3 left-3 flex items-center gap-2 px-3 py-1.5 rounded-full bg-background/90
backdrop-blur text-sm font-medium shadow">
106 |             <Users className="w-4 h-4 text-primary" />
107 |             {detections.length} {detections.length === 1 ? "person" : "people"}
108 |         </div>
109 |         <div
110 |             className="absolute top-3 right-3 flex items-center gap-2 px-3 py-1.5 rounded-full backdrop-blur text-
xs font-mono font-semibold shadow text-white"
111 |             style={{
112 |                 background: `rgba(${bandDef.tint[0]},${bandDef.tint[1]},${bandDef.tint[2]},0.85)` ,
113 |             }}
114 |             >
115 |                 <Layers className="w-3.5 h-3.5" />
116 |                 {bandDef.label} · {bandDef.nm} nm
117 |             </div>
118 |         </>
119 |     )}
120 | </div>
121 | <div className="p-4 space-y-4">
122 |     <div className="flex items-center justify-between gap-3">
123 |         <p className="text-sm text-muted-foreground">
124 |             {error ??
125 |                 (active
126 |                     ? "Hyperspectral mode · pick a spectral band to inspect"
127 |                     : "Start the hyperspectral camera (VIS + near-IR simulation)")
128 |             }
129 |         </p>
130 |         {active ? (
131 |             <Button onClick={stop} variant="secondary">
132 |                 <CameraOff className="w-4 h-4" /> Stop
133 |             </Button>
134 |         ) : (
135 |             <Button onClick={start} disabled={loading}>
136 |                 {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : <Layers className="w-4 h-4" />}
137 |                 {loading ? "Loading model..." : "Start hyperspectral"}
138 |             </Button>
139 |         )}
140 |     </div>
141 |     {active && (
142 |         <div>
143 |             <div className="flex items-center justify-between text-xs text-muted-foreground mb-2">
144 |                 <span>Spectral bands</span>
145 |                 <span className="font-mono">{bandDef.nm} nm</span>
146 |             </div>
147 |             <div className="grid grid-cols-8 gap-1">
148 |                 {BANDS.map((b, i) => {
149 |                     const selected = i === band;
150 |                     return (
151 |                         <button
152 |                             key={b.label}
153 |                             onClick={() => setBand(i)}
154 |                             className={`text-[10px] font-mono py-2 rounded-md transition-all ${
155 |                                 selected ? "ring-2 ring-white/80 scale-105" : "opacity-70 hover:opacity-100"
156 |                             }`}
157 |                             style={{
158 |                                 background: `rgb(${b.tint[0]},${b.tint[1]},${b.tint[2]})`,
159 |                                 color: "white",
160 |                                 textShadow: "0 1px 2px rgba(0,0,0,0.5)",
161 |                             }}
162 |                             title={` ${b.label} – ${b.nm} nm`}
163 |                         >
164 |                             {b.nm}
165 |                         </button>
166 |                     );
167 |                 })}
168 |             </div>
169 |         </div>
170 |     )}
171 |
172 | {active && primarySig && (
173 |     <div>
174 |         <p className="text-xs text-muted-foreground mb-2">
175 |             Subject #1 spectral signature
176 |         </p>
177 |         <div className="flex items-end gap-1 h-16">
178 |             {primarySig.map((v, i) => {
179 |                 const b = BANDS[i];
180 |                 const h = Math.max(2, (v / maxSig) * 100);
181 |                 return (

```

```

182 |         <div key={i} className="flex-1 flex flex-col items-center gap-1">
183 |             <div
184 |                 className="w-full rounded-t"
185 |                 style={{
186 |                     height: `${h}%`,
187 |                     background: `rgb(${b.tint[0]},${b.tint[1]},${b.tint[2]})`,
188 |                 }}
189 |             />
190 |             <span className="text-[9px] font-mono text-muted-foreground">
191 |                 {b.label}
192 |             </span>
193 |         </div>
194 |     );
195 | }}
196 | </div>
197 | </div>
198 | })
199 | </div>
200 | </Card>
201 | {active && (
202 |     <LifeStatus
203 |         source={videoRef.current}
204 |         detections={detections}
205 |         active={active}
206 |         calibration={calibration}
207 |     />
208 | )}
209 | {active && (
210 |     <LifeCalibration
211 |         label="Hyperspectral"
212 |         values={calibration}
213 |         defaults={DEFAULTS_WEBCAM}
214 |         onChange={setCalibration}
215 |     />
216 | )}
217 | {active && <ConfidenceBars detections={detections} />}
218 | </>
219 | );
220 | }

```

src/components/ImageDetector.tsx

```
1 | import { useRef, useState } from "react";
2 | import { Button } from "@components/ui/button";
3 | import { Card } from "@components/ui/card";
4 | import { detectPeople, type Detection } from "@lib/detector";
5 | import { DetectionCanvas } from "../DetectionCanvas";
6 | import { ConfidenceBars } from "../ConfidenceBars";
7 | import { LifeStatus } from "../LifeStatus";
8 | import { LifeCalibration, DEFAULTS_IMAGE, type LifeCalibrationValues } from "../LifeCalibration";
9 | import { ImageIcon, Loader2, Upload, Users } from "lucide-react";
10 |
11 | export function ImageDetector() {
12 |   const inputRef = useRef<HTMLInputElement>(null);
13 |   const imgRef = useRef<HTMLImageElement>(null);
14 |   const [src, setSrc] = useState<string | null>(null);
15 |   const [size, setSize] = useState({ w: 0, h: 0 });
16 |   const [detections, setDetections] = useState<Detection[]>([]);
17 |   const [loading, setLoading] = useState(false);
18 |   const [error, setError] = useState<string | null>(null);
19 |   const [analyzed, setAnalyzed] = useState(false);
20 |   const [calibration, setCalibration] = useState<LifeCalibrationValues>(DEFAULTS_IMAGE);
21 |
22 |   const handleFile = (file: File) => {
23 |     setError(null);
24 |     setAnalyzed(false);
25 |     setDetections([]);
26 |     const url = URL.createObjectURL(file);
27 |     setSrc(url);
28 |   };
29 |
30 |   const onImgLoad = async (e: React.SyntheticEvent<HTMLImageElement>) => {
31 |     const img = e.currentTarget;
32 |     setSize({ w: img.naturalWidth, h: img.naturalHeight });
33 |     setLoading(true);
34 |     try {
35 |       const found = await detectPeople(img);
36 |       setDetections(found);
37 |       setAnalyzed(true);
38 |     } catch (err) {
39 |       setError(err instanceof Error ? err.message : "Detection failed");
40 |     } finally {
41 |       setLoading(false);
42 |     }
43 |   };
44 |
45 |   return (
46 |     <Card className="overflow-hidden">
47 |       <div
48 |         className="relative bg-muted aspect-video flex items-center justify-center"
49 |         onDragOver={(e) => e.preventDefault()}
50 |         onDrop={(e) => {
51 |           e.preventDefault();
52 |           const f = e.dataTransfer.files?.[0];
53 |           if (f && f.type.startsWith("image/")) handleFile(f);
54 |         }}
55 |       >
56 |         {src ? (
57 |           <img
58 |             ref={imgRef}
59 |             src={src}
60 |             alt="Uploaded for detection"
61 |             onLoad={onImgLoad}
62 |             crossOrigin="anonymous"
63 |             className="w-full h-full object-contain"
64 |           />
65 |           {size.w > 0 && (
66 |             <div className="absolute inset-0 flex items-center justify-center">
67 |               <DetectionCanvas
68 |                 width={size.w}
69 |                 height={size.h}
70 |                 detections={detections}
71 |                 className="w-full h-full object-contain"
72 |               />
73 |             )}
74 |           )}
75 |         )}
76 |         {analyzed && (
77 |           <div className="absolute top-3 left-3 flex items-center gap-2 px-3 py-1.5 rounded-full bg-background/90 backdrop-blur text-sm font-medium shadow">
78 |             <Users className="w-4 h-4 text-primary" />
79 |             {detections.length} {detections.length === 1 ? "person" : "people"} detected
80 |           </div>
81 |         )}
82 |         {loading && (
83 |           <div className="absolute inset-0 flex items-center justify-center bg-background/60 backdrop-blur-sm">
84 |             <Loader2 className="w-8 h-8 animate-spin text-primary" />
85 |           </div>
86 |         )}
87 |       </div>
88 |     ) : (
89 |
```

```

90 |         <div className="text-center text-muted-foreground p-8">
91 |             <ImageIcon className="w-12 h-12 mx-auto mb-3 opacity-50" />
92 |             <p className="text-sm">Drop an image here or click upload</p>
93 |         </div>
94 |     )}
95 | </div>
96 | <div className="p-4 flex items-center justify-between gap-3">
97 |     <p className="text-sm text-muted-foreground">
98 |         {error ?? (analyzed ? "Analysis complete" : "Upload a photo to scan for people")}
99 |     </p>
100 |     <input
101 |         ref={inputRef}
102 |         type="file"
103 |         accept="image/*"
104 |         className="hidden"
105 |         onChange={(e) => {
106 |             const f = e.target.files?.[0];
107 |             if (f) handleFile(f);
108 |         }}
109 |     />
110 |     <Button onClick={() => inputRef.current?.click()} variant={src ? "secondary" : "default"}>
111 |         <Upload className="w-4 h-4" />
112 |         {src ? "Choose another" : "Upload image"}
113 |     </Button>
114 | </div>
115 | </Card>
116 | {analyzed && (
117 |     <LifeStatus
118 |         source={imgRef.current}
119 |         detections={detections}
120 |         active={analyzed}
121 |         calibration={calibration}
122 |         still
123 |     />
124 | )}
125 | {analyzed && (
126 |     <LifeCalibration
127 |         label="Uploaded image"
128 |         values={calibration}
129 |         defaults={DEFAULTS_IMAGE}
130 |         onChange={setCalibration}
131 |         stillOnly
132 |     />
133 | )}
134 | {analyzed && <ConfidenceBars detections={detections} />}
135 | </>
136 | );
137 | }

```

src/components/LifeCalibration.tsx

```
1 | import { useState } from "react";
2 | import { Card } from "@components/ui/card";
3 | import { Button } from "@components/ui/button";
4 | import { Slider } from "@components/ui/slider";
5 | import { Label } from "@components/ui/label";
6 | import { ChevronDown, ChevronUp, RotateCcw, SlidersHorizontal } from "lucide-react";
7 |
8 | export interface LifeCalibrationValues {
9 |   /** Seconds of sustained eye-closure required before declaring deceased. */
10 |   closedSeconds: number;
11 |   /** Peak motion at/above which a person is clearly alive (0..1). */
12 |   motionAlive: number;
13 |   /** Peak motion below which a person is clearly still (0..1). */
14 |   motionDead: number;
15 |   /** Absolute floor on the eye-band luminance variance for "eyes open". */
16 |   eyeVarianceMin: number;
17 | }
18 |
19 | interface Props {
20 |   label: string;
21 |   values: LifeCalibrationValues;
22 |   defaults: LifeCalibrationValues;
23 |   onChange: (v: LifeCalibrationValues) => void;
24 |   /** Hide motion controls for still-image mode. */
25 |   stillOnly?: boolean;
26 | }
27 |
28 | export function LifeCalibration({ label, values, defaults, onChange, stillOnly }: Props) {
29 |   const [open, setOpen] = useState(false);
30 |   const set = (patch: Partial<LifeCalibrationValues>) =>
31 |     onChange({ ...values, ...patch });
32 |
33 |   return (
34 |     <Card className="mt-3 p-3">
35 |       <div className="flex items-center justify-between">
36 |         <button
37 |           type="button"
38 |           onClick={() => setOpen((o) => !o)}
39 |           className="flex items-center gap-2 text-sm font-medium hover:text-primary transition-colors"
40 |         >
41 |           <SlidersHorizontal className="w-4 h-4" />
42 |           Calibration · {label}
43 |           {open ? <ChevronUp className="w-4 h-4" /> : <ChevronDown className="w-4 h-4" />}
44 |         </button>
45 |         {open && (
46 |           <Button
47 |             type="button"
48 |             variant="ghost"
49 |             size="sm"
50 |             onClick={() => onChange(defaults)}
51 |             className="h-7 gap-1 text-xs"
52 |           >
53 |             <RotateCcw className="w-3 h-3" /> Reset
54 |           </Button>
55 |         )}
56 |       </div>
57 |       {open && (
58 |         <div className="grid gap-4 mt-3 sm:grid-cols-2">
59 |           {!stillOnly && (
60 |             <div className="space-y-1">
61 |               <Label className="text-xs flex justify-between">
62 |                 <span>Eye-closure to confirm deceased</span>
63 |                 <span className="font-mono text-muted-foreground">
64 |                   {values.closedSeconds.toFixed(1)}s
65 |                 </span>
66 |               </Label>
67 |               <Slider
68 |                 min={1}
69 |                 max={15}
70 |                 step={0.5}
71 |                 value={[values.closedSeconds]}
72 |                 onValueChange={([v]) => set({ closedSeconds: v })}
73 |               />
74 |             </div>
75 |           )}
76 |           {!stillOnly && (
77 |             <div className="space-y-1">
78 |               <Label className="text-xs flex justify-between">
79 |                 <span>Alive motion threshold</span>
80 |                 <span className="font-mono text-muted-foreground">
81 |                   {(values.motionAlive * 1000).toFixed(1)}
82 |                 </span>
83 |               </Label>
84 |               <Slider
85 |                 min={1}
86 |                 max={40}
87 |                 step={1}
88 |                 value={[values.motionAlive * 1000]}
89 |                 onValueChange={([v]) => set({ motionAlive: v / 1000 })}
90 |               />
91 |             </div>
92 |           )}
93 |         </div>
94 |       )}
95 |     </Card>
96 |   );
97 | }
```

```

91 |         </div>
92 |     })
93 |     {!stillOnly && (
94 |         <div className="space-y-1">
95 |             <Label className="text-xs flex justify-between">
96 |                 <span>Still motion threshold</span>
97 |                 <span className="font-mono text-muted-foreground">
98 |                     {(values.motionDead * 1000).toFixed(1)}
99 |                 </span>
100 |             </Label>
101 |             <Slider
102 |                 min={0}
103 |                 max={20}
104 |                 step={0.5}
105 |                 value={[values.motionDead * 1000]}
106 |                 onChange={([v]) => set({ motionDead: v / 1000 })}
107 |             />
108 |         </div>
109 |     )}
110 |     <div className="space-y-1">
111 |         <Label className="text-xs flex justify-between">
112 |             <span>Eye-open sensitivity (variance floor)</span>
113 |             <span className="font-mono text-muted-foreground">
114 |                 {Math.round(values.eyeVarianceMin)}
115 |             </span>
116 |         </Label>
117 |         <Slider
118 |             min={50}
119 |             max={800}
120 |             step={10}
121 |             value={[values.eyeVarianceMin]}
122 |             onChange={([v]) => set({ eyeVarianceMin: v })}
123 |         />
124 |     </div>
125 | </div>
126 | )}
127 | </Card>
128 | );
129 | }
130 |
131 | export const DEFAULTS_WEBCAM: LifeCalibrationValues = {
132 |     closedSeconds: 4,
133 |     motionAlive: 0.01,
134 |     motionDead: 0.004,
135 |     eyeVarianceMin: 180,
136 | };
137 |
138 | export const DEFAULTS_THERMAL: LifeCalibrationValues = {
139 |     closedSeconds: 5,
140 |     motionAlive: 0.008,
141 |     motionDead: 0.003,
142 |     eyeVarianceMin: 120,
143 | };
144 |
145 | export const DEFAULTS_IMAGE: LifeCalibrationValues = {
146 |     closedSeconds: 4,
147 |     motionAlive: 0.01,
148 |     motionDead: 0.004,
149 |     eyeVarianceMin: 350,
150 | };

```

src/components/LifeStatus.tsx

```
1 | import { useEffect, useRef, useState } from "react";
2 | import type { Detection } from "@lib/detector";
3 | import { Activity, HeartPulse, Skull, Eye, EyeOff } from "lucide-react";
4 |
5 | export type LifeState = "alive" | "deceased" | "checking";
6 |
7 | export interface PersonLife {
8 |   state: LifeState;
9 |   motion: number; // 0..1
10 |   eyesOpen: boolean | null;
11 | }
12 |
13 | export interface LifeCalibration {
14 |   closedSeconds: number;
15 |   motionAlive: number;
16 |   motionDead: number;
17 |   eyeVarianceMin: number;
18 | }
19 |
20 | interface Props {
21 |   source: HTMLVideoElement | HTMLImageElement | null;
22 |   detections: Detection[];
23 |   /** If true, treat source as a single still image (no motion possible). */
24 |   still?: boolean;
25 |   active: boolean;
26 |   calibration?: LifeCalibration;
27 | }
28 |
29 | // Sample the eye-band of each detection bbox and compare to previous frame.
30 | // Returns an array of per-detection life status.
31 | interface PersonTrack {
32 |   motionHist: number[]; // luminance diff per sample (0..1)
33 |   varHist: number[]; // eye-band luminance variance
34 |   closedStreak: number; // consecutive samples with eyes-closed
35 |   openStreak: number; // consecutive samples with eyes-open
36 |   lastAliveAt: number; // tick index when we last saw life
37 |   state: LifeState;
38 |   samples: number;
39 | }
40 |
41 | // Sampling cadence (~6 Hz).
42 | const TICK_MS = 160;
43 | const TICKS_PER_SEC = 1000 / TICK_MS;
44 | // Windows expressed in samples.
45 | const WIN_MOTION = 18; // ~3s rolling window for motion
46 | const WIN_VAR = 30; // ~5s rolling window to learn an eye-variance baseline
47 | const OPEN_REQUIRED = 2; // brief consecutive opens count as "eyes open"
48 | const MIN_SAMPLES = 12; // ~2s warmup before we commit to any verdict
49 | const HOLD_ALIVE_TICKS = 12; // hysteresis: stay 'alive' for ~2s after last life sign
50 |
51 | export function LifeStatus({ source, detections, still = false, active, calibration }: Props) {
52 |   const CLOSED_REQUIRED = Math.max(
53 |     1,
54 |     Math.round((calibration?.closedSeconds ?? 4) * TICKS_PER_SEC),
55 |   );
56 |   const MOTION_ALIVE = calibration?.motionAlive ?? 0.01;
57 |   const MOTION_DEAD = calibration?.motionDead ?? 0.004;
58 |   const EYE_VAR_MIN = calibration?.eyeVarianceMin ?? 180;
59 |   const offRef = useRef<HTMLCanvasElement | null>(null);
60 |   const prevRef = useRef<Map<number, Uint8ClampedArray>>(new Map());
61 |   const trackRef = useRef<Map<number, PersonTrack>>(new Map());
62 |   const tickCountRef = useRef(0);
63 |   const rafRef = useRef<number | null>(null);
64 |   const [people, setPeople] = useState<PersonLife[]>([]);
65 |
66 |   useEffect(() => {
67 |     if (!active || !source) {
68 |       setPeople([]);
69 |       return;
70 |     }
71 |     if (!offRef.current) offRef.current = document.createElement("canvas");
72 |     const off = offRef.current;
73 |     const ctx = off.getContext("2d", { willReadFrequently: true });
74 |
75 |     let lastTick = 0;
76 |
77 |     const tick = (t: number) => {
78 |       if (t - lastTick > TICK_MS) {
79 |         lastTick = t;
80 |         tickCountRef.current++;
81 |         const now = tickCountRef.current;
82 |         try {
83 |           const srcW =
84 |             (source as HTMLVideoElement).videoWidth ||
85 |             (source as HTMLImageElement).naturalWidth;
86 |           const srcH =
87 |             (source as HTMLVideoElement).videoHeight ||
88 |             (source as HTMLImageElement).naturalHeight;
89 |           if (srcW > 0 && srcH > 0) {
90 |             off.width = srcW;
```

```

91 |     off.height = srcH;
92 |     ctx.drawImage(source, 0, 0, srcW, srcH);
93 |
94 |     const next: PersonLife[] = detections.map((d, idx) => {
95 |         const [x, y, w, h] = d.bbox;
96 |         const ex = Math.max(0, Math.round(x + w * 0.2));
97 |         const ey = Math.max(0, Math.round(y + h * 0.18));
98 |         const ew = Math.max(4, Math.round(w * 0.6));
99 |         const eh = Math.max(4, Math.round(h * 0.2));
100 |
101 |         let frame: Uint8ClampedArray | null = null;
102 |         let variance = 0;
103 |         try {
104 |             const img = ctx.getImageData(ex, ey, ew, eh);
105 |             frame = img.data;
106 |             let sum = 0,
107 |                 sum2 = 0,
108 |                 n = 0;
109 |             for (let i = 0; i < frame.length; i += 16) {
110 |                 const lum =
111 |                     0.299 * frame[i] +
112 |                     0.587 * frame[i + 1] +
113 |                     0.114 * frame[i + 2];
114 |                 sum += lum;
115 |                 sum2 += lum * lum;
116 |                 n++;
117 |             }
118 |             const mean = sum / Math.max(1, n);
119 |             variance = sum2 / Math.max(1, n) - mean * mean;
120 |         } catch {
121 |             /* out of bounds */
122 |         }
123 |
124 |         // Motion vs prior frame, computed over the same eye band.
125 |         let motion = 0;
126 |         const prev = prevRef.current.get(idx);
127 |         if (frame && prev && prev.length === frame.length) {
128 |             let diff = 0,
129 |                 n = 0;
130 |             for (let i = 0; i < frame.length; i += 16) {
131 |                 const a =
132 |                     0.299 * frame[i] +
133 |                     0.587 * frame[i + 1] +
134 |                     0.114 * frame[i + 2];
135 |                 const b =
136 |                     0.299 * prev[i] + 0.587 * prev[i + 1] + 0.114 * prev[i + 2];
137 |                 diff += Math.abs(a - b);
138 |                 n++;
139 |             }
140 |             motion = diff / Math.max(1, n) / 255;
141 |         }
142 |         if (frame) prevRef.current.set(idx, new Uint8ClampedArray(frame));
143 |
144 |         const track: PersonTrack = trackRef.current.get(idx) ?? {
145 |             motionHist: [],
146 |             varHist: [],
147 |             closedStreak: 0,
148 |             openStreak: 0,
149 |             lastAliveAt: -Infinity,
150 |             state: "checking",
151 |             samples: 0,
152 |         };
153 |         track.samples++;
154 |         track.motionHist.push(motion);
155 |         if (track.motionHist.length > WIN_MOTION) track.motionHist.shift();
156 |         track.varHist.push(variance);
157 |         if (track.varHist.length > WIN_VAR) track.varHist.shift();
158 |
159 |         // Adaptive eye-open threshold: 55% of recent variance peak,
160 |         // floored at an absolute minimum so a blank/dark frame can't
161 |         // qualify as "open".
162 |         const varPeak = track.varHist.length
163 |             ? Math.max(...track.varHist)
164 |             : 0;
165 |         const eyeThreshold = Math.max(EYE_VAR_MIN, varPeak * 0.55);
166 |         const eyesOpen = variance >= eyeThreshold;
167 |         if (eyesOpen) {
168 |             track.openStreak++;
169 |             track.closedStreak = 0;
170 |         } else {
171 |             track.closedStreak++;
172 |             track.openStreak = 0;
173 |         }
174 |
175 |         const peakMotion = track.motionHist.length
176 |             ? Math.max(...track.motionHist)
177 |             : 0;
178 |         // Sort motion samples to get median (robust to single spikes).
179 |         const sorted = [...track.motionHist].sort((a, b) => a - b);
180 |         const medMotion =
181 |             sorted.length > 0 ? sorted[Math.floor(sorted.length / 2)] : 0;
182 |
183 |         const hasMotion = peakMotion > MOTION_ALIVE;

```

```

184 |     const isStill = peakMotion < MOTION_DEAD && medMotion < MOTION_DEAD;
185 |     const eyesOpenConfirmed = track.openStreak >= OPEN_REQUIRED;
186 |     const sustainedClosure = track.closedStreak >= CLOSED_REQUIRED;
187 |
188 |     if (hasMotion || eyesOpenConfirmed) {
189 |       track.lastAliveAt = now;
190 |     }
191 |     const sinceAlive = now - track.lastAliveAt;
192 |
193 |     let state: LifeState;
194 |     if (still) {
195 |       // Single image: only the eye-open heuristic is available.
196 |       state = eyesOpen ? "alive" : "deceased";
197 |     } else if (track.samples < MIN_SAMPLES) {
198 |       state = "checking";
199 |     } else if (
200 |       // Need BOTH: long eye-closure AND a still motion window
201 |       // AND no recent life signal – hysteresis prevents flicker.
202 |       sustainedClosure &&
203 |       isStill &&
204 |       sinceAlive > HOLD_ALIVE_TICKS
205 |     ) {
206 |       state = "deceased";
207 |     } else if (
208 |       hasMotion ||
209 |       eyesOpenConfirmed ||
210 |       sinceAlive <= HOLD_ALIVE_TICKS
211 |     ) {
212 |       state = "alive";
213 |     } else {
214 |       state = track.state === "deceased" ? "deceased" : "checking";
215 |     }
216 |
217 |     track.state = state;
218 |     trackRef.current.set(idx, track);
219 |     return { state, motion: peakMotion, eyesOpen };
220 |   });
221 |
222 |   // Prune stale tracks when detections shrink.
223 |   for (const k of trackRef.current.keys()) {
224 |     if (k >= detections.length) {
225 |       trackRef.current.delete(k);
226 |       prevRef.current.delete(k);
227 |     }
228 |   }
229 |
230 |   setPeople(next);
231 | }
232 | } catch {
233 |   /* ignore */
234 | }
235 | }
236 | if (!still) rafRef.current = requestAnimationFrame(tick);
237 | };
238 |
239 | if (still) {
240 |   tick(0);
241 |   tick(1000);
242 | } else {
243 |
244 |   rafRef.current = requestAnimationFrame(tick);
245 | }
246 |
247 | return () => {
248 |   if (rafRef.current) cancelAnimationFrame(rafRef.current);
249 |   prevRef.current.clear();
250 |   trackRef.current.clear();
251 | };
252 | }, [source, detections, still, active, CLOSED_REQUIRED, MOTION_ALIVE, MOTION_DEAD, EYE_VAR_MIN]);
253 |
254 | if (!active || detections.length === 0) return null;
255 |
256 | return (
257 |   <div className="mt-3 grid gap-2 sm:grid-cols-2">
258 |     {detections.map((_, idx) => {
259 |       const p = people[idx];
260 |       const state: LifeState = p?.state ?? "checking";
261 |       const alive = state === "alive";
262 |       const dead = state === "deceased";
263 |       const eyeSeen = p?.eyesOpen === true;
264 |       const eyeDetected = p?.eyesOpen !== null && p?.eyesOpen !== undefined;
265 |       const label = alive
266 |         ? eyeSeen
267 |           ? "EYE DETECTED · ALIVE"
268 |           : "VITALS DETECTED · ALIVE"
269 |         : dead
270 |           ? eyeDetected
271 |             ? "EYE DETECTED · NO RESPONSE · DECEASED"
272 |             : "NO EYE ACTIVITY · DECEASED"
273 |           : "SCANNING EYES...";
274 |       return (
275 |         <div
276 |           key={idx}

```

```

277 |         className={`flex items-center justify-between gap-3 px-3 py-2 rounded-lg border text-sm shadow-sm ${
278 |             alive
279 |             ? "bg-emerald-500/10 border-emerald-500/40 text-emerald-700 dark:text-emerald-300"
280 |             : dead
281 |             ? "bg-red-500/10 border-red-500/40 text-red-700 dark:text-red-300"
282 |             : "bg-muted border-border text-muted-foreground"
283 |         }}
284 |     >
285 |     <div className="flex items-center gap-2 font-medium">
286 |         {alive ? (
287 |             <HeartPulse className="w-4 h-4 animate-pulse" />
288 |         ) : dead ? (
289 |             <Skull className="w-4 h-4" />
290 |         ) : (
291 |             <Activity className="w-4 h-4 animate-pulse" />
292 |         )}
293 |         <span className="font-mono text-xs sm:text-sm tracking-wide">
294 |             #{idx + 1} · {label}
295 |         </span>
296 |     </div>
297 |     <div className="flex items-center gap-2 text-xs opacity-80">
298 |         {p?.eyesOpen ? (
299 |             <span className="flex items-center gap-1">
300 |                 <Eye className="w-3 h-3" /> eyes open
301 |             </span>
302 |         ) : p?.eyesOpen === false ? (
303 |             <span className="flex items-center gap-1">
304 |                 <EyeOff className="w-3 h-3" /> eyes closed
305 |             </span>
306 |         ) : null}
307 |         {!still && p && (
308 |             <span className="font-mono">
309 |                 motion {(p.motion * 100).toFixed(1)}
310 |             </span>
311 |         )}
312 |     </div>
313 | </div>
314 | );
315 | }}
316 | </div>
317 | );
318 | }

```

src/components/PratishaInfo.tsx

```
1 | import { Card } from "@components/ui/card";
2 | import {
3 |   Cpu,
4 |   Camera,
5 |   Bluetooth,
6 |   Radar,
7 |   Wind,
8 |   Sparkles,
9 |   Workflow,
10 | Target,
11 | Layers,
12 | Wallet,
13 | Rocket,
14 | Bug,
15 | } from "lucide-react";
16 | import { useSettings } from "@lib/settings";
17 | import { getContent } from "@lib/pratisha-content";
18 |
19 | function Section({
20 |   id,
21 |   icon: Icon,
22 |   eyebrow,
23 |   title,
24 |   children,
25 | }: {
26 |   id?: string;
27 |   icon: React.ComponentType<{ className?: string }>;
28 |   eyebrow?: string;
29 |   title: string;
30 |   children: React.ReactNode;
31 | }) {
32 |   return (
33 |     <section id={id} className="scroll-mt-24">
34 |       <div className="flex items-center gap-3 mb-4">
35 |         <div className="w-10 h-10 rounded-xl bg-primary/10 flex items-center justify-center">
36 |           <Icon className="w-5 h-5 text-primary" />
37 |         </div>
38 |         <div>
39 |           {eyebrow && (
40 |             <p className="text-xs uppercase tracking-widest text-muted-foreground">
41 |               {eyebrow}
42 |             </p>
43 |           )}
44 |           <h2 className="text-2xl font-semibold tracking-tight">{title}</h2>
45 |         </div>
46 |       </div>
47 |       <div className="text-sm leading-relaxed text-muted-foreground space-y-3">
48 |         {children}
49 |       </div>
50 |     </section>
51 |   );
52 | }
53 |
54 | function FeatureCard({
55 |   icon: Icon,
56 |   title,
57 |   children,
58 | }: {
59 |   icon: React.ComponentType<{ className?: string }>;
60 |   title: string;
61 |   children: React.ReactNode;
62 | }) {
63 |   return (
64 |     <Card className="p-5 space-y-2">
65 |       <div className="flex items-center gap-2">
66 |         <Icon className="w-4 h-4 text-primary" />
67 |         <h3 className="font-semibold text-foreground">{title}</h3>
68 |       </div>
69 |       <p className="text-sm text-muted-foreground leading-relaxed">{children}</p>
70 |     </Card>
71 |   );
72 | }
73 |
74 | const COMPONENT_ICONS = [Cpu, Camera, Bluetooth, Radar, Wind, Layers];
75 | const WORKFLOW_ICONS = [Bluetooth, Workflow, Radar, Camera];
76 | const MATERIAL_ICONS = [Workflow, Layers, Target, Camera];
77 | const FUTURE_ICONS = [Sparkles, Radar, Bluetooth, Target, Camera, Cpu];
78 |
79 | export function PratishaInfo() {
80 |   const { lang } = useSettings();
81 |   const c = getContent(lang.code);
82 |
83 |   return (
84 |     <div className="space-y-16 mt-16">
85 |       <Section id="about" icon={Sparkles} eyebrow={c.about.eyebrow} title={c.about.title}>
86 |         <p>{c.about.p1}</p>
87 |         <p>{c.about.p2}</p>
88 |         <p>{c.about.p3}</p>
89 |       </Section>
90 |     </div>
91 |   );
92 | }
```

```

91 | <Section id="components" icon={Cpu} eyebrow={c.components.eyebrow} title={c.components.title}>
92 |   <p>{c.components.intro}</p>
93 |   <div className="grid sm:grid-cols-2 gap-4 pt-2">
94 |     {c.components.cards.map((card, i) => (
95 |       <FeatureCard key={i} icon={COMPONENT_ICONS[i]} title={card.title}>
96 |         {card.body}
97 |       </FeatureCard>
98 |     ))}
99 |   </div>
100 | </Section>
101 |
102 | <Section id="features" icon={Sparkles} eyebrow={c.features.eyebrow} title={c.features.title}>
103 |   <ul className="space-y-3 list-disc pl-5">
104 |     {c.features.items.map((it, i) => (
105 |       <li key={i}>
106 |         <strong className="text-foreground">{it.label}</strong> {it.body}
107 |       </li>
108 |     ))}
109 |   </ul>
110 | </Section>
111 |
112 | <Section id="workflow" icon={Workflow} eyebrow={c.workflow.eyebrow} title={c.workflow.title}>
113 |   <div className="grid sm:grid-cols-2 gap-4">
114 |     {c.workflow.cards.map((card, i) => (
115 |       <FeatureCard key={i} icon={WORKFLOW_ICONS[i]} title={card.title}>
116 |         {card.body}
117 |       </FeatureCard>
118 |     ))}
119 |   </div>
120 | </Section>
121 |
122 | <Section id="applications" icon={Target} eyebrow={c.applications.eyebrow} title={c.applications.title}>
123 |   <div className="space-y-4">
124 |     {c.applications.items.map((it, i) => (
125 |       <div key={i}>
126 |         <h3 className="font-semibold text-foreground">{it.title}</h3>
127 |         <p>{it.body}</p>
128 |       </div>
129 |     ))}
130 |   </div>
131 | </Section>
132 |
133 | <Section id="materials" icon={Layers} eyebrow={c.materials.eyebrow} title={c.materials.title}>
134 |   <p>{c.materials.intro}</p>
135 |   <div className="grid sm:grid-cols-2 gap-4 pt-2">
136 |     {c.materials.cards.map((card, i) => (
137 |       <FeatureCard key={i} icon={MATERIAL_ICONS[i]} title={card.title}>
138 |         {card.body}
139 |       </FeatureCard>
140 |     ))}
141 |   </div>
142 | </Section>
143 |
144 | <Section id="cost" icon={Wallet} eyebrow={c.cost.eyebrow} title={c.cost.title}>
145 |   <h3 className="font-semibold text-foreground">{c.cost.prototypeHeading}</h3>
146 |   <Card className="p-5">
147 |     <table className="w-full text-sm">
148 |       <tbody className="divide-y">
149 |         {c.cost.rows.map((label, i) => (
150 |           <tr key={i} className="grid grid-cols-2 py-2">
151 |             <td>{label}</td>
152 |             <td className="text-right text-foreground">
153 |               {["₹500 – ₹1,000", "₹250 – ₹500", "₹300 – ₹600", "₹150 – ₹300", "₹200 – ₹400", "₹300 – ₹500", "₹20
0 – ₹500"]}[i]}
154 |             </td>
155 |           </tr>
156 |         ))}
157 |         <tr className="grid grid-cols-2 py-2 font-semibold">
158 |           <td className="text-foreground">{c.cost.totalLabel}</td>
159 |           <td className="text-right text-primary">₹2,500 – ₹5,500</td>
160 |         </tr>
161 |       </tbody>
162 |     </table>
163 |   </Card>
164 |   <h3 className="font-semibold text-foreground pt-4">{c.cost.productionHeading}</h3>
165 |   <p>{c.cost.productionPara}</p>
166 |   <div className="grid sm:grid-cols-2 gap-4 pt-2">
167 |     <Card className="p-5">
168 |       <p className="text-xs uppercase tracking-widest text-muted-foreground">{c.cost.lowEnd}</p>
169 |       <p className="text-foreground font-semibold mt-1">₹12,45,000 – ₹20,75,000</p>
170 |       <p className="text-xs">USD 150,000 – 250,000</p>
171 |     </Card>
172 |     <Card className="p-5">
173 |       <p className="text-xs uppercase tracking-widest text-muted-foreground">{c.cost.highEnd}</p>
174 |       <p className="text-foreground font-semibold mt-1">₹29,05,000 – ₹49,80,000</p>
175 |       <p className="text-xs">USD 350,000 – 600,000</p>
176 |     </Card>
177 |   </div>
178 | </Section>
179 |
180 | <Section id="future" icon={Rocket} eyebrow={c.future.eyebrow} title={c.future.title}>
181 |   <div className="grid sm:grid-cols-2 gap-4">
182 |     {c.future.cards.map((card, i) => (

```

```

183 |         <FeatureCard key={i} icon={FUTURE_ICONS[i]} title={card.title}>
184 |             {card.body}
185 |         </FeatureCard>
186 |     )}}
187 | </div>
188 | </Section>
189 |
190 |     <Section id="crab" icon={Bug} eyebrow={c.crab.eyebrow} title={c.crab.title}>
191 |         <p>{c.crab.p1}</p>
192 |         <p>{c.crab.p2}</p>
193 |     </Section>
194 | </div>
195 | );
196 | }

```

src/components/SettingsBar.tsx

```
1 | import { Palette, Languages, Check, Zap, Waves, Pause } from "lucide-react";
2 | import {
3 |   DropdownMenu,
4 |   DropdownMenuContent,
5 |   DropdownMenuItem,
6 |   DropdownMenuLabel,
7 |   DropdownMenuTrigger,
8 |   DropdownMenuSeparator,
9 | } from "@components/ui/dropdown-menu";
10 | import { Button } from "@components/ui/button";
11 | import { useSettings, THEME_COLORS, LANGS, GRADIENTS, buildGradientCss, type MotionMode } from "@lib/settings";
12 | import { GradientBuilder } from "@components/GradientBuilder";
13 |
14 | const MOTION_ORDER: MotionMode[] = ["full", "calm", "off"];
15 | const MOTION_META: Record<MotionMode, { icon: typeof Zap; label: string; hint: string }> = {
16 |   full: { icon: Zap, label: "Motion: Full", hint: "All HUD animations at full intensity" },
17 |   calm: { icon: Waves, label: "Motion: Calm", hint: "Slower sweep, ticker & pulse" },
18 |   off: { icon: Pause, label: "Motion: Off", hint: "Static HUD, no animation" },
19 | };
20 |
21 | export function SettingsBar() {
22 |   const { color, setColor, gradient, setGradient, custom, lang, setLang, motion, setMotion, t } = useSettings();
23 |   const customPreview = buildGradientCss(custom);
24 |   const swatchFor = (id: string, preview: string) =>
25 |     id === "none" ? color.swatch : id === "custom" ? customPreview : preview;
26 |   const MotionIcon = MOTION_META[motion].icon;
27 |   const cycleMotion = () => setMotion(MOTION_ORDER[(MOTION_ORDER.indexOf(motion) + 1) % MOTION_ORDER.length]);
28 |
29 |   return (
30 |     <div className="flex items-center gap-1">
31 |       <Button
32 |         variant="ghost"
33 |         size="sm"
34 |         className="gap-1.5 h-8"
35 |         onClick={cycleMotion}
36 |         aria-label={MOTION_META[motion].label}
37 |         title={` ${MOTION_META[motion].label} – ${MOTION_META[motion].hint}. Click to cycle.`}
38 |       >
39 |         <MotionIcon className="w-4 h-4" />
40 |         <span className="text-[10px] uppercase tracking-widest hidden sm:inline">
41 |           {motion}
42 |         </span>
43 |       </Button>
44 |
45 |       <DropdownMenu>
46 |         <DropdownMenuTrigger asChild>
47 |           <Button variant="ghost" size="sm" className="gap-2 h-8" aria-label={t("theme")}>
48 |             <span
49 |               className="w-3.5 h-3.5 rounded-full ring-2 ring-offset-1 ring-offset-background"
50 |               style={{ background: color.swatch, boxShadow: `0 0 0 1px ${color.swatch}` }}
51 |             />
52 |             <Palette className="w-4 h-4" />
53 |           </Button>
54 |         </DropdownMenuTrigger>
55 |         <DropdownMenuContent align="end" className="w-56">
56 |           <DropdownMenuLabel>{t("theme")}</DropdownMenuLabel>
57 |           <DropdownMenuSeparator />
58 |           <div className="grid grid-cols-3 gap-2 p-2">
59 |             {THEME_COLORS.map((c) => (
60 |               <button
61 |                 key={c.id}
62 |                 onClick={() => setColor(c.id)}
63 |                 className="flex flex-col items-center gap-1 p-2 rounded-md hover:bg-accent transition-colors"
64 |                 title={c.label}
65 |               >
66 |                 <span
67 |                   className="w-6 h-6 rounded-full border"
68 |                   style={{ background: c.swatch }}
69 |                 />
70 |                 <span className="text-[10px] text-muted-foreground flex items-center gap-0.5">
71 |                   {c.id === color.id && <Check className="w-2.5 h-2.5" />}
72 |                   {c.label}
73 |                 </span>
74 |               </button>
75 |             ))}
76 |           </div>
77 |         </DropdownMenuContent>
78 |       </DropdownMenu>
79 |
80 |       <DropdownMenu>
81 |         <DropdownMenuTrigger asChild>
82 |           <Button variant="ghost" size="sm" className="gap-2 h-8" aria-label={t("gradient")}>
83 |             <span
84 |               className="w-5 h-3.5 rounded-sm border"
85 |               style={{ background: swatchFor(gradient.id, gradient.preview) }}
86 |             />
87 |           </Button>
88 |         </DropdownMenuTrigger>
89 |         <DropdownMenuContent align="end" className="w-60">
90 |           <DropdownMenuLabel>{t("gradient")}</DropdownMenuLabel>
```

```

91 |         <DropdownMenuSeparator />
92 |         <div className="grid grid-cols-3 gap-2 p-2">
93 |             {GRADIENTS.map((g) => (
94 |                 <button
95 |                     key={g.id}
96 |                     onClick={() => setGradient(g.id)}
97 |                     className="flex flex-col items-center gap-1 p-1.5 rounded-md hover:bg-accent transition-colors"
98 |                     title={g.label}
99 |                 >
100 |                     <span
101 |                         className="w-full h-6 rounded-md border"
102 |                         style={{ background: swatchFor(g.id, g.preview) }}
103 |                     />
104 |                     <span className="text-[10px] text-muted-foreground flex items-center gap-0.5">
105 |                         {g.id === gradient.id && <Check className="w-2.5 h-2.5" />}
106 |                         {g.label}
107 |                     </span>
108 |                 </button>
109 |             )}
110 |         </div>
111 |         <DropdownMenuSeparator />
112 |         <div className="p-2">
113 |             <GradientBuilder />
114 |         </div>
115 |     </DropdownMenuContent>
116 | </DropdownMenu>
117 |
118 | <DropdownMenu>
119 |     <DropdownMenuTrigger asChild>
120 |         <Button variant="ghost" size="sm" className="gap-2 h-8" aria-label={t("language")}>
121 |             <Languages className="w-4 h-4" />
122 |             <span className="text-xs hidden sm:inline">{lang.native}</span>
123 |         </Button>
124 |     </DropdownMenuTrigger>
125 |     <DropdownMenuContent align="end" className="w-56 max-h-80 overflow-y-auto">
126 |         <DropdownMenuLabel>{t("language")}</DropdownMenuLabel>
127 |         <DropdownMenuSeparator />
128 |         {LANGS.map((l) => (
129 |             <DropdownMenuItem
130 |                 key={l.code}
131 |                 onClick={() => setLang(l.code)}
132 |                 className="flex items-center justify-between gap-2"
133 |             >
134 |                 <span className="flex flex-col">
135 |                     <span className="text-sm">{l.native}</span>
136 |                     <span className="text-[10px] text-muted-foreground">{l.label}</span>
137 |                 </span>
138 |                 {l.code === lang.code && <Check className="w-4 h-4 text-primary" />}
139 |             </DropdownMenuItem>
140 |         )}
141 |     </DropdownMenuContent>
142 | </DropdownMenu>
143 | </div>
144 | );
145 | }

```

src/components/SystemTicker.tsx

```
1 | const FEED = [  
2 |   "● LIDAR MESH OK",  
3 |   "▲ CORE TEMP 41.2°C",  
4 |   "◆ IMU DRIFT 0.02°/s",  
5 |   "● BATTERY 78% · 42m",  
6 |   "▼ THERMAL ARRAY LOCK",  
7 |   "● COMMS 2.4GHz MESH",  
8 |   "▲ CO2 412 ppm",  
9 |   "◆ NEURAL NET v3.14",  
10 |  "● SLAM POSE 6-DOF",  
11 |  "▼ TELEMETRY UPLINK",  
12 |  "● SERVO TORQUE NOMINAL",  
13 |  "▲ RUBBLE MAP: 87%",  
14 | ];  
15 |  
16 | export function SystemTicker() {  
17 |   const items = [...FEED, ...FEED];  
18 |   return (  
19 |     <div className="relative overflow-hidden border-y border-primary/25 bg-background/60 backdrop-blur">  
20 |       <div className="flex whitespace-nowrap animate-ticker py-2 font-mono text-[11px] uppercase tracking-widest text-  
    primary/80">  
21 |         {items.map((s, i) => (  
22 |           <span key={i} className="px-6 shrink-0">  
23 |             {s}  
24 |           </span>  
25 |         ))}  
26 |       </div>  
27 |     </div>  
28 |   );  
29 | }
```

src/components/ThermalCanvas.tsx

```
1 | import { useEffect, useRef } from "react";
2 | import type { Detection } from "@lib/detector";
3 |
4 | // Inferno-ish colormap: maps 0..1 luminance to thermal RGB
5 | function thermalColor(t: number): [number, number, number] {
6 |   t = Math.max(0, Math.min(1, t));
7 |   // Piecewise gradient: black -> purple -> red -> orange -> yellow -> white
8 |   const stops: Array<number, [number, number, number]> = [
9 |     [0.0, [0, 0, 4]],
10 |    [0.2, [40, 11, 84]],
11 |    [0.4, [101, 21, 110]],
12 |    [0.6, [187, 55, 84]],
13 |    [0.75, [237, 104, 37]],
14 |    [0.9, [251, 180, 26]],
15 |    [1.0, [252, 255, 164]],
16 |  ];
17 |   for (let i = 0; i < stops.length - 1; i++) {
18 |     const [a, ca] = stops[i];
19 |     const [b, cb] = stops[i + 1];
20 |     if (t >= a && t <= b) {
21 |       const k = (t - a) / (b - a);
22 |       return [
23 |         ca[0] + (cb[0] - ca[0]) * k,
24 |         ca[1] + (cb[1] - ca[1]) * k,
25 |         ca[2] + (cb[2] - ca[2]) * k,
26 |       ];
27 |     }
28 |   }
29 |   return [255, 255, 255];
30 | }
31 |
32 | export interface ThermalCanvasProps {
33 |   source: HTMLVideoElement | HTMLImageElement | null;
34 |   width: number;
35 |   height: number;
36 |   detections: Detection[];
37 |   className?: string;
38 |   onTemps?: (temps: number[]) => void;
39 | }
40 |
41 | // Map normalized luminance (sampled from the head area) to an approximate
42 | // body-surface temperature in Celsius. Real thermal cameras measure IR, but
43 | // in this simulation we use brightness as a proxy: brighter face -> warmer.
44 | function lumToCelsius(avgLum: number): number {
45 |   // Skin in normal light ~ 0.45..0.85 luminance. Map to 35.5..38.2 C.
46 |   const clamped = Math.max(0.2, Math.min(0.95, avgLum));
47 |   const t = (clamped - 0.2) / (0.95 - 0.2);
48 |   const base = 35.5 + t * 2.7;
49 |   // Small jitter for realism
50 |   return Math.round((base + (Math.random() - 0.5) * 0.2) * 10) / 10;
51 | }
52 |
53 | export function ThermalCanvas({
54 |   source,
55 |   width,
56 |   height,
57 |   detections,
58 |   className,
59 |   onTemps,
60 | }: ThermalCanvasProps) {
61 |   const canvasRef = useRef<HTMLCanvasElement>(null);
62 |   const offRef = useRef<HTMLCanvasElement | null>(null);
63 |   const rafRef = useRef<number | null>(null);
64 |   const tempsRef = useRef<number[]>([]);
65 |   const lastEmitRef = useRef(0);
66 |
67 |   useEffect(() => {
68 |     const canvas = canvasRef.current;
69 |     if (!canvas || !source || width === 0 || height === 0) return;
70 |     canvas.width = width;
71 |     canvas.height = height;
72 |     if (!offRef.current) offRef.current = document.createElement("canvas");
73 |     const off = offRef.current;
74 |     off.width = width;
75 |     off.height = height;
76 |     const offCtx = off.getContext("2d", { willReadFrequently: true });
77 |     const ctx = canvas.getContext("2d");
78 |
79 |     const draw = () => {
80 |       try {
81 |         offCtx.drawImage(source, 0, 0, width, height);
82 |         const img = offCtx.getImageData(0, 0, width, height);
83 |         const data = img.data;
84 |         for (let i = 0; i < data.length; i += 4) {
85 |           const r = data[i], g = data[i + 1], b = data[i + 2];
86 |           const lum = (0.299 * r + 0.587 * g + 0.114 * b) / 255;
87 |           const [tr, tg, tb] = thermalColor(lum);
88 |           data[i] = tr;
89 |           data[i + 1] = tg;
90 |           data[i + 2] = tb;
```

```

91 |     }
92 |     ctx.putImageData(img, 0, 0);
93 |
94 |     // Per-detection temperature estimation (sample face/upper region)
95 |     const temps: number[] = [];
96 |     ctx.font = `${Math.max(14, Math.round(height / 28))}px ui-sans-serif, system-ui`;
97 |     ctx.lineWidth = 3;
98 |     detections.forEach((d, idx) => {
99 |         const [x, y, w, h] = d.bbox;
100 |         // Upper ~25% = head/face region
101 |         const fx = Math.max(0, Math.round(x + w * 0.25));
102 |         const fy = Math.max(0, Math.round(y));
103 |         const fw = Math.max(1, Math.round(w * 0.5));
104 |         const fh = Math.max(1, Math.round(h * 0.25));
105 |         let sum = 0;
106 |         let count = 0;
107 |         try {
108 |             const face = offCtx.getImageData(fx, fy, fw, fh).data;
109 |             for (let i = 0; i < face.length; i += 16) {
110 |                 const lum =
111 |                     (0.299 * face[i] + 0.587 * face[i + 1] + 0.114 * face[i + 2]) /
112 |                     255;
113 |                 sum += lum;
114 |                 count++;
115 |             }
116 |         } catch {
117 |             /* out of bounds */
118 |         }
119 |         const avg = count > 0 ? sum / count : 0.5;
120 |         const temp = lumToCelsius(avg);
121 |         temps.push(temp);
122 |
123 |         // Bounding box + label
124 |         const fever = temp >= 37.8;
125 |         ctx.strokeStyle = fever ? "#ff3b30" : "#00e5ff";
126 |         ctx.strokeRect(x, y, w, h);
127 |         const tempF = temp * 9 / 5 + 32;
128 |         const label = `${idx + 1} ${temp.toFixed(1)}°C / ${tempF.toFixed(1)}°F${fever ? " FEVER" : ""}`;
129 |         const padding = 6;
130 |         const metrics = ctx.measureText(label);
131 |         const lh = Math.max(18, Math.round(height / 24));
132 |         ctx.fillStyle = fever ? "rgba(255,59,48,0.92)" : "rgba(0,0,0,0.7)";
133 |         ctx.fillRect(x, Math.max(0, y - lh - 2), metrics.width + padding * 2, lh + 2);
134 |         ctx.fillStyle = "#fff";
135 |         ctx.fillText(label, x + padding, Math.max(lh - 4, y - 6));
136 |     });
137 |
138 |     tempsRef.current = temps;
139 |     const now = performance.now();
140 |     if (onTemps && now - lastEmitRef.current > 300) {
141 |         lastEmitRef.current = now;
142 |         onTemps(temps);
143 |     }
144 | } catch {
145 |     /* source not ready */
146 | }
147 | rafRef.current = requestAnimationFrame(draw);
148 | };
149 | draw();
150 | return () => {
151 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
152 | };
153 | }, [source, width, height, detections, onTemps]);
154 |
155 | return <canvas ref={canvasRef} className={className} />;
156 | }

```

src/components/ThermalDetector.tsx

```
1 | import { describeCameraError } from "@lib/camera-error";
2 | import { useEffect, useRef, useState } from "react";
3 | import { Button } from "@components/ui/button";
4 | import { Card } from "@components/ui/card";
5 | import { detectPeople, loadModel, type Detection } from "@lib/detector";
6 | import { ThermalCanvas } from "../ThermalCanvas";
7 | import { ConfidenceBars } from "../ConfidenceBars";
8 | import { LifeStatus } from "../LifeStatus";
9 | import { LifeCalibration, DEFAULTS_THERMAL, type LifeCalibrationValues } from "../LifeCalibration";
10 | import { CameraOff, Loader2, Thermometer, Users } from "lucide-react";
11 |
12 | export function ThermalDetector() {
13 |   const videoRef = useRef<HTMLVideoElement>(null);
14 |   const rafRef = useRef<number | null>(null);
15 |   const [active, setActive] = useState(false);
16 |   const [loading, setLoading] = useState(false);
17 |   const [error, setError] = useState<string | null>(null);
18 |   const [detections, setDetections] = useState<Detection[]>([]);
19 |   const [size, setSize] = useState({ w: 640, h: 480 });
20 |   const [temps, setTemps] = useState<number[]>([]);
21 |   const [calibration, setCalibration] = useState<LifeCalibrationValues>(DEFAULTS_THERMAL);
22 |
23 |   const stop = () => {
24 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
25 |     rafRef.current = null;
26 |     const stream = videoRef.current?.srcObject as MediaStream | null;
27 |     stream?.getTracks().forEach((t) => t.stop());
28 |     if (videoRef.current) videoRef.current.srcObject = null;
29 |     setActive(false);
30 |     setDetections([]);
31 |     setTemps([]);
32 |   };
33 |
34 |   const start = async () => {
35 |     setError(null);
36 |     setLoading(true);
37 |     try {
38 |       await loadModel();
39 |       const stream = await navigator.mediaDevices.getUserMedia({
40 |         video: { facingMode: "user" },
41 |         audio: false,
42 |       });
43 |       const video = videoRef.current!;
44 |       video.srcObject = stream;
45 |       await video.play();
46 |       setSize({ w: video.videoWidth, h: video.videoHeight });
47 |       setActive(true);
48 |       const loop = async () => {
49 |         if (!videoRef.current) return;
50 |         try {
51 |           const found = await detectPeople(videoRef.current);
52 |           setDetections(found);
53 |         } catch {
54 |           /* ignore */
55 |         }
56 |         rafRef.current = requestAnimationFrame(loop);
57 |       };
58 |       loop();
59 |     } catch (e) {
60 |       setError(describeCameraError(e));
61 |     } finally {
62 |       setLoading(false);
63 |     }
64 |   };
65 |
66 |   useEffect(() => () => stop(), []);
67 |
68 |   const maxTemp = temps.length ? Math.max(...temps) : null;
69 |   const maxTempF = maxTemp !== null ? maxTemp * 9 / 5 + 32 : null;
70 |   const fever = maxTemp !== null && maxTemp >= 37.8;
71 |
72 |   return (
73 |     <>
74 |       <Card className="overflow-hidden">
75 |         <div className="relative bg-black aspect-video flex items-center justify-center">
76 |           <video
77 |             ref={videoRef}
78 |             playsInline
79 |             muted
80 |             className="absolute inset-0 w-full h-full object-contain opacity-0 pointer-events-none"
81 |           />
82 |           {active && (
83 |             <ThermalCanvas
84 |               source={videoRef.current}
85 |               width={size.w}
86 |               height={size.h}
87 |               detections={detections}
88 |               onTemps={setTemps}
89 |               className="w-full h-full object-contain"
90 |             />

```

```

91 |     })
92 |     {!active && (
93 |       <div className="text-center text-muted-foreground p-8">
94 |         <Thermometer className="w-12 h-12 mx-auto mb-3 opacity-50" />
95 |         <p className="text-sm">Thermal camera is off</p>
96 |       </div>
97 |     })
98 |     {active && (
99 |       <
100 |         <div className="absolute top-3 left-3 flex items-center gap-2 px-3 py-1.5 rounded-full bg-background/90
backdrop-blur text-sm font-medium shadow">
101 |           <Users className="w-4 h-4 text-primary" />
102 |           {detections.length} {detections.length === 1 ? "person" : "people"}
103 |         </div>
104 |         {maxTemp !== null && maxTempF !== null && (
105 |           <div
106 |             className={`absolute top-3 right-3 flex items-center gap-2 px-3 py-1.5 rounded-full backdrop-blur te
xt-sm font-semibold shadow ${
107 |               fever
108 |                 ? "bg-red-500/90 text-white"
109 |                 : "bg-background/90 text-foreground"
110 |             }`}
111 |           >
112 |             <Thermometer className="w-4 h-4" />
113 |             {maxTemp.toFixed(1)}°C / {maxTempF.toFixed(1)}°F {fever ? "· Fever" : ""}
114 |           </div>
115 |         )}
116 |         <div className="absolute bottom-3 left-3 right-3 h-2 rounded-full overflow-hidden shadow"
117 |           style={{
118 |             background:
119 |               "linear-gradient(90deg, rgb(0,0,4), rgb(40,11,84), rgb(101,21,110), rgb(187,55,84), rgb(237,104,37
), rgb(251,180,26), rgb(252,255,164))",
120 |           }}
121 |         />
122 |         <div className="absolute bottom-6 left-3 right-3 flex justify-between text-[10px] text-white/80 font-mon
o">
123 |           <span>cold</span>
124 |           <span>~36°C / 96.8°F</span>
125 |           <span>~37°C / 98.6°F</span>
126 |           <span>hot</span>
127 |         </div>
128 |       </>
129 |     )}
130 |   </div>
131 |   <div className="p-4 flex items-center justify-between gap-3">
132 |     <p className="text-sm text-muted-foreground">
133 |       {error ??
134 |         (active
135 |           ? "Thermal mode · body temperature is an approximation"
136 |           : "Start the thermal camera to estimate body temperature (°C & °F)")}
137 |     </p>
138 |     {active ? (
139 |       <Button onClick={stop} variant="secondary">
140 |         <CameraOff className="w-4 h-4" /> Stop
141 |       </Button>
142 |     ) : (
143 |       <Button onClick={start} disabled={loading}>
144 |         {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : <Thermometer className="w-4 h-4" />}
145 |         {loading ? "Loading model..." : "Start thermal camera"}
146 |       </Button>
147 |     )}
148 |   </div>
149 | </Card>
150 |   {active && (
151 |     <LifeStatus
152 |       source={videoRef.current}
153 |       detections={detections}
154 |       active={active}
155 |       calibration={calibration}
156 |     />
157 |   )}
158 |   {active && (
159 |     <LifeCalibration
160 |       label="Thermal"
161 |       values={calibration}
162 |       defaults={DEFAULTS_THERMAL}
163 |       onChange={setCalibration}
164 |     />
165 |   )}
166 |   {active && <ConfidenceBars detections={detections} />}
167 | </>
168 | );
169 | }

```

src/components/WebcamDetector.tsx

```
1 | import { describeCameraError } from "@lib/camera-error";
2 | import { useEffect, useRef, useState } from "react";
3 | import { Button } from "@components/ui/button";
4 | import { Card } from "@components/ui/card";
5 | import { detectPeople, loadModel, type Detection } from "@lib/detector";
6 | import { DetectionCanvas } from "../DetectionCanvas";
7 | import { ConfidenceBars } from "../ConfidenceBars";
8 | import { LifeStatus } from "../LifeStatus";
9 | import { LifeCalibration, DEFAULTS_WEBCAM, type LifeCalibrationValues } from "../LifeCalibration";
10 | import { Camera, CameraOff, Loader2, Users } from "lucide-react";
11 |
12 | export function WebcamDetector() {
13 |   const videoRef = useRef<HTMLVideoElement>(null);
14 |   const rafRef = useRef<number | null>(null);
15 |   const [active, setActive] = useState(false);
16 |   const [loading, setLoading] = useState(false);
17 |   const [error, setError] = useState<string | null>(null);
18 |   const [detections, setDetections] = useState<Detection[]>([]);
19 |   const [size, setSize] = useState({ w: 640, h: 480 });
20 |   const [calibration, setCalibration] = useState<LifeCalibrationValues>(DEFAULTS_WEBCAM);
21 |
22 |   const stop = () => {
23 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
24 |     rafRef.current = null;
25 |     const stream = videoRef.current?.srcObject as MediaStream | null;
26 |     stream?.getTracks().forEach((t) => t.stop());
27 |     if (videoRef.current) videoRef.current.srcObject = null;
28 |     setActive(false);
29 |     setDetections([]);
30 |   };
31 |
32 |   const start = async () => {
33 |     setError(null);
34 |     setLoading(true);
35 |     try {
36 |       await loadModel();
37 |       const stream = await navigator.mediaDevices.getUserMedia({
38 |         video: { facingMode: "user" },
39 |         audio: false,
40 |       });
41 |       const video = videoRef.current!;
42 |       video.srcObject = stream;
43 |       await video.play();
44 |       setSize({ w: video.videoWidth, h: video.videoHeight });
45 |       setActive(true);
46 |       const loop = async () => {
47 |         if (!videoRef.current) return;
48 |         try {
49 |           const found = await detectPeople(videoRef.current);
50 |           setDetections(found);
51 |         } catch {
52 |           /* ignore */
53 |         }
54 |         rafRef.current = requestAnimationFrame(loop);
55 |       };
56 |       loop();
57 |     } catch (e) {
58 |       setError(describeCameraError(e));
59 |     } finally {
60 |       setLoading(false);
61 |     }
62 |   };
63 |
64 |   useEffect(() => () => stop(), []);
65 |
66 |   return (
67 |     <
68 |       <Card className="overflow-hidden">
69 |         <div className="relative bg-black aspect-video flex items-center justify-center">
70 |           <video
71 |             ref={videoRef}
72 |             playsInline
73 |             muted
74 |             className="absolute inset-0 w-full h-full object-contain"
75 |           />
76 |           {active && size.w > 0 && (
77 |             <div className="absolute inset-0">
78 |               <DetectionCanvas
79 |                 width={size.w}
80 |                 height={size.h}
81 |                 detections={detections}
82 |                 className="w-full h-full object-contain"
83 |               />
84 |             </div>
85 |           )}
86 |           {!active && (
87 |             <div className="text-center text-muted-foreground p-8">
88 |               <Camera className="w-12 h-12 mx-auto mb-3 opacity-50" />
89 |               <p className="text-sm">Camera is off</p>
90 |             </div>
91 |           )}
92 |         </div>
93 |       </Card>
94 |     )
95 |   );
96 | }
```

```

91 |     })
92 |     {active && (
93 |         <div className="absolute top-3 left-3 flex items-center gap-2 px-3 py-1.5 rounded-full bg-background/90 ba
ckdrop-blur text-sm font-medium shadow">
94 |             <Users className="w-4 h-4 text-primary" />
95 |             {detections.length} {detections.length === 1 ? "person" : "people"}
96 |         </div>
97 |     })
98 | </div>
99 | <div className="p-4 flex items-center justify-between gap-3">
100 |     <p className="text-sm text-muted-foreground">
101 |         {error ?? (active ? "Live detection running" : "Start the camera to scan for people")}
102 |     </p>
103 |     {active ? (
104 |         <Button onClick={stop} variant="secondary">
105 |             <CameraOff className="w-4 h-4" /> Stop
106 |         </Button>
107 |     ) : (
108 |         <Button onClick={start} disabled={loading}>
109 |             {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : <Camera className="w-4 h-4" />}
110 |             {loading ? "Loading model..." : "Start camera"}
111 |         </Button>
112 |     )}
113 | </div>
114 | </Card>
115 | {active && (
116 |     <LifeStatus
117 |         source={videoRef.current}
118 |         detections={detections}
119 |         active={active}
120 |         calibration={calibration}
121 |     />
122 | )}
123 | {active && (
124 |     <LifeCalibration
125 |         label="Webcam"
126 |         values={calibration}
127 |         defaults={DEFAULTS_WEBCAM}
128 |         onChange={setCalibration}
129 |     />
130 | )}
131 | {active && <ConfidenceBars detections={detections} />}
132 | </>
133 | );
134 | }

```

src/components/XRayCanvas.tsx

```
1 | import { useEffect, useRef } from "react";
2 | import type { Detection } from "@lib/detector";
3 |
4 | export interface XRayCanvasProps {
5 |   source: HTMLVideoElement | HTMLImageElement | null;
6 |   width: number;
7 |   height: number;
8 |   detections: Detection[];
9 |   className?: string;
10 | }
11 |
12 | // Sobel edge detection + invert + cyan tint = simulated X-ray look.
13 | export function XRayCanvas({
14 |   source,
15 |   width,
16 |   height,
17 |   detections,
18 |   className,
19 | }: XRayCanvasProps) {
20 |   const canvasRef = useRef<HTMLCanvasElement>(null);
21 |   const offRef = useRef<HTMLCanvasElement | null>(null);
22 |   const rafRef = useRef<number | null>(null);
23 |
24 |   useEffect(() => {
25 |     const canvas = canvasRef.current;
26 |     if (!canvas || !source || width === 0 || height === 0) return;
27 |     canvas.width = width;
28 |     canvas.height = height;
29 |     if (!offRef.current) offRef.current = document.createElement("canvas");
30 |     const off = offRef.current;
31 |     off.width = width;
32 |     off.height = height;
33 |     const offCtx = off.getContext("2d", { willReadFrequently: true });
34 |     const ctx = canvas.getContext("2d");
35 |
36 |     const draw = () => {
37 |       try {
38 |         offCtx.drawImage(source, 0, 0, width, height);
39 |         const img = offCtx.getImageData(0, 0, width, height);
40 |         const src = img.data;
41 |
42 |         // Luminance buffer
43 |         const lum = new Float32Array(width * height);
44 |         for (let i = 0, p = 0; i < src.length; i += 4, p++) {
45 |           lum[p] = 0.299 * src[i] + 0.587 * src[i + 1] + 0.114 * src[i + 2];
46 |         }
47 |
48 |         // Sobel edges + inverted grayscale blend, tinted cyan.
49 |         const out = new Uint8ClampedArray(src.length);
50 |         for (let y = 1; y < height - 1; y++) {
51 |           for (let x = 1; x < width - 1; x++) {
52 |             const i = y * width + x;
53 |             const gx =
54 |               -lum[i - width - 1] - 2 * lum[i - 1] - lum[i + width - 1] +
55 |               lum[i - width + 1] + 2 * lum[i + 1] + lum[i + width + 1];
56 |             const gy =
57 |               -lum[i - width - 1] - 2 * lum[i - width] - lum[i - width + 1] +
58 |               lum[i + width - 1] + 2 * lum[i + width] + lum[i + width + 1];
59 |             const edge = Math.min(255, Math.sqrt(gx * gx + gy * gy));
60 |             const inv = 255 - lum[i];
61 |             const v = Math.min(255, inv * 0.55 + edge * 1.4);
62 |             const o = i * 4;
63 |             // Cyan/blue x-ray tint
64 |             out[o] = v * 0.55;
65 |             out[o + 1] = v * 0.9;
66 |             out[o + 2] = Math.min(255, v * 1.05 + 20);
67 |             out[o + 3] = 255;
68 |           }
69 |         }
70 |         const outImg = new ImageData(out, width, height);
71 |         ctx.putImageData(outImg, 0, 0);
72 |
73 |         // Scanline overlay
74 |         ctx.fillStyle = "rgba(0, 200, 255, 0.05)";
75 |         const bandH = 3;
76 |         for (let y = 0; y < height; y += bandH * 2) {
77 |           ctx.fillRect(0, y, width, bandH);
78 |         }
79 |
80 |         // Detection overlays with a stylized "skeleton" hint (spine + ribs).
81 |         ctx.font = `${Math.max(14, Math.round(height / 28))}px ui-monospace, monospace`;
82 |         ctx.lineWidth = 2;
83 |         ctx.strokeStyle = "#7fe7ff";
84 |         ctx.fillStyle = "rgba(0, 20, 40, 0.7)";
85 |
86 |         detections.forEach((d, idx) => {
87 |           const [x, y, w, h] = d.bbox;
88 |           ctx.strokeStyle = "#7fe7ff";
89 |           ctx.strokeRect(x, y, w, h);
90 |         });
```

```

91 | // Skeleton hint
92 | const cx = x + w / 2;
93 | ctx.save();
94 | ctx.strokeStyle = "rgba(220, 245, 255, 0.55)";
95 | ctx.lineWidth = 1.5;
96 | // spine
97 | ctx.beginPath();
98 | ctx.moveTo(cx, y + h * 0.2);
99 | ctx.lineTo(cx, y + h * 0.7);
100 | ctx.stroke();
101 | // ribs
102 | for (let r = 0; r < 4; r++) {
103 |   const ry = y + h * (0.25 + r * 0.08);
104 |   ctx.beginPath();
105 |   ctx.moveTo(x + w * 0.28, ry);
106 |   ctx.quadraticCurveTo(cx, ry - 4, x + w * 0.72, ry);
107 |   ctx.stroke();
108 | }
109 | // skull
110 | ctx.beginPath();
111 | ctx.arc(cx, y + h * 0.1, Math.min(w, h) * 0.09, 0, Math.PI * 2);
112 | ctx.stroke();
113 | ctx.restore();
114 |
115 | const label = `SUBJECT #${idx + 1} ${((d.score * 100).toFixed(0))}%`;
116 | const pad = 6;
117 | const lh = Math.max(18, Math.round(height / 24));
118 | const mw = ctx.measureText(label).width + pad * 2;
119 | ctx.fillStyle = "rgba(0, 20, 40, 0.75)";
120 | ctx.fillRect(x, Math.max(0, y - lh - 2), mw, lh + 2);
121 | ctx.fillStyle = "#c8f4ff";
122 | ctx.fillText(label, x + pad, Math.max(lh - 4, y - 6));
123 | });
124 | } catch {
125 |   /* source not ready */
126 | }
127 | rafRef.current = requestAnimationFrame(draw);
128 | };
129 | draw();
130 | return () => {
131 |   if (rafRef.current) cancelAnimationFrame(rafRef.current);
132 | };
133 | }, [source, width, height, detections]);
134 |
135 | return <canvas ref={canvasRef} className={className} />;
136 | }

```

src/components/XRayDetector.tsx

```
1 | import { describeCameraError } from "@lib/camera-error";
2 | import { useEffect, useRef, useState } from "react";
3 | import { Button } from "@components/ui/button";
4 | import { Card } from "@components/ui/card";
5 | import { detectPeople, loadModel, type Detection } from "@lib/detector";
6 | import { XRayCanvas } from "./XRayCanvas";
7 | import { ConfidenceBars } from "./ConfidenceBars";
8 | import { LifeStatus } from "./LifeStatus";
9 | import {
10 |   LifeCalibration,
11 |   DEFAULTS_WEBCAM,
12 |   type LifeCalibrationValues,
13 | } from "./LifeCalibration";
14 | import { CameraOff, Loader2, ScanLine, Users } from "lucide-react";
15 |
16 | export function XRayDetector() {
17 |   const videoRef = useRef<HTMLVideoElement>(null);
18 |   const rafRef = useRef<number | null>(null);
19 |   const [active, setActive] = useState(false);
20 |   const [loading, setLoading] = useState(false);
21 |   const [error, setError] = useState<string | null>(null);
22 |   const [detections, setDetections] = useState<Detection[]>([]);
23 |   const [size, setSize] = useState({ w: 640, h: 480 });
24 |   const [calibration, setCalibration] =
25 |     useState<LifeCalibrationValues>(DEFAULTS_WEBCAM);
26 |
27 |   const stop = () => {
28 |     if (rafRef.current) cancelAnimationFrame(rafRef.current);
29 |     rafRef.current = null;
30 |     const stream = videoRef.current?.srcObject as MediaStream | null;
31 |     stream?.getTracks().forEach((t) => t.stop());
32 |     if (videoRef.current) videoRef.current.srcObject = null;
33 |     setActive(false);
34 |     setDetections([]);
35 |   };
36 |
37 |   const start = async () => {
38 |     setError(null);
39 |     setLoading(true);
40 |     try {
41 |       await loadModel();
42 |       const stream = await navigator.mediaDevices.getUserMedia({
43 |         video: { facingMode: "user" },
44 |         audio: false,
45 |       });
46 |       const video = videoRef.current!;
47 |       video.srcObject = stream;
48 |       await video.play();
49 |       setSize({ w: video.videoWidth, h: video.videoHeight });
50 |       setActive(true);
51 |       const loop = async () => {
52 |         if (!videoRef.current) return;
53 |         try {
54 |           const found = await detectPeople(videoRef.current);
55 |           setDetections(found);
56 |         } catch {
57 |           /* ignore */
58 |         }
59 |         rafRef.current = requestAnimationFrame(loop);
60 |       };
61 |       loop();
62 |     } catch (e) {
63 |       setError(describeCameraError(e));
64 |     } finally {
65 |       setLoading(false);
66 |     }
67 |   };
68 |
69 |   useEffect(() => () => stop(), []);
70 |
71 |   return (
72 |     <Card className="overflow-hidden">
73 |       <div className="relative bg-black aspect-video flex items-center justify-center">
74 |         <video
75 |           ref={videoRef}
76 |           playsInline
77 |           muted
78 |           className="absolute inset-0 w-full h-full object-contain opacity-0 pointer-events-none"
79 |         />
80 |         {active && (
81 |           <XRayCanvas
82 |             source={videoRef.current}
83 |             width={size.w}
84 |             height={size.h}
85 |             detections={detections}
86 |             className="w-full h-full object-contain"
87 |           />
88 |         )}
89 |       </div>
90 |     </Card>
91 |   );
92 | }
```

```

91 |         <div className="text-center text-muted-foreground p-8">
92 |             <ScanLine className="w-12 h-12 mx-auto mb-3 opacity-50" />
93 |             <p className="text-sm">X-ray vision is off</p>
94 |         </div>
95 |     )}
96 |     {active && (
97 |         <
98 |             <div className="absolute top-3 left-3 flex items-center gap-2 px-3 py-1.5 rounded-full bg-cyan-500/20 bo
99 | rder border-cyan-400/40 text-cyan-100 backdrop-blur text-sm font-medium shadow">
100 |                 <Users className="w-4 h-4" />
101 |                 {detections.length} {detections.length === 1 ? "subject" : "subjects"}
102 |             </div>
103 |             <div className="absolute top-3 right-3 flex items-center gap-2 px-3 py-1.5 rounded-full bg-cyan-500/20 b
104 | order border-cyan-400/40 text-cyan-100 backdrop-blur text-xs font-mono shadow">
105 |                 <ScanLine className="w-3.5 h-3.5" /> X-RAY · SIM
106 |             </div>
107 |         </>
108 |     )}
109 | </div>
110 | <div className="p-4 flex items-center justify-between gap-3">
111 |     <p className="text-sm text-muted-foreground">
112 |         {error ??
113 |             (active
114 |                 ? "Simulated X-ray vision · edge + inverse luminance rendering"
115 |                 : "Start the X-ray camera to reveal a simulated skeletal view")}
116 |     </p>
117 |     {active ? (
118 |         <Button onClick={stop} variant="secondary">
119 |             <CameraOff className="w-4 h-4" /> Stop
120 |         </Button>
121 |     ) : (
122 |         <Button onClick={start} disabled={loading}>
123 |             {loading ? (
124 |                 <Loader2 className="w-4 h-4 animate-spin" />
125 |             ) : (
126 |                 <ScanLine className="w-4 h-4" />
127 |             )}
128 |             {loading ? "Loading model..." : "Start X-ray camera"}
129 |         </Button>
130 |     )}
131 | </div>
132 | </Card>
133 | {active && (
134 |     <LifeStatus
135 |         source={videoRef.current}
136 |         detections={detections}
137 |         active={active}
138 |         calibration={calibration}
139 |     />
140 | )}
141 | {active && (
142 |     <LifeCalibration
143 |         label="X-ray"
144 |         values={calibration}
145 |         defaults={DEFAULTS_WEBCAM}
146 |         onChange={setCalibration}
147 |     />
148 | )}
149 | {active && <ConfidenceBars detections={detections} />}
150 | </>

```

src/components/ui/accordion.tsx

```
1 | import * as React from "react";
2 | import * as AccordionPrimitive from "@radix-ui/react-accordion";
3 | import { ChevronDown } from "lucide-react";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const Accordion = AccordionPrimitive.Root;
8 |
9 | const AccordionItem = React.forwardRef<
10 |   React.ElementRef<typeof AccordionPrimitive.Item>,
11 |   React.ComponentPropsWithoutRef<typeof AccordionPrimitive.Item>
12 | >(({ className, ...props }, ref) => (
13 |   <AccordionPrimitive.Item ref={ref} className={cn("border-b", className)} {...props} />
14 | ));
15 | AccordionItem.displayName = "AccordionItem";
16 |
17 | const AccordionTrigger = React.forwardRef<
18 |   React.ElementRef<typeof AccordionPrimitive.Trigger>,
19 |   React.ComponentPropsWithoutRef<typeof AccordionPrimitive.Trigger>
20 | >(({ className, children, ...props }, ref) => (
21 |   <AccordionPrimitive.Header className="flex">
22 |     <AccordionPrimitive.Trigger
23 |       ref={ref}
24 |       className={cn(
25 |         "flex flex-1 items-center justify-between py-4 text-sm font-medium cursor-pointer transition-all hover:underline text-left [&[data-state=open]>svg]:rotate-180",
26 |         className,
27 |       )}
28 |       {...props}
29 |     >
30 |       {children}
31 |       <ChevronDown className="h-4 w-4 shrink-0 text-muted-foreground transition-transform duration-200" />
32 |     </AccordionPrimitive.Trigger>
33 |   </AccordionPrimitive.Header>
34 | ));
35 | AccordionTrigger.displayName = AccordionPrimitive.Trigger.displayName;
36 |
37 | const AccordionContent = React.forwardRef<
38 |   React.ElementRef<typeof AccordionPrimitive.Content>,
39 |   React.ComponentPropsWithoutRef<typeof AccordionPrimitive.Content>
40 | >(({ className, children, ...props }, ref) => (
41 |   <AccordionPrimitive.Content
42 |     ref={ref}
43 |     className="overflow-hidden text-sm data-[state=closed]:animate-accordion-up data-[state=open]:animate-accordion-down"
44 |     {...props}
45 |   >
46 |     <div className={cn("pb-4 pt-0", className)}>{children}</div>
47 |   </AccordionPrimitive.Content>
48 | ));
49 | AccordionContent.displayName = AccordionPrimitive.Content.displayName;
50 |
51 | export { Accordion, AccordionItem, AccordionTrigger, AccordionContent };
```

src/components/ui/alert-dialog.tsx

```
1 | import * as React from "react";
2 | import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog";
3 |
4 | import { cn } from "@lib/utils";
5 | import { buttonVariants } from "@components/ui/button";
6 |
7 | const AlertDialog = AlertDialogPrimitive.Root;
8 |
9 | const AlertDialogTrigger = AlertDialogPrimitive.Trigger;
10 |
11 | const AlertDialogPortal = AlertDialogPrimitive.Portal;
12 |
13 | const AlertDialogOverlay = React.forwardRef<
14 |   React.ElementRef<typeof AlertDialogPrimitive.Overlay>,
15 |   React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Overlay>
16 | >(({ className, ...props }, ref) => (
17 |   <AlertDialogPrimitive.Overlay
18 |     className={cn(
19 |       "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]
20 |       :fade-out-0 data-[state=open]:fade-in-0",
21 |       className,
22 |     )}
23 |     {...props}
24 |     ref={ref}
25 |   />
26 | ));
27 | AlertDialogOverlay.displayName = AlertDialogPrimitive.Overlay.displayName;
28 |
29 | const AlertDialogContent = React.forwardRef<
30 |   React.ElementRef<typeof AlertDialogPrimitive.Content>,
31 |   React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Content>
32 | >(({ className, ...props }, ref) => (
33 |   <AlertDialogPortal>
34 |     <AlertDialogOverlay />
35 |     <AlertDialogPrimitive.Content
36 |       ref={ref}
37 |       className={cn(
38 |         "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border bg-ba
39 |         ckground p-6 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=close
40 |         d]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 sm:rounded-
41 |         lg",
42 |         className,
43 |       )}
44 |       {...props}
45 |     />
46 |   </AlertDialogPortal>
47 | ));
48 | AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName;
49 |
50 | const AlertDialogHeader = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => (
51 |   <div className={cn("flex flex-col space-y-2 text-center sm:text-left", className)} {...props} />
52 | );
53 | AlertDialogHeader.displayName = "AlertDialogHeader";
54 |
55 | const AlertDialogFooter = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => (
56 |   <div
57 |     className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
58 |     {...props}
59 |   />
60 | );
61 | AlertDialogFooter.displayName = "AlertDialogFooter";
62 |
63 | const AlertDialogTitle = React.forwardRef<
64 |   React.ElementRef<typeof AlertDialogPrimitive.Title>,
65 |   React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Title>
66 | >(({ className, ...props }, ref) => (
67 |   <AlertDialogPrimitive.Title
68 |     ref={ref}
69 |     className={cn("text-lg font-semibold", className)}
70 |     {...props}
71 |   />
72 | ));
73 | AlertDialogTitle.displayName = AlertDialogPrimitive.Title.displayName;
74 |
75 | const AlertDialogDescription = React.forwardRef<
76 |   React.ElementRef<typeof AlertDialogPrimitive.Description>,
77 |   React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Description>
78 | >(({ className, ...props }, ref) => (
79 |   <AlertDialogPrimitive.Description
80 |     ref={ref}
81 |     className={cn("text-sm text-muted-foreground", className)}
82 |     {...props}
83 |   />
84 | ));
85 | AlertDialogDescription.displayName = AlertDialogPrimitive.Description.displayName;
86 |
87 | const AlertDialogAction = React.forwardRef<
88 |   React.ElementRef<typeof AlertDialogPrimitive.Action>,
89 |   React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Action>
90 | >(({ className, ...props }, ref) => (
91 |   <AlertDialogPrimitive.Action
92 |     ref={ref}
93 |     className={cn("text-sm font-medium text-white", className)}
94 |     {...props}
95 |   />
96 | ));
97 | AlertDialogAction.displayName = "AlertDialogAction";
```

```

87 |   <AlertDialogPrimitive.Action ref={ref} className={cn(buttonVariants(), className)} {...props} />
88 | ));
89 | AlertDialogAction.displayName = AlertDialogPrimitive.Action.displayName;
90 |
91 | const AlertDialogCancel = React.forwardRef<
92 |   React.ElementRef<typeof AlertDialogPrimitive.Cancel>,
93 |   React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Cancel>
94 | >(({ className, ...props }, ref) => (
95 |   <AlertDialogPrimitive.Cancel
96 |     ref={ref}
97 |     className={cn(buttonVariants({ variant: "outline" }), "mt-2 sm:mt-0", className)}
98 |     {...props}
99 |   />
100 | ));
101 | AlertDialogCancel.displayName = AlertDialogPrimitive.Cancel.displayName;
102 |
103 | export {
104 |   AlertDialog,
105 |   AlertDialogPortal,
106 |   AlertDialogOverlay,
107 |   AlertDialogTrigger,
108 |   AlertDialogContent,
109 |   AlertDialogHeader,
110 |   AlertDialogFooter,
111 |   AlertDialogTitle,
112 |   AlertDialogDescription,
113 |   AlertDialogAction,
114 |   AlertDialogCancel,
115 | };

```

src/components/ui/alert.tsx

```
1 | import * as React from "react";
2 | import { cva, type VariantProps } from "class-variance-authority";
3 |
4 | import { cn } from "@lib/utils";
5 |
6 | const alertVariants = cva(
7 |   "relative w-full rounded-lg border px-4 py-3 text-sm [&svg+div]:translate-y-[-3px] [&svg]:absolute [&svg]:left-4
   |   [&svg]:top-4 [&svg]:text-foreground [&svg~*]:pl-7",
8 |   {
9 |     variants: {
10 |       variant: {
11 |         default: "bg-background text-foreground",
12 |         destructive:
13 |           "border-destructive/50 text-destructive dark:border-destructive [&svg]:text-destructive",
14 |       },
15 |     },
16 |     defaultVariants: {
17 |       variant: "default",
18 |     },
19 |   },
20 | );
21 |
22 | const Alert = React.forwardRef<
23 |   HTMLDivElement,
24 |   React.HTMLAttributes<HTMLDivElement> & VariantProps<typeof alertVariants>
25 | >(({ className, variant, ...props }, ref) => (
26 |   <div ref={ref} role="alert" className={cn(alertVariants({ variant }), className)} {...props} />
27 | ));
28 | Alert.displayName = "Alert";
29 |
30 | const AlertTitle = React.forwardRef<HTMLParagraphElement, React.HTMLAttributes<HTMLHeadingElement>>(
31 |   ({ className, ...props }, ref) => (
32 |     <h5
33 |       ref={ref}
34 |       className={cn("mb-1 font-medium leading-none tracking-tight", className)}
35 |       {...props}
36 |     />
37 |   ),
38 | );
39 | AlertTitle.displayName = "AlertTitle";
40 |
41 | const AlertDescription = React.forwardRef<
42 |   HTMLParagraphElement,
43 |   React.HTMLAttributes<HTMLParagraphElement>
44 | >(({ className, ...props }, ref) => (
45 |   <div ref={ref} className={cn("text-sm [&_p]:leading-relaxed", className)} {...props} />
46 | ));
47 | AlertDescription.displayName = "AlertDescription";
48 |
49 | export { Alert, AlertTitle, AlertDescription };
```

src/components/ui/aspect-ratio.tsx

```
1 | import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio";
2 |
3 | const AspectRatio = AspectRatioPrimitive.Root;
4 |
5 | export { AspectRatio };
```

src/components/ui/avatar.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as AvatarPrimitive from "@radix-ui/react-avatar";
5 |
6 | import { cn } from "@lib/utils";
7 |
8 | const Avatar = React.forwardRef<
9 |   React.ElementRef<typeof AvatarPrimitive.Root>,
10 |   React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Root>
11 | >(({ className, ...props }, ref) => (
12 |   <AvatarPrimitive.Root
13 |     ref={ref}
14 |     className={cn("relative flex h-10 w-10 shrink-0 overflow-hidden rounded-full", className)}
15 |     {...props}
16 |   />
17 | ));
18 | Avatar.displayName = AvatarPrimitive.Root.displayName;
19 |
20 | const AvatarImage = React.forwardRef<
21 |   React.ElementRef<typeof AvatarPrimitive.Image>,
22 |   React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Image>
23 | >(({ className, ...props }, ref) => (
24 |   <AvatarPrimitive.Image
25 |     ref={ref}
26 |     className={cn("aspect-square h-full w-full", className)}
27 |     {...props}
28 |   />
29 | ));
30 | AvatarImage.displayName = AvatarPrimitive.Image.displayName;
31 |
32 | const AvatarFallback = React.forwardRef<
33 |   React.ElementRef<typeof AvatarPrimitive.Fallback>,
34 |   React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Fallback>
35 | >(({ className, ...props }, ref) => (
36 |   <AvatarPrimitive.Fallback
37 |     ref={ref}
38 |     className={cn(
39 |       "flex h-full w-full items-center justify-center rounded-full bg-muted",
40 |       className,
41 |     )}
42 |     {...props}
43 |   />
44 | ));
45 | AvatarFallback.displayName = AvatarPrimitive.Fallback.displayName;
46 |
47 | export { Avatar, AvatarImage, AvatarFallback };
```

src/components/ui/badge.tsx

```
1 | import * as React from "react";
2 | import { cva, type VariantProps } from "class-variance-authority";
3 |
4 | import { cn } from "@lib/utils";
5 |
6 | const badgeVariants = cva(
7 |   "inline-flex items-center rounded-md border px-2.5 py-0.5 text-xs font-semibold transition-colors focus:outline-none
8 |   focus:ring-2 focus:ring-ring focus:ring-offset-2",
9 |   {
10 |     variant: {
11 |       default: "border-transparent bg-primary text-primary-foreground shadow hover:bg-primary/80",
12 |       secondary: "border-transparent bg-secondary text-secondary-foreground hover:bg-secondary/80",
13 |       destructive: "border-transparent bg-destructive text-destructive-foreground shadow hover:bg-destructive/80",
14 |       outline: "text-foreground",
15 |     },
16 |     defaultVariants: {
17 |       variant: "default",
18 |     },
19 |   },
20 | );
21 |
22 | export interface BadgeProps
23 |   extends React.HTMLAttributes<HTMLDivElement>, VariantProps<typeof badgeVariants> {}
24 |
25 | function Badge({ className, variant, ...props }: BadgeProps) {
26 |   return <div className={cn(badgeVariants({ variant }), className)} {...props} />;
27 | }
28 |
29 | export { Badge, badgeVariants };
```

src/components/ui/breadcrumb.tsx

```
1 | import * as React from "react";
2 | import { Slot } from "@radix-ui/react-slot";
3 | import { ChevronRight, MoreHorizontal } from "lucide-react";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const Breadcrumb = React.forwardRef<
8 |   HTMLDivElement,
9 |   React.ComponentPropsWithoutRef<"nav"> & {
10 |     separator?: React.ReactNode;
11 |   }
12 | >(({ ...props }, ref) => <nav ref={ref} aria-label="breadcrumb" {...props} />);
13 | Breadcrumb.displayName = "Breadcrumb";
14 |
15 | const BreadcrumbList = React.forwardRef<HTMLListElement, React.ComponentPropsWithoutRef<"ol">>(
16 |   ({ className, ...props }, ref) => (
17 |     <ol
18 |       ref={ref}
19 |       className={cn(
20 |         "flex flex-wrap items-center gap-1.5 break-words text-sm text-muted-foreground sm:gap-2.5",
21 |         className,
22 |       )}
23 |       {...props}
24 |     />
25 |   ),
26 | );
27 | BreadcrumbList.displayName = "BreadcrumbList";
28 |
29 | const BreadcrumbItem = React.forwardRef<HTMLLIElement, React.ComponentPropsWithoutRef<"li">>(
30 |   ({ className, ...props }, ref) => (
31 |     <li ref={ref} className={cn("inline-flex items-center gap-1.5", className)} {...props} />
32 |   ),
33 | );
34 | BreadcrumbItem.displayName = "BreadcrumbItem";
35 |
36 | const BreadcrumbLink = React.forwardRef<
37 |   HTMLAnchorElement,
38 |   React.ComponentPropsWithoutRef<"a"> & {
39 |     asChild?: boolean;
40 |   }
41 | >(({ asChild, className, ...props }, ref) => {
42 |   const Comp = asChild ? Slot : "a";
43 |
44 |   return (
45 |     <Comp
46 |       ref={ref}
47 |       className={cn("transition-colors hover:text-foreground", className)}
48 |       {...props}
49 |     />
50 |   );
51 | });
52 | BreadcrumbLink.displayName = "BreadcrumbLink";
53 |
54 | const BreadcrumbPage = React.forwardRef<HTMLSpanElement, React.ComponentPropsWithoutRef<"span">>(
55 |   ({ className, ...props }, ref) => (
56 |     <span
57 |       ref={ref}
58 |       role="link"
59 |       aria-disabled="true"
60 |       aria-current="page"
61 |       className={cn("font-normal text-foreground", className)}
62 |       {...props}
63 |     />
64 |   ),
65 | );
66 | BreadcrumbPage.displayName = "BreadcrumbPage";
67 |
68 | const BreadcrumbSeparator = ({ children, className, ...props }: React.ComponentProps<"li">) => (
69 |   <li
70 |     role="presentation"
71 |     aria-hidden="true"
72 |     className={cn("[&svg]:w-3.5 [&svg]:h-3.5", className)}
73 |     {...props}
74 |   >
75 |     {children ?? <ChevronRight />}
76 |   </li>
77 | );
78 | BreadcrumbSeparator.displayName = "BreadcrumbSeparator";
79 |
80 | const BreadcrumbEllipsis = ({ className, ...props }: React.ComponentProps<"span">) => (
81 |   <span
82 |     role="presentation"
83 |     aria-hidden="true"
84 |     className={cn("flex h-9 w-9 items-center justify-center", className)}
85 |     {...props}
86 |   >
87 |     <MoreHorizontal className="h-4 w-4" />
88 |     <span className="sr-only">More</span>
89 |   </span>
90 | );
```

```
91 | BreadcrumbEllipsis.displayName = "BreadcrumbElipssis";
92 |
93 | export {
94 |   Breadcrumb,
95 |   BreadcrumbList,
96 |   BreadcrumbItem,
97 |   BreadcrumbLink,
98 |   BreadcrumbPage,
99 |   BreadcrumbSeparator,
100 |   BreadcrumbEllipsis,
101 | };;
```

src/components/ui/button.tsx

```
1 | import * as React from "react";
2 | import { Slot } from "@radix-ui/react-slot";
3 | import { cva, type VariantProps } from "class-variance-authority";
4 |
5 | import { cn } from "@lib/utlis";
6 |
7 | const buttonVariants = cva(
8 |   "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium cursor-pointer transition-colors focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
9 |   {
10 |     variants: {
11 |       variant: {
12 |         default: "bg-primary text-primary-foreground shadow hover:bg-primary/90",
13 |         destructive: "bg-destructive text-destructive-foreground shadow-sm hover:bg-destructive/90",
14 |         outline:
15 |           "border border-input bg-background shadow-sm hover:bg-accent hover:text-accent-foreground",
16 |         secondary: "bg-secondary text-secondary-foreground shadow-sm hover:bg-secondary/80",
17 |         ghost: "hover:bg-accent hover:text-accent-foreground",
18 |         link: "text-primary underline-offset-4 hover:underline",
19 |       },
20 |       size: {
21 |         default: "h-9 px-4 py-2",
22 |         sm: "h-8 rounded-md px-3 text-xs",
23 |         lg: "h-10 rounded-md px-8",
24 |         icon: "h-9 w-9",
25 |       },
26 |     },
27 |     defaultVariants: {
28 |       variant: "default",
29 |       size: "default",
30 |     },
31 |   },
32 | );
33 |
34 | export interface ButtonProps
35 |   extends React.ButtonHTMLAttributes<HTMLButtonElement>, VariantProps<typeof buttonVariants> {
36 |   asChild?: boolean;
37 | }
38 |
39 | const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
40 |   ({ className, variant, size, asChild = false, ...props }, ref) => {
41 |     const Comp = asChild ? Slot : "button";
42 |     return (
43 |       <Comp className={cn(buttonVariants({ variant, size, className }))} ref={ref} {...props} />
44 |     );
45 |   },
46 | );
47 | Button.displayName = "Button";
48 |
49 | export { Button, buttonVariants };
```

src/components/ui/calendar.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import { ChevronDownIcon, ChevronLeftIcon, ChevronRightIcon } from "lucide-react";
5 | import { DayButton, DayPicker, getDefaultClassNames } from "react-day-picker";
6 |
7 | import { cn } from "@lib/utlis";
8 | import { Button, buttonVariants } from "@components/ui/button";
9 |
10 | function Calendar({
11 |   className,
12 |   classNames,
13 |   showOutsideDays = true,
14 |   captionLayout = "label",
15 |   buttonVariant = "ghost",
16 |   formatters,
17 |   components,
18 |   ...props
19 | }: React.ComponentProps<typeof DayPicker> & {
20 |   buttonVariant?: React.ComponentProps<typeof Button>["variant"];
21 | }) {
22 |   const defaultClassNames = getDefaultClassNames();
23 |
24 |   return (
25 |     <DayPicker
26 |       showOutsideDays={showOutsideDays}
27 |       className=cn(
28 |         "bg-background group/calendar p-3 [--cell-size:2rem] [[data-slot=card-content]_&]:bg-transparent [[data-slot=p
29 |         popover-content]_&]:bg-transparent",
30 |         String.raw`rtl:*.:[.rdp-button\_next>svg]:rotate-180`,
31 |         String.raw`rtl:*.:[.rdp-button\_previous>svg]:rotate-180`,
32 |         className,
33 |       )
34 |       captionLayout={captionLayout}
35 |       formatters={{
36 |         formatMonthDropdown: (date) => date.toLocaleString("default", { month: "short" }),
37 |         ...formatters,
38 |       }}
39 |       classNames={{
40 |         root: cn("w-fit", defaultClassNames.root),
41 |         months: cn("relative flex flex-col gap-4 md:flex-row", defaultClassNames.months),
42 |         month: cn("flex w-full flex-col gap-4", defaultClassNames.month),
43 |         nav: cn(
44 |           "absolute inset-x-0 top-0 flex w-full items-center justify-between gap-1",
45 |           defaultClassNames.nav,
46 |         ),
47 |         button_previous: cn(
48 |           buttonVariants({ variant: buttonVariant }),
49 |           "h-(-cell-size) w-(-cell-size) select-none p-0 aria-disabled:opacity-50",
50 |           defaultClassNames.button_previous,
51 |         ),
52 |         button_next: cn(
53 |           buttonVariants({ variant: buttonVariant }),
54 |           "h-(-cell-size) w-(-cell-size) select-none p-0 aria-disabled:opacity-50",
55 |           defaultClassNames.button_next,
56 |         ),
57 |         month_caption: cn(
58 |           "flex h-(-cell-size) w-full items-center justify-center px-(-cell-size)",
59 |           defaultClassNames.month_caption,
60 |         ),
61 |         dropdowns: cn(
62 |           "flex h-(-cell-size) w-full items-center justify-center gap-1.5 text-sm font-medium",
63 |           defaultClassNames.dropdowns,
64 |         ),
65 |         dropdown_root: cn(
66 |           "has-focus:border-ring border-input shadow-xs has-focus:ring-ring/50 has-focus:ring-[3px] relative rounded-m
67 |           d border",
68 |           defaultClassNames.dropdown_root,
69 |         ),
70 |         dropdown: cn("bg-popover absolute inset-0 opacity-0", defaultClassNames.dropdown),
71 |         caption_label: cn(
72 |           "select-none font-medium",
73 |           captionLayout === "label"
74 |             ? "text-sm"
75 |             : "[&>svg]:text-muted-foreground flex h-8 items-center gap-1 rounded-md pl-2 pr-1 text-sm [&>svg]:size-3.5",
76 |           defaultClassNames.caption_label,
77 |         ),
78 |         table: "w-full border-collapse",
79 |         weekdays: cn("flex", defaultClassNames.weekdays),
80 |         weekday: cn(
81 |           "text-muted-foreground flex-1 select-none rounded-md text-[0.8rem] font-normal",
82 |           defaultClassNames.weekday,
83 |         ),
84 |         week: cn("mt-2 flex w-full", defaultClassNames.week),
85 |         week_number_header: cn("w-(-cell-size) select-none", defaultClassNames.week_number_header),
86 |         week_number: cn(
87 |           "text-muted-foreground select-none text-[0.8rem]",
88 |           defaultClassNames.week_number,
89 |         ),
90 |       }}
91 |     />
92 |   );
93 | }
```

```

88 |     day: cn(
89 |       "group/day relative aspect-square h-full w-full select-none p-0 text-center [&:first-child[data-selected=true]_button]:rounded-l-md [&:last-child[data-selected=true]_button]:rounded-r-md",
90 |       defaultClassNames.day,
91 |     ),
92 |     range_start: cn("bg-accent rounded-l-md", defaultClassNames.range_start),
93 |     range_middle: cn("rounded-none", defaultClassNames.range_middle),
94 |     range_end: cn("bg-accent rounded-r-md", defaultClassNames.range_end),
95 |     today: cn(
96 |       "bg-accent text-accent-foreground rounded-md data-[selected=true]:rounded-none",
97 |       defaultClassNames.today,
98 |     ),
99 |     outside: cn(
100 |       "text-muted-foreground aria-selected:text-muted-foreground",
101 |       defaultClassNames.outside,
102 |     ),
103 |     disabled: cn("text-muted-foreground opacity-50", defaultClassNames.disabled),
104 |     hidden: cn("invisible", defaultClassNames.hidden),
105 |     ...classNames,
106 |   })
107 |   components={
108 |     Root: ({ className, rootRef, ...props }) => {
109 |       return <div data-slot="calendar" ref={rootRef} className={cn(className)} {...props} />;
110 |     },
111 |     Chevron: ({ className, orientation, ...props }) => {
112 |       if (orientation === "left") {
113 |         return <ChevronLeftIcon className={cn("size-4", className)} {...props} />;
114 |       }
115 |
116 |       if (orientation === "right") {
117 |         return <ChevronRightIcon className={cn("size-4", className)} {...props} />;
118 |       }
119 |
120 |       return <ChevronDownIcon className={cn("size-4", className)} {...props} />;
121 |     },
122 |     DayButton: CalendarDayButton,
123 |     WeekNumber: ({ children, ...props }) => {
124 |       return (
125 |         <td {...props}>
126 |           <div className="flex size-(--cell-size) items-center justify-center text-center">
127 |             {children}
128 |           </div>
129 |         </td>
130 |       );
131 |     },
132 |     ...components,
133 |   })
134 |   {...props}
135 | </>
136 | );
137 | }
138 |
139 | function CalendarDayButton({
140 |   className,
141 |   day,
142 |   modifiers,
143 |   ...props
144 | }: React.ComponentProps<typeof DayButton>) {
145 |   const defaultClassNames = getDefaultClassNames();
146 |
147 |   const ref = React.useRef<HTMLButtonElement>(null);
148 |   React.useEffect(() => {
149 |     if (modifiers.focused) ref.current?.focus();
150 |   }, [modifiers.focused]);
151 |
152 |   return (
153 |     <Button
154 |       ref={ref}
155 |       variant="ghost"
156 |       size="icon"
157 |       data-day={day.date.toLocaleDateString()}
158 |       data-selected-single={
159 |         modifiers.selected &&
160 |         !modifiers.range_start &&
161 |         !modifiers.range_end &&
162 |         !modifiers.range_middle
163 |       }
164 |       data-range-start={modifiers.range_start}
165 |       data-range-end={modifiers.range_end}
166 |       data-range-middle={modifiers.range_middle}
167 |       className=cn(
168 |         "data-[selected-single=true]:bg-primary data-[selected-single=true]:text-primary-foreground data-[range-middle=true]:bg-accent data-[range-middle=true]:text-accent-foreground data-[range-start=true]:bg-primary data-[range-start=true]:text-primary-foreground data-[range-end=true]:bg-primary data-[range-end=true]:text-primary-foreground group-data-[focused=true]/day:border-ring group-data-[focused=true]/day:ring-ring/50 flex aspect-square h-auto w-full min-w-(--cell-size) flex-col gap-1 font-normal leading-none data-[range-end=true]:rounded-md data-[range-middle=true]:rounded-none data-[range-start=true]:rounded-md group-data-[focused=true]/day:relative group-data-[focused=true]/day:z-10 group-data-[focused=true]/day:ring-[3px] [&span]:text-xs [&span]:opacity-70",
169 |         defaultClassNames.day,
170 |         className,
171 |       )
172 |       {...props}
173 |     </>

```

```
174 | );  
175 | }  
176 |  
177 | export { Calendar, CalendarDayButton };
```

src/components/ui/card.tsx

```
1 | import * as React from "react";
2 |
3 | import { cn } from "@lib/utils";
4 |
5 | const Card = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(<
6 |   ({ className, ...props }, ref) => (
7 |     <div
8 |       ref={ref}
9 |       className={cn("rounded-xl border bg-card text-card-foreground shadow", className)}
10 |      {...props}
11 |    />
12 |   ),
13 | );
14 | Card.displayName = "Card";
15 |
16 | const CardHeader = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(<
17 |   ({ className, ...props }, ref) => (
18 |     <div ref={ref} className={cn("flex flex-col space-y-1.5 p-6", className)} {...props} />
19 |   ),
20 | );
21 | CardHeader.displayName = "CardHeader";
22 |
23 | const CardTitle = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(<
24 |   ({ className, ...props }, ref) => (
25 |     <div
26 |       ref={ref}
27 |       className={cn("font-semibold leading-none tracking-tight", className)}
28 |      {...props}
29 |    />
30 |   ),
31 | );
32 | CardTitle.displayName = "CardTitle";
33 |
34 | const CardDescription = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(<
35 |   ({ className, ...props }, ref) => (
36 |     <div ref={ref} className={cn("text-sm text-muted-foreground", className)} {...props} />
37 |   ),
38 | );
39 | CardDescription.displayName = "CardDescription";
40 |
41 | const CardContent = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(<
42 |   ({ className, ...props }, ref) => (
43 |     <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
44 |   ),
45 | );
46 | CardContent.displayName = "CardContent";
47 |
48 | const CardFooter = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(<
49 |   ({ className, ...props }, ref) => (
50 |     <div ref={ref} className={cn("flex items-center p-6 pt-0", className)} {...props} />
51 |   ),
52 | );
53 | CardFooter.displayName = "CardFooter";
54 |
55 | export { Card, CardHeader, CardFooter, CardTitle, CardDescription, CardContent };
```

src/components/ui/carousel.tsx

```
1 | import * as React from "react";
2 | import useEmblaCarousel, { type UseEmblaCarouselType } from "embla-carousel-react";
3 | import { ArrowLeft, ArrowRight } from "lucide-react";
4 |
5 | import { cn } from "@lib/utils";
6 | import { Button } from "@components/ui/button";
7 |
8 | type CarouselApi = UseEmblaCarouselType[1];
9 | type UseCarouselParameters = Parameters<typeof useEmblaCarousel>;
10 | type CarouselOptions = UseCarouselParameters[0];
11 | type CarouselPlugin = UseCarouselParameters[1];
12 |
13 | type CarouselProps = {
14 |   opts?: CarouselOptions;
15 |   plugins?: CarouselPlugin;
16 |   orientation?: "horizontal" | "vertical";
17 |   setApi?: (api: CarouselApi) => void;
18 | };
19 |
20 | type CarouselContextProps = {
21 |   carouselRef: ReturnType<typeof useEmblaCarousel>[0];
22 |   api: ReturnType<typeof useEmblaCarousel>[1];
23 |   scrollPrev: () => void;
24 |   scrollNext: () => void;
25 |   canScrollPrev: boolean;
26 |   canScrollNext: boolean;
27 | } & CarouselProps;
28 |
29 | const CarouselContext = React.createContext<CarouselContextProps | null>(null);
30 |
31 | function useCarousel() {
32 |   const context = React.useContext(CarouselContext);
33 |
34 |   if (!context) {
35 |     throw new Error("useCarousel must be used within a <Carousel />");
36 |   }
37 |
38 |   return context;
39 | }
40 |
41 | const Carousel = React.forwardRef<
42 |   HTMLDivElement,
43 |   React.HTMLAttributes<HTMLDivElement> & CarouselProps
44 | >(({ orientation = "horizontal", opts, setApi, plugins, className, children, ...props }, ref) => {
45 |   const [carouselRef, api] = useEmblaCarousel(
46 |     {
47 |       ...opts,
48 |       axis: orientation === "horizontal" ? "x" : "y",
49 |     },
50 |     plugins,
51 |   );
52 |   const [canScrollPrev, setCanScrollPrev] = React.useState(false);
53 |   const [canScrollNext, setCanScrollNext] = React.useState(false);
54 |
55 |   const onSelect = React.useCallback((api: CarouselApi) => {
56 |     if (!api) {
57 |       return;
58 |     }
59 |
60 |     setCanScrollPrev(api.canScrollPrev());
61 |     setCanScrollNext(api.canScrollNext());
62 |   }, []);
63 |
64 |   const scrollPrev = React.useCallback(() => {
65 |     api?.scrollPrev();
66 |   }, [api]);
67 |
68 |   const scrollNext = React.useCallback(() => {
69 |     api?.scrollNext();
70 |   }, [api]);
71 |
72 |   const handleKeyDown = React.useCallback(
73 |     (event: React.KeyboardEvent<HTMLDivElement>) => {
74 |       if (event.key === "ArrowLeft") {
75 |         event.preventDefault();
76 |         scrollPrev();
77 |       } else if (event.key === "ArrowRight") {
78 |         event.preventDefault();
79 |         scrollNext();
80 |       }
81 |     },
82 |     [scrollPrev, scrollNext],
83 |   );
84 |
85 |   React.useEffect(() => {
86 |     if (!api || !setApi) {
87 |       return;
88 |     }
89 |
90 |     setApi(api);
```

```

91 | }, [api, setApi]);
92 |
93 | React.useEffect(() => {
94 |   if (!api) {
95 |     return;
96 |   }
97 |
98 |   onSelect(api);
99 |   api.on("reInit", onSelect);
100 |   api.on("select", onSelect);
101 |
102 |   return () => {
103 |     api?.off("select", onSelect);
104 |   };
105 | }, [api, onSelect]);
106 |
107 | return (
108 |   <CarouselContext.Provider
109 |     value={{
110 |       carouselRef,
111 |       api: api,
112 |       opts,
113 |       orientation: orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
114 |       scrollPrev,
115 |       scrollNext,
116 |       canScrollPrev,
117 |       canScrollNext,
118 |     }}
119 |   >
120 |     <div
121 |       ref={ref}
122 |       onKeyDownCapture={handleKeyDown}
123 |       className={cn("relative", className)}
124 |       role="region"
125 |       aria-roledescription="carousel"
126 |       {...props}
127 |     >
128 |       {children}
129 |     </div>
130 |   </CarouselContext.Provider>
131 | );
132 | });
133 | Carousel.displayName = "Carousel";
134 |
135 | const CarouselContent = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(
136 |   ({ className, ...props }, ref) => {
137 |     const { carouselRef, orientation } = useCarousel();
138 |
139 |     return (
140 |       <div ref={carouselRef} className="overflow-hidden">
141 |         <div
142 |           ref={ref}
143 |           className={cn(
144 |             "flex",
145 |             orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
146 |             className,
147 |           )}
148 |           {...props}
149 |         />
150 |       </div>
151 |     );
152 |   },
153 | );
154 | CarouselContent.displayName = "CarouselContent";
155 |
156 | const CarouselItem = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(
157 |   ({ className, ...props }, ref) => {
158 |     const { orientation } = useCarousel();
159 |
160 |     return (
161 |       <div
162 |         ref={ref}
163 |         role="group"
164 |         aria-roledescription="slide"
165 |         className={cn(
166 |           "min-w-0 shrink-0 grow-0 basis-full",
167 |           orientation === "horizontal" ? "pl-4" : "pt-4",
168 |           className,
169 |         )}
170 |         {...props}
171 |       />
172 |     );
173 |   },
174 | );
175 | CarouselItem.displayName = "CarouselItem";
176 |
177 | const CarouselPrevious = React.forwardRef<HTMLButtonElement, React.ComponentProps<typeof Button>>(
178 |   ({ className, variant = "outline", size = "icon", ...props }, ref) => {
179 |     const { orientation, scrollPrev, canScrollPrev } = useCarousel();
180 |
181 |     return (
182 |       <Button
183 |         ref={ref}

```

```

184 |         variant={variant}
185 |         size={size}
186 |         className={cn(
187 |           "absolute h-8 w-8 rounded-full",
188 |           orientation === "horizontal"
189 |             ? "-left-12 top-1/2 -translate-y-1/2"
190 |             : "-top-12 left-1/2 -translate-x-1/2 rotate-90",
191 |           className,
192 |         )}
193 |         disabled={!canScrollPrev}
194 |         onClick={scrollPrev}
195 |         {...props}
196 |       >
197 |         <ArrowLeft className="h-4 w-4" />
198 |         <span className="sr-only">Previous slide</span>
199 |       </Button>
200 |     );
201 |   },
202 | );
203 | CarouselPrevious.displayName = "CarouselPrevious";
204 |
205 | const CarouselNext = React.forwardRef<HTMLButtonElement, React.ComponentProps<typeof Button>>(
206 |   ({ className, variant = "outline", size = "icon", ...props }, ref) => {
207 |     const { orientation, scrollNext, canScrollNext } = useCarousel();
208 |
209 |     return (
210 |       <Button
211 |         ref={ref}
212 |         variant={variant}
213 |         size={size}
214 |         className={cn(
215 |           "absolute h-8 w-8 rounded-full",
216 |           orientation === "horizontal"
217 |             ? "-right-12 top-1/2 -translate-y-1/2"
218 |             : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90",
219 |           className,
220 |         )}
221 |         disabled={!canScrollNext}
222 |         onClick={scrollNext}
223 |         {...props}
224 |       >
225 |         <ArrowRight className="h-4 w-4" />
226 |         <span className="sr-only">Next slide</span>
227 |       </Button>
228 |     );
229 |   },
230 | );
231 | CarouselNext.displayName = "CarouselNext";
232 |
233 | export {
234 |   type CarouselApi,
235 |   Carousel,
236 |   CarouselContent,
237 |   CarouselItem,
238 |   CarouselPrevious,
239 |   CarouselNext,
240 | };

```

src/components/ui/chart.tsx

```
1 | import * as React from "react";
2 | import * as RechartsPrimitive from "recharts";
3 |
4 | import { cn } from "@lib/utlis";
5 |
6 | // Format: { THEME_NAME: CSS_SELECTOR }
7 | const THEMES = { light: "", dark: ".dark" } as const;
8 |
9 | export type ChartConfig = {
10 |   [k in string]: {
11 |     label?: React.ReactNode;
12 |     icon?: React.ComponentType;
13 |   } & (
14 |     | { color?: string; theme?: never }
15 |     | { color?: never; theme: Record<keyof typeof THEMES, string> }
16 |   );
17 | };
18 |
19 | type ChartContextProps = {
20 |   config: ChartConfig;
21 | };
22 |
23 | const ChartContext = React.createContext<ChartContextProps | null>(null);
24 |
25 | function useChart() {
26 |   const context = React.useContext(ChartContext);
27 |
28 |   if (!context) {
29 |     throw new Error("useChart must be used within a <ChartContainer />");
30 |   }
31 |
32 |   return context;
33 | }
34 |
35 | const ChartContainer = React.forwardRef<
36 |   HTMLDivElement,
37 |   React.ComponentProps<"div"> & {
38 |     config: ChartConfig;
39 |     children: React.ComponentProps<typeof RechartsPrimitive.ResponsiveContainer>["children"];
40 |   }
41 | >(({ id, className, children, config, ...props }, ref) => {
42 |   const uniqueId = React.useId();
43 |   const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`;
44 |
45 |   return (
46 |     <ChartContext.Provider value={{ config }}>
47 |       <div
48 |         data-chart={chartId}
49 |         ref={ref}
50 |         className={cn(
51 |           "flex aspect-video justify-center text-xs [&_.recharts-cartesian-axis-tick_text]:fill-muted-foreground [&_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50 [&_.recharts-curve.recharts-tooltip-cursor]:stroke-border [&_.recharts-dot[stroke='#fff']]:stroke-transparent [&_.recharts-layer]:outline-none [&_.recharts-polar-grid_line[stroke='#ccc']]:stroke-border [&_.recharts-radial-bar-background-sector]:fill-muted [&_.recharts-rectangle.recharts-tooltip-cursor]:fill-muted [&_.recharts-reference-line[stroke='#ccc']]:stroke-border [&_.recharts-sector[stroke='#fff']]:stroke-transparent [&_.recharts-sector]:outline-none [&_.recharts-surface]:outline-none",
52 |           className,
53 |         )}
54 |         {...props}
55 |       >
56 |         <ChartStyle id={chartId} config={config} />
57 |         <RechartsPrimitive.ResponsiveContainer>{children}</RechartsPrimitive.ResponsiveContainer>
58 |       </div>
59 |     </ChartContext.Provider>
60 |   );
61 | });
62 | ChartContainer.displayName = "Chart";
63 |
64 | const ChartStyle = ({ id, config }: { id: string; config: ChartConfig }) => {
65 |   const colorConfig = Object.entries(config).filter(([_, config]) => config.theme || config.color);
66 |
67 |   if (!colorConfig.length) {
68 |     return null;
69 |   }
70 |
71 |   return (
72 |     <style
73 |       dangerouslySetInnerHTML={{
74 |         __html: Object.entries(THEMES)
75 |           .map(
76 |             ([theme, prefix]) => `
77 | ${prefix} [data-chart=${id}] {
78 | ${colorConfig
79 |   .map(([key, itemConfig]) => {
80 |     const color = itemConfig.theme?.[theme as keyof typeof itemConfig.theme] || itemConfig.color;
81 |     return color ? `  --color-${key}: ${color};` : null;
82 |   })
83 |   .join("\n")}
84 | }
85 | `,
```

```

86 |         )
87 |         .join("\n"),
88 |     }}
89 |     />
90 | );
91 | };
92 |
93 | const ChartTooltip = RechartsPrimitive.Tooltip;
94 |
95 | const ChartTooltipContent = React.forwardRef<
96 |     HTMLDivElement,
97 |     React.ComponentProps<typeof RechartsPrimitive.Tooltip> &
98 |     React.ComponentProps<"div"> & {
99 |         hideLabel?: boolean;
100 |         hideIndicator?: boolean;
101 |         indicator?: "line" | "dot" | "dashed";
102 |         nameKey?: string;
103 |         labelKey?: string;
104 |     }
105 | > (
106 |     (
107 |         {
108 |             active,
109 |             payload,
110 |             className,
111 |             indicator = "dot",
112 |             hideLabel = false,
113 |             hideIndicator = false,
114 |             label,
115 |             labelFormatter,
116 |             labelClassName,
117 |             formatter,
118 |             color,
119 |             nameKey,
120 |             labelKey,
121 |         },
122 |         ref,
123 |     ) => {
124 |         const { config } = useChart();
125 |
126 |         const tooltipLabel = React.useMemo(() => {
127 |             if (hideLabel || !payload?.length) {
128 |                 return null;
129 |             }
130 |
131 |             const [item] = payload;
132 |             const key = `${labelKey || item?.dataKey || item?.name || "value"}`;
133 |             const itemConfig = getPayloadConfigFromPayload(config, item, key);
134 |             const value =
135 |                 !labelKey && typeof label === "string"
136 |                 ? config[label as keyof typeof config]?.label || label
137 |                 : itemConfig?.label;
138 |
139 |             if (labelFormatter) {
140 |                 return (
141 |                     <div className={cn("font-medium", labelClassName)}>{labelFormatter(value, payload)}</div>
142 |                 );
143 |             }
144 |
145 |             if (!value) {
146 |                 return null;
147 |             }
148 |
149 |             return <div className={cn("font-medium", labelClassName)}>{value}</div>;
150 |         }, [label, labelFormatter, payload, hideLabel, labelClassName, config, labelKey]);
151 |
152 |         if (!active || !payload?.length) {
153 |             return null;
154 |         }
155 |
156 |         const nestLabel = payload.length === 1 && indicator !== "dot";
157 |
158 |         return (
159 |             <div
160 |                 ref={ref}
161 |                 className={cn(
162 |                     "grid min-w-[8rem] items-start gap-1.5 rounded-lg border border-border/50 bg-background px-2.5 py-1.5 text-x
163 |                     s shadow-xl",
164 |                     className,
165 |                 )}
166 |             >
167 |                 {!nestLabel ? tooltipLabel : null}
168 |                 <div className="grid gap-1.5">
169 |                     {payload
170 |                         .filter((item) => item.type !== "none")
171 |                         .map((item, index) => {
172 |                             const key = `${nameKey || item.name || item.dataKey || "value"}`;
173 |                             const itemConfig = getPayloadConfigFromPayload(config, item, key);
174 |                             const indicatorColor = color || item.payload.fill || item.color;
175 |
176 |                             return (
177 |                                 <div

```

```

178 |         className={cn(
179 |             "flex w-full flex-wrap items-stretch gap-2 [>svg]:h-2.5 [>svg]:w-2.5 [>svg]:text-muted-foreground",
180 |             nd",
181 |             indicator === "dot" && "items-center",
182 |         )}
183 |     >
184 |     {formatter && item?.value !== undefined && item.name ? (
185 |         formatter(item.value, item.name, item, index, item.payload)
186 |     ) : (
187 |         <>
188 |             {itemConfig?.icon ? (
189 |                 <itemConfig.icon />
190 |             ) : (
191 |                 !hideIndicator && (
192 |                     <div
193 |                         className={cn(
194 |                             "shrink-0 rounded-[2px] border-(--color-border) bg-(--color-bg)",
195 |                             {
196 |                                 "h-2.5 w-2.5": indicator === "dot",
197 |                                 "w-1": indicator === "line",
198 |                                 "w-0 border-[1.5px] border-dashed bg-transparent":
199 |                                     indicator === "dashed",
200 |                                 "my-0.5": nestLabel && indicator === "dashed",
201 |                             },
202 |                         )}
203 |                         style={
204 |                             {
205 |                                 "--color-bg": indicatorColor,
206 |                                 "--color-border": indicatorColor,
207 |                             } as React.CSSProperties
208 |                         }
209 |                     />
210 |                 )}
211 |             <div
212 |                 className={cn(
213 |                     "flex flex-1 justify-between leading-none",
214 |                     nestLabel ? "items-end" : "items-center",
215 |                 )}
216 |             >
217 |                 <div className="grid gap-1.5">
218 |                     {nestLabel ? tooltipLabel : null}
219 |                     <span className="text-muted-foreground">
220 |                         {itemConfig?.label || item.name}
221 |                     </span>
222 |                 </div>
223 |                 {item.value && (
224 |                     <span className="font-mono font-medium tabular-nums text-foreground">
225 |                         {item.value.toLocaleString()}
226 |                     </span>
227 |                 )}
228 |             </div>
229 |         </>
230 |     )}
231 | </div>
232 | );
233 | }}}
234 | </div>
235 | </div>
236 | );
237 | },
238 | );
239 | ChartTooltipContent.displayName = "ChartTooltip";
240 |
241 | const ChartLegend = RechartsPrimitive.Legend;
242 |
243 | const ChartLegendContent = React.forwardRef<
244 |     HTMLDivElement,
245 |     React.ComponentProps<"div"> &
246 |         Pick<RechartsPrimitive.LegendProps, "payload" | "verticalAlign"> & {
247 |             hideIcon?: boolean;
248 |             nameKey?: string;
249 |         }
250 | >(({ className, hideIcon = false, payload, verticalAlign = "bottom", nameKey }, ref) => {
251 |     const { config } = useChart();
252 |
253 |     if (!payload?.length) {
254 |         return null;
255 |     }
256 |
257 |     return (
258 |         <div
259 |             ref={ref}
260 |             className={cn(
261 |                 "flex items-center justify-center gap-4",
262 |                 verticalAlign === "top" ? "pb-3" : "pt-3",
263 |                 className,
264 |             )}
265 |         >
266 |             {payload
267 |                 .filter((item) => item.type !== "none")
268 |                 .map((item) => {
269 |                     const key = `${nameKey || item.dataKey || "value"}`;

```

```

270 |         const itemConfig = getPayloadConfigFromPayload(config, item, key);
271 |
272 |         return (
273 |             <div
274 |                 key={item.value}
275 |                 className={cn(
276 |                     "flex items-center gap-1.5 [>svg]:h-3 [>svg]:w-3 [>svg]:text-muted-foreground",
277 |                 )}
278 |             >
279 |                 {itemConfig?.icon && !hideIcon ? (
280 |                     <itemConfig.icon />
281 |                 ) : (
282 |                     <div
283 |                         className="h-2 w-2 shrink-0 rounded-[2px]"
284 |                         style={{
285 |                             backgroundColor: item.color,
286 |                         }}
287 |                     />
288 |                 )}
289 |                 {itemConfig?.label}
290 |             </div>
291 |         );
292 |     }}
293 | </div>
294 | );
295 | });
296 | ChartLegendContent.displayName = "ChartLegend";
297 |
298 | // Helper to extract item config from a payload.
299 | function getPayloadConfigFromPayload(config: ChartConfig, payload: unknown, key: string) {
300 |     if (typeof payload !== "object" || payload === null) {
301 |         return undefined;
302 |     }
303 |
304 |     const payloadPayload =
305 |         "payload" in payload && typeof payload.payload === "object" && payload.payload !== null
306 |         ? payload.payload
307 |         : undefined;
308 |
309 |     let configLabelKey: string = key;
310 |
311 |     if (key in payload && typeof payload[key as keyof typeof payload] === "string") {
312 |         configLabelKey = payload[key as keyof typeof payload] as string;
313 |     } else if (
314 |         payloadPayload &&
315 |         key in payloadPayload &&
316 |         typeof payloadPayload[key as keyof typeof payloadPayload] === "string"
317 |     ) {
318 |         configLabelKey = payloadPayload[key as keyof typeof payloadPayload] as string;
319 |     }
320 |
321 |     return configLabelKey in config ? config[configLabelKey] : config[key as keyof typeof config];
322 | }
323 |
324 | export {
325 |     ChartContainer,
326 |     ChartTooltip,
327 |     ChartTooltipContent,
328 |     ChartLegend,
329 |     ChartLegendContent,
330 |     ChartStyle,
331 | };

```

src/components/ui/checkbox.tsx

```
1 | import * as React from "react";
2 | import * as CheckboxPrimitive from "@radix-ui/react-checkbox";
3 | import { Check } from "lucide-react";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const Checkbox = React.forwardRef<
8 |   React.ElementRef<typeof CheckboxPrimitive.Root>,
9 |   React.ComponentPropsWithoutRef<typeof CheckboxPrimitive.Root>
10 | >(({ className, ...props }, ref) => {
11 |   <CheckboxPrimitive.Root
12 |     ref={ref}
13 |     className={cn(
14 |       "grid place-content-center peer h-4 w-4 shrink-0 rounded-sm border border-primary shadow cursor-pointer focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50 data-[state=checked]:bg-primary data-[state=checked]:text-primary-foreground",
15 |       className,
16 |     )}
17 |     {...props}
18 |   >
19 |     <CheckboxPrimitive.Indicator className={cn("grid place-content-center text-current")}>
20 |       <Check className="h-4 w-4" />
21 |     </CheckboxPrimitive.Indicator>
22 |   </CheckboxPrimitive.Root>
23 | );
24 | Checkbox.displayName = CheckboxPrimitive.Root.displayName;
25 |
26 | export { Checkbox };
```

src/components/ui/collapsible.tsx

```
1 | "use client";
2 |
3 | import * as CollapsiblePrimitive from "@radix-ui/react-collapsible";
4 |
5 | const Collapsible = CollapsiblePrimitive.Root;
6 |
7 | const CollapsibleTrigger = CollapsiblePrimitive.CollapsibleTrigger;
8 |
9 | const CollapsibleContent = CollapsiblePrimitive.CollapsibleContent;
10 |
11 | export { Collapsible, CollapsibleTrigger, CollapsibleContent };
```

src/components/ui/command.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import { type DialogProps } from "@radix-ui/react-dialog";
5 | import { Command as CommandPrimitive } from "cmdk";
6 | import { Search } from "lucide-react";
7 |
8 | import { cn } from "@lib/utils";
9 | import { Dialog, DialogContent } from "@components/ui/dialog";
10 |
11 | const Command = React.forwardRef<
12 |   React.ElementRef<typeof CommandPrimitive>,
13 |   React.ComponentPropsWithoutRef<typeof CommandPrimitive>
14 | >(({ className, ...props }, ref) => (
15 |   <CommandPrimitive
16 |     ref={ref}
17 |     className={cn(
18 |       "flex h-full w-full flex-col overflow-hidden rounded-md bg-popover text-popover-foreground",
19 |       className,
20 |     )}
21 |     {...props}
22 |   />
23 | ));
24 | Command.displayName = CommandPrimitive.displayName;
25 |
26 | const CommandDialog = ({ children, ...props }: DialogProps) => {
27 |   return (
28 |     <Dialog {...props}>
29 |       <DialogContent className="overflow-hidden p-0">
30 |         <Command className="[&_cmdk-group-heading]:px-2 [&_cmdk-group-heading]:font-medium [&_cmdk-group-heading]:text-muted-foreground [&_cmdk-group]:not([hidden])_~[&_cmdk-group]:pt-0 [&_cmdk-group]:px-2 [&_cmdk-input-wrapper]_svg:h-5 [&_cmdk-input-wrapper]_svg:w-5 [&_cmdk-input]:h-12 [&_cmdk-item]:px-2 [&_cmdk-item]:py-3 [&_cmdk-item]_svg:h-5 [&_cmdk-item]_svg:w-5">
31 |           {children}
32 |         </Command>
33 |       </DialogContent>
34 |     </Dialog>
35 |   );
36 | };
37 |
38 | const CommandInput = React.forwardRef<
39 |   React.ElementRef<typeof CommandPrimitive.Input>,
40 |   React.ComponentPropsWithoutRef<typeof CommandPrimitive.Input>
41 | >(({ className, ...props }, ref) => (
42 |   <div className="flex items-center border-b px-3 cmdk-input-wrapper="
43 |     <Search className="mr-2 h-4 w-4 shrink-0 opacity-50" />
44 |     <CommandPrimitive.Input
45 |       ref={ref}
46 |       className={cn(
47 |         "flex h-10 w-full rounded-md bg-transparent py-3 text-sm outline-none placeholder:text-muted-foreground disabled:cursor-not-allowed disabled:opacity-50",
48 |         className,
49 |       )}
50 |       {...props}
51 |     />
52 |   </div>
53 | ));
54 |
55 | CommandInput.displayName = CommandPrimitive.Input.displayName;
56 |
57 | const CommandList = React.forwardRef<
58 |   React.ElementRef<typeof CommandPrimitive.List>,
59 |   React.ComponentPropsWithoutRef<typeof CommandPrimitive.List>
60 | >(({ className, ...props }, ref) => (
61 |   <CommandPrimitive.List
62 |     ref={ref}
63 |     className={cn("max-h-[300px] overflow-y-auto overflow-x-hidden", className)}
64 |     {...props}
65 |   />
66 | ));
67 |
68 | CommandList.displayName = CommandPrimitive.List.displayName;
69 |
70 | const CommandEmpty = React.forwardRef<
71 |   React.ElementRef<typeof CommandPrimitive.Empty>,
72 |   React.ComponentPropsWithoutRef<typeof CommandPrimitive.Empty>
73 | >((props, ref) => (
74 |   <CommandPrimitive.Empty ref={ref} className="py-6 text-center text-sm" {...props} />
75 | ));
76 |
77 | CommandEmpty.displayName = CommandPrimitive.Empty.displayName;
78 |
79 | const CommandGroup = React.forwardRef<
80 |   React.ElementRef<typeof CommandPrimitive.Group>,
81 |   React.ComponentPropsWithoutRef<typeof CommandPrimitive.Group>
82 | >(({ className, ...props }, ref) => (
83 |   <CommandPrimitive.Group
84 |     ref={ref}
85 |     className={cn(
86 |       "overflow-hidden p-1 text-foreground [&_cmdk-group-heading]:px-2 [&_cmdk-group-heading]:py-1.5 [&_cmdk-grou
```

```

87 |         p-heading]]:text-xs [&[_cmdk-group-heading]]:font-medium [&[_cmdk-group-heading]]:text-muted-foreground",
88 |         className,
89 |       {...props}
90 |     />
91 |   ));
92 |
93 | CommandGroup.displayName = CommandPrimitive.Group.displayName;
94 |
95 | const CommandSeparator = React.forwardRef<
96 |   React.ElementRef<typeof CommandPrimitive.Separator>,
97 |   React.ComponentPropsWithoutRef<typeof CommandPrimitive.Separator>
98 | >(({ className, ...props }, ref) => (
99 |   <CommandPrimitive.Separator
100 |     ref={ref}
101 |     className={cn("-mx-1 h-px bg-border", className)}
102 |     {...props}
103 |   />
104 | ));
105 | CommandSeparator.displayName = CommandPrimitive.Separator.displayName;
106 |
107 | const CommandItem = React.forwardRef<
108 |   React.ElementRef<typeof CommandPrimitive.Item>,
109 |   React.ComponentPropsWithoutRef<typeof CommandPrimitive.Item>
110 | >(({ className, ...props }, ref) => (
111 |   <CommandPrimitive.Item
112 |     ref={ref}
113 |     className={cn(
114 |       "relative flex cursor-default gap-2 select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none data-[disabled=true]:pointer-events-none data-[selected=true]:bg-accent data-[selected=true]:text-accent-foreground data-[disabled=true]:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
115 |       className,
116 |     )}
117 |     {...props}
118 |   />
119 | ));
120 |
121 | CommandItem.displayName = CommandPrimitive.Item.displayName;
122 |
123 | const CommandShortcut = ({ className, ...props }: React.HTMLAttributes<HTMLSpanElement>) => {
124 |   return (
125 |     <span
126 |       className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
127 |       {...props}
128 |     />
129 |   );
130 | };
131 | CommandShortcut.displayName = "CommandShortcut";
132 |
133 | export {
134 |   Command,
135 |   CommandDialog,
136 |   CommandInput,
137 |   CommandList,
138 |   CommandEmpty,
139 |   CommandGroup,
140 |   CommandItem,
141 |   CommandShortcut,
142 |   CommandSeparator,
143 | };

```

src/components/ui/context-menu.tsx

```
1 | import * as React from "react";
2 | import * as ContextMenuPrimitive from "@radix-ui/react-context-menu";
3 | import { Check, ChevronRight, Circle } from "lucide-react";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const ContextMenu = ContextMenuPrimitive.Root;
8 |
9 | const ContextMenuTrigger = ContextMenuPrimitive.Trigger;
10 |
11 | const ContextMenuGroup = ContextMenuPrimitive.Group;
12 |
13 | const ContextMenuPortal = ContextMenuPrimitive.Portal;
14 |
15 | const ContextMenuSub = ContextMenuPrimitive.Sub;
16 |
17 | const ContextMenuRadioGroup = ContextMenuPrimitive.RadioGroup;
18 |
19 | const ContextMenuSubTrigger = React.forwardRef<
20 |   React.ElementRef<typeof ContextMenuPrimitive.SubTrigger>,
21 |   React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.SubTrigger> & {
22 |     inset?: boolean;
23 |   }
24 | >(({ className, inset, children, ...props }, ref) => (
25 |   <ContextMenuPrimitive.SubTrigger
26 |     ref={ref}
27 |     className={cn(
28 |       "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent focus:
29 |       text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
30 |       inset && "pl-8",
31 |       className,
32 |     )}
33 |     {...props}
34 |   >
35 |     {children}
36 |     <ChevronRight className="ml-auto h-4 w-4" />
37 |   </ContextMenuPrimitive.SubTrigger>
38 | ));
39 | ContextMenuSubTrigger.displayName = ContextMenuPrimitive.SubTrigger.displayName;
40 |
41 | const ContextMenuSubContent = React.forwardRef<
42 |   React.ElementRef<typeof ContextMenuPrimitive.SubContent>,
43 |   React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.SubContent>
44 | >(({ className, ...props }, ref) => (
45 |   <ContextMenuPrimitive.SubContent
46 |     ref={ref}
47 |     className={cn(
48 |       "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg data-[stat
49 |       e=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data
50 |       -[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:s
51 |       lide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-(--radix
52 |       -context-menu-content-transform-origin)",
53 |       className,
54 |     )}
55 |     {...props}
56 |   />
57 | ));
58 | ContextMenuSubContent.displayName = ContextMenuPrimitive.SubContent.displayName;
59 |
60 | const ContextMenuContent = React.forwardRef<
61 |   React.ElementRef<typeof ContextMenuPrimitive.Content>,
62 |   React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.Content>
63 | >(({ className, ...props }, ref) => (
64 |   <ContextMenuPrimitive.Portal>
65 |     <ContextMenuPrimitive.Content
66 |       ref={ref}
67 |       className={cn(
68 |         "z-50 max-h-(--radix-context-menu-content-available-height) min-w-[8rem] overflow-y-auto overflow-x-hidden rou
69 |         nded-md border bg-popover p-1 text-popover-foreground shadow-md data-[state=open]:animate-in data-[state=closed]:a
70 |         nimate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=
71 |         open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:s
72 |         lide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-(--radix-context-menu-content-transform-origin)"
73 |         ,
74 |         className,
75 |       )}
76 |       {...props}
77 |     />
78 |   </ContextMenuPrimitive.Portal>
79 | ));
80 | ContextMenuContent.displayName = ContextMenuPrimitive.Content.displayName;
81 |
82 | const ContextMenuItem = React.forwardRef<
83 |   React.ElementRef<typeof ContextMenuPrimitive.Item>,
84 |   React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.Item> & {
85 |     inset?: boolean;
86 |   }
87 | >(({ className, inset, ...props }, ref) => (
88 |   <ContextMenuPrimitive.Item
89 |     ref={ref}
90 |     className={cn(
91 |       "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent focus:
92 |       text-accent-foreground data-[disabled]:pointer-events-none data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
93 |       inset && "pl-8",
94 |       className,
95 |     )}
96 |     {...props}
97 |   >
98 |     {props.children}
99 |   </ContextMenuPrimitive.Item>
100 | ));
101 | ContextMenuItem.displayName = ContextMenuPrimitive.Item.displayName;
```

```

81 |     "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-acce
82 |     nt focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
83 |     className,
84 |   )}
85 |   {...props}
86 | />
87 | ));
88 | ContextMenuItem.displayName = ContextMenuPrimitive.Item.displayName;
89 |
90 | const ContextMenuCheckboxItem = React.forwardRef<
91 |   React.ElementRef<typeof ContextMenuPrimitive.CheckboxItem>,
92 |   React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.CheckboxItem>
93 | >(({ className, children, checked, ...props }, ref) => (
94 |   <ContextMenuPrimitive.CheckboxItem
95 |     ref={ref}
96 |     className={cn(
97 |       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:bg
98 |       -accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
99 |       className,
100 |     )}
101 |     checked={checked}
102 |     {...props}
103 |   >
104 |     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
105 |       <ContextMenuItemIndicator>
106 |         <Check className="h-4 w-4" />
107 |       </ContextMenuItemIndicator>
108 |     </span>
109 |     {children}
110 |   </ContextMenuPrimitive.CheckboxItem>
111 | ));
112 | ContextMenuCheckboxItem.displayName = ContextMenuPrimitive.CheckboxItem.displayName;
113 |
114 | const ContextMenuRadioItem = React.forwardRef<
115 |   React.ElementRef<typeof ContextMenuPrimitive.RadioItem>,
116 |   React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.RadioItem>
117 | >(({ className, children, ...props }, ref) => (
118 |   <ContextMenuPrimitive.RadioItem
119 |     ref={ref}
120 |     className={cn(
121 |       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:bg
122 |       -accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
123 |       className,
124 |     )}
125 |     {...props}
126 |   >
127 |     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
128 |       <ContextMenuItemIndicator>
129 |         <Circle className="h-4 w-4 fill-current" />
130 |       </ContextMenuItemIndicator>
131 |     </span>
132 |     {children}
133 |   </ContextMenuPrimitive.RadioItem>
134 | ));
135 | ContextMenuRadioItem.displayName = ContextMenuPrimitive.RadioItem.displayName;
136 |
137 | const ContextMenuLabel = React.forwardRef<
138 |   React.ElementRef<typeof ContextMenuPrimitive.Label>,
139 |   React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.Label> & {
140 |     inset?: boolean;
141 |   }
142 | >(({ className, inset, ...props }, ref) => (
143 |   <ContextMenuPrimitive.Label
144 |     ref={ref}
145 |     className={cn("px-2 py-1.5 text-sm font-semibold text-foreground", inset && "pl-8", className)}
146 |     {...props}
147 |   />
148 | ));
149 | ContextMenuLabel.displayName = ContextMenuPrimitive.Label.displayName;
150 |
151 | const ContextMenuSeparator = React.forwardRef<
152 |   React.ElementRef<typeof ContextMenuPrimitive.Separator>,
153 |   React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.Separator>
154 | >(({ className, ...props }, ref) => (
155 |   <ContextMenuPrimitive.Separator
156 |     ref={ref}
157 |     className={cn("-mx-1 my-1 h-px bg-border", className)}
158 |     {...props}
159 |   />
160 | ));
161 | ContextMenuSeparator.displayName = ContextMenuPrimitive.Separator.displayName;
162 |
163 | const ContextMenuShortcut = ({ className, ...props }: React.HTMLAttributes<HTMLSpanElement>) => {
164 |   return (
165 |     <span
166 |       className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
167 |       {...props}
168 |     />
169 |   );
170 | };
171 | ContextMenuShortcut.displayName = "ContextMenuShortcut";

```

```
171 | export {
172 |   ContextMenu,
173 |   ContextMenuTrigger,
174 |   ContextMenuContent,
175 |   ContextMenuItem,
176 |   ContextMenuCheckboxItem,
177 |   ContextMenuRadioItem,
178 |   ContextMenuLabel,
179 |   ContextMenuSeparator,
180 |   ContextMenuShortcut,
181 |   ContextMenuGroup,
182 |   ContextMenuPortal,
183 |   ContextMenuSub,
184 |   ContextMenuSubContent,
185 |   ContextMenuSubTrigger,
186 |   ContextMenuRadioGroup,
187 | };
```

src/components/ui/dialog.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as DialogPrimitive from "@radix-ui/react-dialog";
5 | import { X } from "lucide-react";
6 |
7 | import { cn } from "@lib/utls";
8 |
9 | const Dialog = DialogPrimitive.Root;
10 |
11 | const DialogTrigger = DialogPrimitive.Trigger;
12 |
13 | const DialogPortal = DialogPrimitive.Portal;
14 |
15 | const DialogClose = DialogPrimitive.Close;
16 |
17 | const DialogOverlay = React.forwardRef<
18 |   React.ElementRef<typeof DialogPrimitive.Overlay>,
19 |   React.ComponentPropsWithoutRef<typeof DialogPrimitive.Overlay>
20 | >(({ className, ...props }, ref) => (
21 |   <DialogPrimitive.Overlay
22 |     ref={ref}
23 |     className={cn(
24 |       "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]
25 |       ):fade-out-0 data-[state=open]:fade-in-0",
26 |       className,
27 |     )}
28 |     {...props}
29 |   />
30 | ));
31 | DialogOverlay.displayName = DialogPrimitive.Overlay.displayName;
32 |
33 | const DialogContent = React.forwardRef<
34 |   React.ElementRef<typeof DialogPrimitive.Content>,
35 |   React.ComponentPropsWithoutRef<typeof DialogPrimitive.Content>
36 | >(({ className, children, ...props }, ref) => (
37 |   <DialogPortal>
38 |     <DialogOverlay />
39 |     <DialogPrimitive.Content
40 |       ref={ref}
41 |       className={cn(
42 |         "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border bg-ba
43 |         ckground p-6 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 sm:rounded-
44 |         lg",
45 |         className,
46 |       )}
47 |       {...props}
48 |     >
49 |       {children}
50 |       <DialogPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background cursor-poi
51 |       nter transition-opacity hover:opacity-100 focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disa
52 |       bled:pointer-events-none data-[state=open]:bg-accent data-[state=open]:text-muted-foreground">
53 |         <X className="h-4 w-4" />
54 |         <span className="sr-only">Close</span>
55 |       </DialogPrimitive.Close>
56 |     </DialogPrimitive.Content>
57 |   </DialogPortal>
58 | ));
59 | DialogContent.displayName = DialogPrimitive.Content.displayName;
60 |
61 | const DialogHeader = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => (
62 |   <div className={cn("flex flex-col space-y-1.5 text-center sm:text-left", className)} {...props} />
63 | );
64 | DialogHeader.displayName = "DialogHeader";
65 |
66 | const DialogFooter = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => (
67 |   <div
68 |     className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
69 |     {...props}
70 |   />
71 | );
72 | DialogFooter.displayName = "DialogFooter";
73 |
74 | const DialogTitle = React.forwardRef<
75 |   React.ElementRef<typeof DialogPrimitive.Title>,
76 |   React.ComponentPropsWithoutRef<typeof DialogPrimitive.Title>
77 | >(({ className, ...props }, ref) => (
78 |   <DialogPrimitive.Title
79 |     ref={ref}
80 |     className={cn("text-lg font-semibold leading-none tracking-tight", className)}
81 |     {...props}
82 |   />
83 | ));
84 | DialogTitle.displayName = DialogPrimitive.Title.displayName;
85 |
86 | const DialogDescription = React.forwardRef<
87 |   React.ElementRef<typeof DialogPrimitive.Description>,
88 |   React.ComponentPropsWithoutRef<typeof DialogPrimitive.Description>
89 | >(({ className, ...props }, ref) => (
90 |   <DialogPrimitive.Description
91 |     ref={ref}
92 |     className={cn("text-sm text-muted-foreground", className)}
93 |     {...props}
94 |   />
95 | ));
96 | DialogDescription.displayName = DialogPrimitive.Description.displayName;
```

```

85 | <DialogPrimitive.Description
86 |   ref={ref}
87 |   className={cn("text-sm text-muted-foreground", className)}
88 |   {...props}
89 | />
90 | ));
91 | DialogDescription.displayName = DialogPrimitive.Description.displayName;
92 |
93 | export {
94 |   Dialog,
95 |   DialogPortal,
96 |   DialogOverlay,
97 |   DialogTrigger,
98 |   DialogClose,
99 |   DialogContent,
100 |   DialogHeader,
101 |   DialogFooter,
102 |   DialogTitle,
103 |   DialogDescription,
104 | };

```

src/components/ui/drawer.tsx

```
1 | import * as React from "react";
2 | import { Drawer as DrawerPrimitive } from "vaul";
3 |
4 | import { cn } from "@lib/utills";
5 |
6 | const Drawer = ({
7 |   shouldScaleBackground = true,
8 |   ...props
9 | }: React.ComponentProps<typeof DrawerPrimitive.Root>) => (
10 |   <DrawerPrimitive.Root shouldScaleBackground={shouldScaleBackground} {...props} />
11 | );
12 | Drawer.displayName = "Drawer";
13 |
14 | const DrawerTrigger = DrawerPrimitive.Trigger;
15 |
16 | const DrawerPortal = DrawerPrimitive.Portal;
17 |
18 | const DrawerClose = DrawerPrimitive.Close;
19 |
20 | const DrawerOverlay = React.forwardRef<
21 |   React.ElementRef<typeof DrawerPrimitive.Overlay>,
22 |   React.ComponentPropsWithoutRef<typeof DrawerPrimitive.Overlay>
23 | >(({ className, ...props }, ref) => (
24 |   <DrawerPrimitive.Overlay
25 |     ref={ref}
26 |     className={cn("fixed inset-0 z-50 bg-black/80", className)}
27 |     {...props}
28 |   />
29 | ));
30 | DrawerOverlay.displayName = DrawerPrimitive.Overlay.displayName;
31 |
32 | const DrawerContent = React.forwardRef<
33 |   React.ElementRef<typeof DrawerPrimitive.Content>,
34 |   React.ComponentPropsWithoutRef<typeof DrawerPrimitive.Content>
35 | >(({ className, children, ...props }, ref) => (
36 |   <DrawerPortal>
37 |     <DrawerOverlay />
38 |     <DrawerPrimitive.Content
39 |       ref={ref}
40 |       className={cn(
41 |         "fixed inset-x-0 bottom-0 z-50 mt-24 flex h-auto flex-col rounded-t-[10px] border bg-background",
42 |         className,
43 |       )}
44 |       {...props}
45 |     >
46 |       <div className="mx-auto mt-4 h-2 w-[100px] rounded-full bg-muted" />
47 |       {children}
48 |     </DrawerPrimitive.Content>
49 |   </DrawerPortal>
50 | );
51 | DrawerContent.displayName = "DrawerContent";
52 |
53 | const DrawerHeader = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => (
54 |   <div className={cn("grid gap-1.5 p-4 text-center sm:text-left", className)} {...props} />
55 | );
56 | DrawerHeader.displayName = "DrawerHeader";
57 |
58 | const DrawerFooter = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => (
59 |   <div className={cn("mt-auto flex flex-col gap-2 p-4", className)} {...props} />
60 | );
61 | DrawerFooter.displayName = "DrawerFooter";
62 |
63 | const DrawerTitle = React.forwardRef<
64 |   React.ElementRef<typeof DrawerPrimitive.Title>,
65 |   React.ComponentPropsWithoutRef<typeof DrawerPrimitive.Title>
66 | >(({ className, ...props }, ref) => (
67 |   <DrawerPrimitive.Title
68 |     ref={ref}
69 |     className={cn("text-lg font-semibold leading-none tracking-tight", className)}
70 |     {...props}
71 |   />
72 | ));
73 | DrawerTitle.displayName = DrawerPrimitive.Title.displayName;
74 |
75 | const DrawerDescription = React.forwardRef<
76 |   React.ElementRef<typeof DrawerPrimitive.Description>,
77 |   React.ComponentPropsWithoutRef<typeof DrawerPrimitive.Description>
78 | >(({ className, ...props }, ref) => (
79 |   <DrawerPrimitive.Description
80 |     ref={ref}
81 |     className={cn("text-sm text-muted-foreground", className)}
82 |     {...props}
83 |   />
84 | ));
85 | DrawerDescription.displayName = DrawerPrimitive.Description.displayName;
86 |
87 | export {
88 |   Drawer,
89 |   DrawerPortal,
90 |   DrawerOverlay,
```

```
91 | DrawerTrigger,  
92 | DrawerClose,  
93 | DrawerContent,  
94 | DrawerHeader,  
95 | DrawerFooter,  
96 | DrawerTitle,  
97 | DrawerDescription,  
98 | };
```

src/components/ui/dropdown-menu.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu";
5 | import { Check, ChevronRight, Circle } from "lucide-react";
6 |
7 | import { cn } from "@lib/utls";
8 |
9 | const DropdownMenu = DropdownMenuPrimitive.Root;
10 |
11 | const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger;
12 |
13 | const DropdownMenuGroup = DropdownMenuPrimitive.Group;
14 |
15 | const DropdownMenuPortal = DropdownMenuPrimitive.Portal;
16 |
17 | const DropdownMenuSub = DropdownMenuPrimitive.Sub;
18 |
19 | const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup;
20 |
21 | const DropdownMenuSubTrigger = React.forwardRef<
22 |   React.ElementRef<typeof DropdownMenuPrimitive.SubTrigger>,
23 |   React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.SubTrigger> & {
24 |     inset?: boolean;
25 |   }
26 | >(({ className, inset, children, ...props }, ref) => (
27 |   <DropdownMenuPrimitive.SubTrigger
28 |     ref={ref}
29 |     className={cn(
30 |       "flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent
31 |       data-[state=open]:bg-accent [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
32 |       inset && "pl-8",
33 |       className,
34 |     )}
35 |     {...props}
36 |   >
37 |     {children}
38 |     <ChevronRight className="ml-auto" />
39 |   </DropdownMenuPrimitive.SubTrigger>
40 | ));
41 |
42 | DropdownMenuSubTrigger.displayName = DropdownMenuPrimitive.SubTrigger.displayName;
43 |
44 | const DropdownMenuSubContent = React.forwardRef<
45 |   React.ElementRef<typeof DropdownMenuPrimitive.SubContent>,
46 |   React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.SubContent>
47 | >(({ className, ...props }, ref) => (
48 |   <DropdownMenuPrimitive.SubContent
49 |     ref={ref}
50 |     className={cn(
51 |       "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg data-[stat
52 |       e=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data
53 |       -[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:s
54 |       lide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin(--radix
55 |       -dropdown-menu-content-transform-origin)",
56 |       className,
57 |     )}
58 |     {...props}
59 |   >
60 |   </>
61 | ));
62 |
63 | DropdownMenuSubContent.displayName = DropdownMenuPrimitive.SubContent.displayName;
64 |
65 | const DropdownMenuContent = React.forwardRef<
66 |   React.ElementRef<typeof DropdownMenuPrimitive.Content>,
67 |   React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Content>
68 | >(({ className, sideOffset = 4, ...props }, ref) => (
69 |   <DropdownMenuPrimitive.Portal>
70 |     <DropdownMenuPrimitive.Content
71 |       ref={ref}
72 |       sideOffset={sideOffset}
73 |       className={cn(
74 |         "z-50 max-h-[var(--radix-dropdown-menu-content-available-height)] min-w-[8rem] overflow-y-auto overflow-x-hidd
75 |         en rounded-md border bg-popover p-1 text-popover-foreground shadow-md",
76 |         "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]
77 |         :fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 dat
78 |         a-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2
79 |         origin(--radix-dropdown-menu-content-transform-origin)",
80 |         className,
81 |       )}
82 |       {...props}
83 |     >
84 |     </DropdownMenuPrimitive.Content>
85 |   </DropdownMenuPrimitive.Portal>
86 | ));
87 |
88 | DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName;
89 |
90 | const DropdownMenuItem = React.forwardRef<
91 |   React.ElementRef<typeof DropdownMenuPrimitive.Item>,
92 |   React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Item> & {
93 |     inset?: boolean;
94 |   }
95 | >(({ className, inset, ...props }, ref) => (
```

```

82 | <DropdownMenuPrimitive.Item
83 |   ref={ref}
84 |   className={cn(
85 |     "relative flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-none transit
86 |     ion-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacit
87 |     y-50 [&svg]:size-4 [&svg]:shrink-0",
88 |     inset && "pl-8",
89 |     className,
90 |   )}
91 |   {...props}
92 | />
93 | ));
94 | DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName;
95 |
96 | const DropdownMenuCheckboxItem = React.forwardRef<
97 |   React.ElementRef<typeof DropdownMenuPrimitive.CheckboxItem>,
98 |   React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.CheckboxItem>
99 | >(({ className, children, checked, ...props }, ref) => {
100 |   <DropdownMenuPrimitive.CheckboxItem
101 |     ref={ref}
102 |     className={cn(
103 |       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none transiti
104 |       on-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity
105 |       -50",
106 |       className,
107 |     )}
108 |     checked={checked}
109 |     {...props}
110 |   >
111 |     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
112 |       <DropdownMenuPrimitive.ItemIndicator>
113 |         <Check className="h-4 w-4" />
114 |       </DropdownMenuPrimitive.ItemIndicator>
115 |     </span>
116 |     {children}
117 |   </DropdownMenuPrimitive.CheckboxItem>
118 | ));
119 | DropdownMenuCheckboxItem.displayName = DropdownMenuPrimitive.CheckboxItem.displayName;
120 |
121 | const DropdownMenuRadioItem = React.forwardRef<
122 |   React.ElementRef<typeof DropdownMenuPrimitive.RadioItem>,
123 |   React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.RadioItem>
124 | >(({ className, children, ...props }, ref) => {
125 |   <DropdownMenuPrimitive.RadioItem
126 |     ref={ref}
127 |     className={cn(
128 |       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none transiti
129 |       on-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity
130 |       -50",
131 |       className,
132 |     )}
133 |     {...props}
134 |   >
135 |     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
136 |       <DropdownMenuPrimitive.ItemIndicator>
137 |         <Circle className="h-2 w-2 fill-current" />
138 |       </DropdownMenuPrimitive.ItemIndicator>
139 |     </span>
140 |     {children}
141 |   </DropdownMenuPrimitive.RadioItem>
142 | ));
143 | DropdownMenuRadioItem.displayName = DropdownMenuPrimitive.RadioItem.displayName;
144 |
145 | const DropdownMenuLabel = React.forwardRef<
146 |   React.ElementRef<typeof DropdownMenuPrimitive.Label>,
147 |   React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Label> & {
148 |     inset?: boolean;
149 |   }
150 | >(({ className, inset, ...props }, ref) => {
151 |   <DropdownMenuPrimitive.Label
152 |     ref={ref}
153 |     className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8", className)}
154 |     {...props}
155 |   />
156 | ));
157 | DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName;
158 |
159 | const DropdownMenuSeparator = React.forwardRef<
160 |   React.ElementRef<typeof DropdownMenuPrimitive.Separator>,
161 |   React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Separator>
162 | >(({ className, ...props }, ref) => {
163 |   <DropdownMenuPrimitive.Separator
164 |     ref={ref}
165 |     className={cn("-mx-1 my-1 h-px bg-muted", className)}
166 |     {...props}
167 |   />
168 | ));
169 | DropdownMenuSeparator.displayName = DropdownMenuPrimitive.Separator.displayName;
170 |
171 | const DropdownMenuShortcut = ({ className, ...props }: React.HTMLAttributes<HTMLSpanElement>) => {
172 |   return (
173 |     <span className={cn("ml-auto text-xs tracking-widest opacity-60", className)} {...props} />
174 |   );

```

```
169 | };
170 | DropdownMenuShortcut.displayName = "DropdownMenuShortcut";
171 |
172 | export {
173 |   DropdownMenu,
174 |   DropdownMenuTrigger,
175 |   DropdownMenuContent,
176 |   DropdownMenuItem,
177 |   DropdownMenuCheckboxItem,
178 |   DropdownMenuRadioItem,
179 |   DropdownMenuLabel,
180 |   DropdownMenuSeparator,
181 |   DropdownMenuShortcut,
182 |   DropdownMenuGroup,
183 |   DropdownMenuPortal,
184 |   DropdownMenuSub,
185 |   DropdownMenuSubContent,
186 |   DropdownMenuSubTrigger,
187 |   DropdownMenuRadioGroup,
188 | };
```

src/components/ui/form.tsx

```
1 | import * as React from "react";
2 | import * as LabelPrimitive from "@radix-ui/react-label";
3 | import { Slot } from "@radix-ui/react-slot";
4 | import {
5 |   Controller,
6 |   FormProvider,
7 |   useFormContext,
8 |   type ControllerProps,
9 |   type FieldPath,
10 | type FieldValues,
11 | } from "react-hook-form";
12 |
13 | import { cn } from "@lib/utils";
14 | import { Label } from "@components/ui/label";
15 |
16 | const Form = FormProvider;
17 |
18 | type FormFieldContextValue<
19 |   TFieldValues extends FieldValues = FieldValues,
20 |   TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>,
21 | > = {
22 |   name: TName;
23 | };
24 |
25 | const FormFieldContext = React.createContext<FormFieldContextValue | null>(null);
26 |
27 | const FormField = <
28 |   TFieldValues extends FieldValues = FieldValues,
29 |   TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>,
30 | >({
31 |   ...props
32 | }): ControllerProps<TFieldValues, TName> => {
33 |   return (
34 |     <FormFieldContext.Provider value={{ name: props.name }}>
35 |       <Controller {...props} />
36 |     </FormFieldContext.Provider>
37 |   );
38 | };
39 |
40 | const useFormField = () => {
41 |   const fieldContext = React.useContext(FormFieldContext);
42 |   const itemContext = React.useContext(FormItemContext);
43 |   const { getFieldState, formState } = useFormContext();
44 |
45 |   if (!fieldContext) {
46 |     throw new Error("useFormField should be used within <FormField>");
47 |   }
48 |
49 |   if (!itemContext) {
50 |     throw new Error("useFormField should be used within <FormItem>");
51 |   }
52 |
53 |   const fieldState = getFieldState(fieldContext.name, formState);
54 |
55 |   const { id } = itemContext;
56 |
57 |   return {
58 |     id,
59 |     name: fieldContext.name,
60 |     formItemId: `${id}-form-item`,
61 |     formDescriptionId: `${id}-form-item-description`,
62 |     formMessageId: `${id}-form-item-message`,
63 |     ...fieldState,
64 |   };
65 | };
66 |
67 | type FormItemContextValue = {
68 |   id: string;
69 | };
70 |
71 | const FormItemContext = React.createContext<FormItemContextValue | null>(null);
72 |
73 | const FormItem = React.forwardRef<HTMLDivElement, React.HTMLAttributes<HTMLDivElement>>(<
74 |   ({ className, ...props }, ref) => {
75 |     const id = React.useId();
76 |
77 |     return (
78 |       <FormItemContext.Provider value={{ id }}>
79 |         <div ref={ref} className={cn("space-y-2", className)} {...props} />
80 |       </FormItemContext.Provider>
81 |     );
82 |   },
83 | );
84 | FormItem.displayName = "FormItem";
85 |
86 | const FormLabel = React.forwardRef<
87 |   React.ElementRef<typeof LabelPrimitive.Root>,
88 |   React.ComponentPropsWithoutRef<typeof LabelPrimitive.Root>
89 | >(({ className, ...props }, ref) => {
90 |   const { error, formItemId } = useFormField();
```

```

91 |
92 |     return (
93 |       <Label
94 |         ref={ref}
95 |         className={cn(error && "text-destructive", className)}
96 |         htmlFor={formItemId}
97 |         {...props}
98 |       />
99 |     );
100 |   });
101 |   FormLabel.displayName = "FormLabel";
102 |
103 |   const FormControl = React.forwardRef<
104 |     React.ElementRef<typeof Slot>,
105 |     React.ComponentPropsWithoutRef<typeof Slot>
106 |   >(({ ...props }, ref) => {
107 |     const { error, formItemId, formDescriptionId, formMessageId } = useFormField();
108 |
109 |     return (
110 |       <Slot
111 |         ref={ref}
112 |         id={formItemId}
113 |         aria-describedby={!error ? `${formDescriptionId}` : `${formDescriptionId} ${formMessageId}`}
114 |         aria-invalid={!error}
115 |         {...props}
116 |       />
117 |     );
118 |   });
119 |   FormControl.displayName = "FormControl";
120 |
121 |   const FormDescription = React.forwardRef<
122 |     HTMLParagraphElement,
123 |     React.HTMLAttributes<HTMLParagraphElement>
124 |   >(({ className, ...props }, ref) => {
125 |     const { formDescriptionId } = useFormField();
126 |
127 |     return (
128 |       <p
129 |         ref={ref}
130 |         id={formDescriptionId}
131 |         className={cn("text-[0.8rem] text-muted-foreground", className)}
132 |         {...props}
133 |       />
134 |     );
135 |   });
136 |   FormDescription.displayName = "FormDescription";
137 |
138 |   const FormMessage = React.forwardRef<
139 |     HTMLParagraphElement,
140 |     React.HTMLAttributes<HTMLParagraphElement>
141 |   >(({ className, children, ...props }, ref) => {
142 |     const { error, formMessageId } = useFormField();
143 |     const body = error ? String(error?.message ?? "") : children;
144 |
145 |     if (!body) {
146 |       return null;
147 |     }
148 |
149 |     return (
150 |       <p
151 |         ref={ref}
152 |         id={formMessageId}
153 |         className={cn("text-[0.8rem] font-medium text-destructive", className)}
154 |         {...props}
155 |       >
156 |         {body}
157 |       </p>
158 |     );
159 |   });
160 |   FormMessage.displayName = "FormMessage";
161 |
162 |   export {
163 |     useFormField,
164 |     Form,
165 |     FormItem,
166 |     FormLabel,
167 |     FormControl,
168 |     FormDescription,
169 |     FormMessage,
170 |     FormField,
171 |   };

```

src/components/ui/hover-card.tsx

```
1 | import * as React from "react";
2 | import * as HoverCardPrimitive from "@radix-ui/react-hover-card";
3 |
4 | import { cn } from "@lib/utls";
5 |
6 | const HoverCard = HoverCardPrimitive.Root;
7 |
8 | const HoverCardTrigger = HoverCardPrimitive.Trigger;
9 |
10 | const HoverCardContent = React.forwardRef<
11 |   React.ElementRef<typeof HoverCardPrimitive.Content>,
12 |   React.ComponentPropsWithoutRef<typeof HoverCardPrimitive.Content>
13 | >(({ className, align = "center", sideOffset = 4, ...props }, ref) => (
14 |   <HoverCardPrimitive.Content
15 |     ref={ref}
16 |     align={align}
17 |     sideOffset={sideOffset}
18 |     className={cn(
19 |       "z-50 w-64 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-hover-card-content-transform-origin]",
20 |       className,
21 |     )}
22 |     {...props}
23 |   />
24 | ));
25 | HoverCardContent.displayName = HoverCardPrimitive.Content.displayName;
26 |
27 | export { HoverCard, HoverCardTrigger, HoverCardContent };
```

src/components/ui/input-otp.tsx

```
1 | import * as React from "react";
2 | import { OTPInput, OTPInputContext } from "input-otp";
3 | import { Minus } from "lucide-react";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const InputOTP = React.forwardRef<
8 |   React.ElementRef<typeof OTPInput>,
9 |   React.ComponentPropsWithoutRef<typeof OTPInput>
10 | >(({ className, containerClassName, ...props }, ref) => (
11 |   <OTPInput
12 |     ref={ref}
13 |     containerClassName={cn(
14 |       "flex items-center gap-2 has-[:disabled]:opacity-50",
15 |       containerClassName,
16 |     )}
17 |     className={cn("disabled:cursor-not-allowed", className)}
18 |     {...props}
19 |   />
20 | ));
21 | InputOTP.displayName = "InputOTP";
22 |
23 | const InputOTPGroup = React.forwardRef<
24 |   React.ElementRef<"div">,
25 |   React.ComponentPropsWithoutRef<"div">
26 | >(({ className, ...props }, ref) => (
27 |   <div ref={ref} className={cn("flex items-center", className)} {...props} />
28 | ));
29 | InputOTPGroup.displayName = "InputOTPGroup";
30 |
31 | const InputOTPSlot = React.forwardRef<
32 |   React.ElementRef<"div">,
33 |   React.ComponentPropsWithoutRef<"div"> & { index: number }
34 | >(({ index, className, ...props }, ref) => {
35 |   const inputOTPContext = React.useContext(OTPInputContext);
36 |   const { char, hasFakeCaret, isActive } = inputOTPContext.slots[index];
37 |
38 |   return (
39 |     <div
40 |       ref={ref}
41 |       className={cn(
42 |         "relative flex h-9 w-9 items-center justify-center border-y border-r border-input text-sm shadow-sm transition
43 |         -all first:rounded-l-md first:border-l last:rounded-r-md",
44 |         isActive && "z-10 ring-1 ring-ring",
45 |         className,
46 |       )}
47 |       >
48 |         {char}
49 |         {hasFakeCaret && (
50 |           <div className="pointer-events-none absolute inset-0 flex items-center justify-center">
51 |             <div className="h-4 w-px animate-caret-blink bg-foreground duration-1000" />
52 |           </div>
53 |         )}
54 |       </div>
55 |     );
56 |   });
57 | InputOTPSlot.displayName = "InputOTPSlot";
58 |
59 | const InputOTPSeparator = React.forwardRef<
60 |   React.ElementRef<"div">,
61 |   React.ComponentPropsWithoutRef<"div">
62 | >(({ ...props }, ref) => (
63 |   <div ref={ref} role="separator" {...props}>
64 |     <Minus />
65 |   </div>
66 | ));
67 | InputOTPSeparator.displayName = "InputOTPSeparator";
68 |
69 | export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator };
```

src/components/ui/input.tsx

```
1 | import * as React from "react";
2 |
3 | import { cn } from "@lib/utils";
4 |
5 | const Input = React.forwardRef<HTMLInputElement, React.ComponentProps<"input">>(
6 |   ({ className, type, ...props }, ref) => {
7 |     return (
8 |       <input
9 |         type={type}
10 |        className={cn(
11 |          "flex h-9 w-full rounded-md border border-input bg-transparent px-3 py-1 text-base shadow-sm transition-colors file:border-0 file:bg-transparent file:text-sm file:font-medium file:text-foreground placeholder:text-muted-foreground focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
12 |          className,
13 |        )}
14 |        ref={ref}
15 |        {...props}
16 |      />
17 |    );
18 |  },
19 | );
20 | Input.displayName = "Input";
21 |
22 | export { Input };
```

src/components/ui/label.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as LabelPrimitive from "@radix-ui/react-label";
5 | import { cva, type VariantProps } from "class-variance-authority";
6 |
7 | import { cn } from "@lib/utls";
8 |
9 | const labelVariants = cva(
10 |   "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70",
11 | );
12 |
13 | const Label = React.forwardRef<
14 |   React.ElementRef<typeof LabelPrimitive.Root>,
15 |   React.ComponentPropsWithoutRef<typeof LabelPrimitive.Root> & VariantProps<typeof labelVariants>
16 | >(({ className, ...props }, ref) => (
17 |   <LabelPrimitive.Root ref={ref} className={cn(labelVariants(), className)} {...props} />
18 | ));
19 | Label.displayName = LabelPrimitive.Root.displayName;
20 |
21 | export { Label };
```

src/components/ui/menubar.tsx

```
1 | import * as React from "react";
2 | import * as MenubarPrimitive from "@radix-ui/react-menubar";
3 | import { Check, ChevronRight, Circle } from "lucide-react";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | function MenubarMenu({ ...props }: React.ComponentProps<typeof MenubarPrimitive.Menu>) {
8 |   return <MenubarPrimitive.Menu {...props} />;
9 | }
10 |
11 | function MenubarGroup({ ...props }: React.ComponentProps<typeof MenubarPrimitive.Group>) {
12 |   return <MenubarPrimitive.Group {...props} />;
13 | }
14 |
15 | function MenubarPortal({ ...props }: React.ComponentProps<typeof MenubarPrimitive.Portal>) {
16 |   return <MenubarPrimitive.Portal {...props} />;
17 | }
18 |
19 | function MenubarRadioGroup({ ...props }: React.ComponentProps<typeof MenubarPrimitive.RadioGroup>) {
20 |   return <MenubarPrimitive.RadioGroup {...props} />;
21 | }
22 |
23 | function MenubarSub({ ...props }: React.ComponentProps<typeof MenubarPrimitive.Sub>) {
24 |   return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />;
25 | }
26 |
27 | const Menubar = React.forwardRef<
28 |   React.ElementRef<typeof MenubarPrimitive.Root>,
29 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Root>
30 | >(({ className, ...props }, ref) => {
31 |   <MenubarPrimitive.Root
32 |     ref={ref}
33 |     className={cn(
34 |       "flex h-9 items-center space-x-1 rounded-md border bg-background p-1 shadow-sm",
35 |       className,
36 |     )}
37 |     {...props}
38 |   />
39 | ));
40 | Menubar.displayName = MenubarPrimitive.Root.displayName;
41 |
42 | const MenubarTrigger = React.forwardRef<
43 |   React.ElementRef<typeof MenubarPrimitive.Trigger>,
44 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Trigger>
45 | >(({ className, ...props }, ref) => {
46 |   <MenubarPrimitive.Trigger
47 |     ref={ref}
48 |     className={cn(
49 |       "flex cursor-default select-none items-center rounded-sm px-3 py-1 text-sm font-medium outline-none focus:bg-accent
50 |       ent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
51 |       className,
52 |     )}
53 |     {...props}
54 |   />
55 | ));
56 | MenubarTrigger.displayName = MenubarPrimitive.Trigger.displayName;
57 |
58 | const MenubarSubTrigger = React.forwardRef<
59 |   React.ElementRef<typeof MenubarPrimitive.SubTrigger>,
60 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.SubTrigger> & {
61 |     inset?: boolean;
62 |   }
63 | >(({ className, inset, children, ...props }, ref) => {
64 |   <MenubarPrimitive.SubTrigger
65 |     ref={ref}
66 |     className={cn(
67 |       "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent focus:
68 |       text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
69 |       inset && "pl-8",
70 |       className,
71 |     )}
72 |     {...props}
73 |   >
74 |     {children}
75 |     <ChevronRight className="ml-auto h-4 w-4" />
76 |   </MenubarPrimitive.SubTrigger>
77 | ));
78 | MenubarSubTrigger.displayName = MenubarPrimitive.SubTrigger.displayName;
79 |
80 | const MenubarSubContent = React.forwardRef<
81 |   React.ElementRef<typeof MenubarPrimitive.SubContent>,
82 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.SubContent>
83 | >(({ className, ...props }, ref) => {
84 |   <MenubarPrimitive.SubContent
85 |     ref={ref}
86 |     className={cn(
87 |       "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg data-[stat
88 |       e=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data
89 |       -[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:s
90 |       lide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-(--radix
```

```

    -menubar-content-transform-origin)",
86 |     className,
87 |   })
88 |   {...props}
89 | />
90 | ));
91 | MenubarSubContent.displayName = MenubarPrimitive.SubContent.displayName;
92 |
93 | const MenubarContent = React.forwardRef<
94 |   React.ElementRef<typeof MenubarPrimitive.Content>,
95 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Content>
96 | >(({ className, align = "start", alignOffset = -4, sideOffset = 8, ...props }, ref) => (
97 |   <MenubarPrimitive.Portal>
98 |     <MenubarPrimitive.Content
99 |       ref={ref}
100 |       align={align}
101 |       alignOffset={alignOffset}
102 |       sideOffset={sideOffset}
103 |       className={cn(
104 |         "z-50 min-w-[12rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md data-[state=open]:animate-in data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-menubar-content-transform-origin]",
105 |         className,
106 |       )}
107 |       {...props}
108 |     />
109 |   </MenubarPrimitive.Portal>
110 | ));
111 | MenubarContent.displayName = MenubarPrimitive.Content.displayName;
112 |
113 | const MenuItem = React.forwardRef<
114 |   React.ElementRef<typeof MenubarPrimitive.Item>,
115 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Item> & {
116 |     inset?: boolean;
117 |   }
118 | >(({ className, inset, ...props }, ref) => (
119 |   <MenubarPrimitive.Item
120 |     ref={ref}
121 |     className={cn(
122 |       "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
123 |       inset && "pl-8",
124 |       className,
125 |     )}
126 |     {...props}
127 |   />
128 | ));
129 | MenuItem.displayName = MenubarPrimitive.Item.displayName;
130 |
131 | const MenubarCheckboxItem = React.forwardRef<
132 |   React.ElementRef<typeof MenubarPrimitive.CheckboxItem>,
133 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.CheckboxItem>
134 | >(({ className, children, checked, ...props }, ref) => (
135 |   <MenubarPrimitive.CheckboxItem
136 |     ref={ref}
137 |     className={cn(
138 |       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
139 |       className,
140 |     )}
141 |     checked={checked}
142 |     {...props}
143 |   >
144 |     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
145 |       <MenubarPrimitive.ItemIndicator>
146 |         <Check className="h-4 w-4" />
147 |       </MenubarPrimitive.ItemIndicator>
148 |     </span>
149 |     {children}
150 |   </MenubarPrimitive.CheckboxItem>
151 | ));
152 | MenubarCheckboxItem.displayName = MenubarPrimitive.CheckboxItem.displayName;
153 |
154 | const MenubarRadioItem = React.forwardRef<
155 |   React.ElementRef<typeof MenubarPrimitive.RadioItem>,
156 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.RadioItem>
157 | >(({ className, children, ...props }, ref) => (
158 |   <MenubarPrimitive.RadioItem
159 |     ref={ref}
160 |     className={cn(
161 |       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
162 |       className,
163 |     )}
164 |     {...props}
165 |   >
166 |     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
167 |       <MenubarPrimitive.ItemIndicator>
168 |         <Circle className="h-4 w-4 fill-current" />
169 |       </MenubarPrimitive.ItemIndicator>
170 |     </span>

```

```

171 |     {children}
172 |   </MenubarPrimitive.RadioItem>
173 | ));
174 | MenubarRadioItem.displayName = MenubarPrimitive.RadioItem.displayName;
175 |
176 | const MenubarLabel = React.forwardRef<
177 |   React.ElementRef<typeof MenubarPrimitive.Label>,
178 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Label> & {
179 |     inset?: boolean;
180 |   }
181 | >(({ className, inset, ...props }, ref) => (
182 |   <MenubarPrimitive.Label
183 |     ref={ref}
184 |     className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8", className)}
185 |     {...props}
186 |   />
187 | ));
188 | MenubarLabel.displayName = MenubarPrimitive.Label.displayName;
189 |
190 | const MenubarSeparator = React.forwardRef<
191 |   React.ElementRef<typeof MenubarPrimitive.Separator>,
192 |   React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Separator>
193 | >(({ className, ...props }, ref) => (
194 |   <MenubarPrimitive.Separator
195 |     ref={ref}
196 |     className={cn("-mx-1 my-1 h-px bg-muted", className)}
197 |     {...props}
198 |   />
199 | ));
200 | MenubarSeparator.displayName = MenubarPrimitive.Separator.displayName;
201 |
202 | const MenubarShortcut = ({ className, ...props }: React.HTMLAttributes<HTMLSpanElement>) => {
203 |   return (
204 |     <span
205 |       className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
206 |       {...props}
207 |     />
208 |   );
209 | };
210 | MenubarShortcut.displayName = "MenubarShortcut";
211 |
212 | export {
213 |   Menubar,
214 |   MenubarMenu,
215 |   MenubarTrigger,
216 |   MenubarContent,
217 |   MenubarItem,
218 |   MenubarSeparator,
219 |   MenubarLabel,
220 |   MenubarCheckboxItem,
221 |   MenubarRadioGroup,
222 |   MenubarRadioItem,
223 |   MenubarPortal,
224 |   MenubarSubContent,
225 |   MenubarSubTrigger,
226 |   MenubarGroup,
227 |   MenubarSub,
228 |   MenubarShortcut,
229 | };

```

src/components/ui/navigation-menu.tsx

```
1 | import * as React from "react";
2 | import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu";
3 | import { cva } from "class-variance-authority";
4 | import { ChevronDown } from "lucide-react";
5 |
6 | import { cn } from "@lib/utils";
7 |
8 | const NavigationMenu = React.forwardRef<
9 |   React.ElementRef<typeof NavigationMenuPrimitive.Root>,
10 |   React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Root>
11 | >(({ className, children, ...props }, ref) => (
12 |   <NavigationMenuPrimitive.Root
13 |     ref={ref}
14 |     className={cn("relative z-10 flex max-w-max flex-1 items-center justify-center", className)}
15 |     {...props}
16 |   >
17 |     {children}
18 |     <NavigationMenuViewport />
19 |   </NavigationMenuPrimitive.Root>
20 | ));
21 | NavigationMenu.displayName = NavigationMenuPrimitive.Root.displayName;
22 |
23 | const NavigationMenuList = React.forwardRef<
24 |   React.ElementRef<typeof NavigationMenuPrimitive.List>,
25 |   React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.List>
26 | >(({ className, ...props }, ref) => (
27 |   <NavigationMenuPrimitive.List
28 |     ref={ref}
29 |     className={cn("group flex flex-1 list-none items-center justify-center space-x-1", className)}
30 |     {...props}
31 |   />
32 | ));
33 | NavigationMenuList.displayName = NavigationMenuPrimitive.List.displayName;
34 |
35 | const NavigationMenuItem = NavigationMenuPrimitive.Item;
36 |
37 | const navigationMenuTriggerStyle = cva(
38 |   "group inline-flex h-9 w-max items-center justify-center rounded-md bg-background px-4 py-2 text-sm font-medium curs
39 |   or-pointer transition-colors hover:bg-accent hover:text-accent-foreground focus:bg-accent focus:text-accent-foregr
40 |   ound focus:outline-none disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed data-[state=open]:text-accent-foreground data-[state=open]:bg-accent/50 data-[state=open]:hover:bg-accent data-[state=open]:foc
41 |   us:bg-accent",
42 | );
43 |
44 | const NavigationMenuTrigger = React.forwardRef<
45 |   React.ElementRef<typeof NavigationMenuPrimitive.Trigger>,
46 |   React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Trigger>
47 | >(({ className, children, ...props }, ref) => (
48 |   <NavigationMenuPrimitive.Trigger
49 |     ref={ref}
50 |     className={cn(navigationMenuTriggerStyle(), "group", className)}
51 |     {...props}
52 |   >
53 |     {children}{ " "}
54 |     <ChevronDown
55 |       className={cn("relative top-[1px] ml-1 h-3 w-3 transition duration-300 group-data-[state=open]:rotate-180"
56 |         aria-hidden="true"
57 |       />
58 |   </NavigationMenuPrimitive.Trigger>
59 | ));
60 | NavigationMenuTrigger.displayName = NavigationMenuPrimitive.Trigger.displayName;
61 |
62 | const NavigationMenuContent = React.forwardRef<
63 |   React.ElementRef<typeof NavigationMenuPrimitive.Content>,
64 |   React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Content>
65 | >(({ className, ...props }, ref) => (
66 |   <NavigationMenuPrimitive.Content
67 |     ref={ref}
68 |     className={cn(
69 |       "left-0 top-0 w-full data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out data-[motion^=from-]:fade-in
70 |       data-[motion^=to-]:fade-out data-[motion=from-end]:slide-in-from-right-52 data-[motion=from-start]:slide-in-from-
71 |       left-52 data-[motion=to-end]:slide-out-to-right-52 data-[motion=to-start]:slide-out-to-left-52 md:absolute md:w-au
72 |       to ",
73 |       className,
74 |     )}
75 |     {...props}
76 |   />
77 | ));
78 | NavigationMenuContent.displayName = NavigationMenuPrimitive.Content.displayName;
79 |
80 | const NavigationMenuLink = NavigationMenuPrimitive.Link;
81 |
82 | const NavigationMenuViewport = React.forwardRef<
83 |   React.ElementRef<typeof NavigationMenuPrimitive.Viewport>,
84 |   React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Viewport>
85 | >(({ className, ...props }, ref) => (
86 |   <div className={cn("absolute left-0 top-full flex justify-center")}>
87 |     <NavigationMenuPrimitive.Viewport
88 |       className={cn(
89 |         "origin-top-center relative mt-1.5 h-[var(--radix-navigation-menu-viewport-height)] w-full overflow-hidden rou
```

```

      nded-md border bg-popover text-popover-foreground shadow data-[state=open]:animate-in data-[state=closed]:animate-
      out data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-90 md:w-[var(--radix-navigation-menu-viewport-width
    ]",
84 |     className,
85 |   )}
86 |   ref={ref}
87 |   {...props}
88 | />
89 | </div>
90 | ));
91 | NavigationMenuViewport.displayName = NavigationMenuPrimitive.Viewport.displayName;
92 |
93 | const NavigationMenuIndicator = React.forwardRef<
94 |   React.ElementRef<typeof NavigationMenuPrimitive.Indicator>,
95 |   React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Indicator>
96 | >(({ className, ...props }, ref) => (
97 |   <NavigationMenuPrimitive.Indicator
98 |     ref={ref}
99 |     className={cn(
100 |       "top-full z-[1] flex h-1.5 items-end justify-center overflow-hidden data-[state=visible]:animate-in data-[state=
      hidden]:animate-out data-[state=hidden]:fade-out data-[state=visible]:fade-in",
101 |       className,
102 |     )}
103 |     {...props}
104 |   >
105 |     <div className="relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm bg-border shadow-md" />
106 |   </NavigationMenuPrimitive.Indicator>
107 | ));
108 | NavigationMenuIndicator.displayName = NavigationMenuPrimitive.Indicator.displayName;
109 |
110 | export {
111 |   navigationMenuTriggerStyle,
112 |   NavigationMenu,
113 |   NavigationMenuList,
114 |   NavigationMenuItem,
115 |   NavigationMenuContent,
116 |   NavigationMenuTrigger,
117 |   NavigationMenuLink,
118 |   NavigationMenuIndicator,
119 |   NavigationMenuViewport,
120 | };

```

src/components/ui/pagination.tsx

```
1 | import * as React from "react";
2 | import { ChevronLeft, ChevronRight, MoreHorizontal } from "lucide-react";
3 |
4 | import { cn } from "@lib/utils";
5 | import { ButtonProps, buttonVariants } from "@components/ui/button";
6 |
7 | const Pagination = ({ className, ...props }: React.ComponentProps<"nav">) => (
8 |   <nav
9 |     role="navigation"
10 |    aria-label="pagination"
11 |    className={cn("mx-auto flex w-full justify-center", className)}
12 |    {...props}
13 |  />
14 | );
15 | Pagination.displayName = "Pagination";
16 |
17 | const PaginationContent = React.forwardRef<HTMLUListElement, React.ComponentProps<"ul">>(<
18 |   ({ className, ...props }, ref) => (
19 |     <ul ref={ref} className={cn("flex flex-row items-center gap-1", className)} {...props} />
20 |   ),
21 | );
22 | PaginationContent.displayName = "PaginationContent";
23 |
24 | const PaginationItem = React.forwardRef<HTMLLIElement, React.ComponentProps<"li">>(<
25 |   ({ className, ...props }, ref) => <li ref={ref} className={cn("", className)} {...props} />,
26 | );
27 | PaginationItem.displayName = "PaginationItem";
28 |
29 | type PaginationLinkProps = {
30 |   isActive?: boolean;
31 | } & Pick<ButtonProps, "size"> &
32 |   React.ComponentProps<"a">;
33 |
34 | const PaginationLink = ({ className, isActive, size = "icon", ...props }: PaginationLinkProps) => (
35 |   <a
36 |     aria-current={isActive ? "page" : undefined}
37 |     className={cn(
38 |       buttonVariants({
39 |         variant: isActive ? "outline" : "ghost",
40 |         size,
41 |       }),
42 |       className,
43 |     )}
44 |     {...props}
45 |   />
46 | );
47 | PaginationLink.displayName = "PaginationLink";
48 |
49 | const PaginationPrevious = ({
50 |   className,
51 |   ...props
52 | }: React.ComponentProps<typeof PaginationLink>) => (
53 |   <PaginationLink
54 |     aria-label="Go to previous page"
55 |     size="default"
56 |     className={cn("gap-1 pl-2.5", className)}
57 |     {...props}
58 |   >
59 |     <ChevronLeft className="h-4 w-4" />
60 |     <span>Previous</span>
61 |   </PaginationLink>
62 | );
63 | PaginationPrevious.displayName = "PaginationPrevious";
64 |
65 | const PaginationNext = ({ className, ...props }: React.ComponentProps<typeof PaginationLink>) => (
66 |   <PaginationLink
67 |     aria-label="Go to next page"
68 |     size="default"
69 |     className={cn("gap-1 pr-2.5", className)}
70 |     {...props}
71 |   >
72 |     <span>Next</span>
73 |     <ChevronRight className="h-4 w-4" />
74 |   </PaginationLink>
75 | );
76 | PaginationNext.displayName = "PaginationNext";
77 |
78 | const PaginationEllipsis = ({ className, ...props }: React.ComponentProps<"span">) => (
79 |   <span
80 |     aria-hidden
81 |     className={cn("flex h-9 w-9 items-center justify-center", className)}
82 |     {...props}
83 |   >
84 |     <MoreHorizontal className="h-4 w-4" />
85 |     <span className="sr-only">More pages</span>
86 |   </span>
87 | );
88 | PaginationEllipsis.displayName = "PaginationEllipsis";
89 |
90 | export {
```

```
91 |     Pagination,  
92 |     PaginationContent,  
93 |     PaginationLink,  
94 |     PaginationItem,  
95 |     PaginationPrevious,  
96 |     PaginationNext,  
97 |     PaginationEllipsis,  
98 | };
```

src/components/ui/popover.tsx

```
1 | import * as React from "react";
2 | import * as PopoverPrimitive from "@radix-ui/react-popover";
3 |
4 | import { cn } from "@lib/utils";
5 |
6 | const Popover = PopoverPrimitive.Root;
7 |
8 | const PopoverTrigger = PopoverPrimitive.Trigger;
9 |
10 | const PopoverAnchor = PopoverPrimitive.Anchor;
11 |
12 | const PopoverContent = React.forwardRef<
13 |   React.ElementRef<typeof PopoverPrimitive.Content>,
14 |   React.ComponentPropsWithoutRef<typeof PopoverPrimitive.Content>
15 | >(({ className, align = "center", sideOffset = 4, ...props }, ref) => {
16 |   <PopoverPrimitive.Portal>
17 |     <PopoverPrimitive.Content
18 |       ref={ref}
19 |       align={align}
20 |       sideOffset={sideOffset}
21 |       className={cn(
22 |         "z-50 w-72 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:a
nimate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=c
losed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-f
rom-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-(--radix-popover-
content-transform-origin)",
23 |         className,
24 |       )}
25 |       {...props}
26 |     />
27 |   </PopoverPrimitive.Portal>
28 | ));
29 | PopoverContent.displayName = PopoverPrimitive.Content.displayName;
30 |
31 | export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor };
```

src/components/ui/progress.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as ProgressPrimitive from "@radix-ui/react-progress";
5 |
6 | import { cn } from "@lib/utils";
7 |
8 | const Progress = React.forwardRef<
9 |   React.ElementRef<typeof ProgressPrimitive.Root>,
10 |   React.ComponentPropsWithoutRef<typeof ProgressPrimitive.Root>
11 | >(({ className, value, ...props }, ref) => (
12 |   <ProgressPrimitive.Root
13 |     ref={ref}
14 |     className={cn("relative h-2 w-full overflow-hidden rounded-full bg-primary/20", className)}
15 |     {...props}
16 |   >
17 |     <ProgressPrimitive.Indicator
18 |       className="h-full w-full flex-1 bg-primary transition-all"
19 |       style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
20 |     />
21 |   </ProgressPrimitive.Root>
22 | );
23 | Progress.displayName = ProgressPrimitive.Root.displayName;
24 |
25 | export { Progress };
```

src/components/ui/radio-group.tsx

```
1 | import * as React from "react";
2 | import * as RadioGroupPrimitive from "@radix-ui/react-radio-group";
3 | import { Circle } from "lucide-react";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const RadioGroup = React.forwardRef<
8 |   React.ElementRef<typeof RadioGroupPrimitive.Root>,
9 |   React.ComponentPropsWithoutRef<typeof RadioGroupPrimitive.Root>
10 | >(({ className, ...props }, ref) => {
11 |   return <RadioGroupPrimitive.Root className={cn("grid gap-2", className)} {...props} ref={ref} />;
12 | });
13 | RadioGroup.displayName = RadioGroupPrimitive.Root.displayName;
14 |
15 | const RadioGroupItem = React.forwardRef<
16 |   React.ElementRef<typeof RadioGroupPrimitive.Item>,
17 |   React.ComponentPropsWithoutRef<typeof RadioGroupPrimitive.Item>
18 | >(({ className, ...props }, ref) => {
19 |   return (
20 |     <RadioGroupPrimitive.Item
21 |       ref={ref}
22 |       className={cn(
23 |         "aspect-square h-4 w-4 rounded-full border border-primary text-primary shadow cursor-pointer focus:outline-non
24 | e focus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50",
25 |         className,
26 |       )}
27 |       {...props}
28 |     >
29 |       <RadioGroupPrimitive.Indicator className="flex items-center justify-center">
30 |         <Circle className="h-3.5 w-3.5 fill-primary" />
31 |       </RadioGroupPrimitive.Indicator>
32 |     </RadioGroupPrimitive.Item>
33 |   );
34 | RadioGroupItem.displayName = RadioGroupPrimitive.Item.displayName;
35 |
36 | export { RadioGroup, RadioGroupItem };
```

src/components/ui/resizable.tsx

```
1 | import { GripVertical } from "lucide-react";
2 | import { Group, Panel, Separator } from "react-resizable-panels";
3 |
4 | import { cn } from "@lib/utills";
5 |
6 | const ResizablePanelGroup = ({ className, ...props }: React.ComponentProps<typeof Group>) => (
7 |   <Group
8 |     className={cn("flex h-full w-full data-[panel-group-direction=vertical]:flex-col", className)}
9 |     {...props}
10 |   />
11 | );
12 |
13 | const ResizablePanel = Panel;
14 |
15 | const ResizableHandle = ({
16 |   withHandle,
17 |   className,
18 |   ...props
19 | }: React.ComponentProps<typeof Separator> & {
20 |   withHandle?: boolean;
21 | }) => (
22 |   <Separator
23 |     className={cn(
24 |       "relative flex w-px items-center justify-center bg-border after:absolute after:inset-y-0 after:left-1/2 after:w-1 after:-translate-x-1/2 focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring focus-visible:ring-offset-1 data-[panel-group-direction=vertical]:h-px data-[panel-group-direction=vertical]:after:left-0 data-[panel-group-direction=vertical]:after:h-1 data-[panel-group-direction=vertical]:after:w-full data-[panel-group-direction=vertical]:after:-translate-y-1/2 data-[panel-group-direction=vertical]:after:translate-x-0 [&[data-panel-group-direction=vertical]>div]:rotate-90",
25 |       className,
26 |     )}
27 |     {...props}
28 |   >
29 |     {withHandle && (
30 |       <div className="z-10 flex h-4 w-3 items-center justify-center rounded-sm border bg-border">
31 |         <GripVertical className="h-2.5 w-2.5" />
32 |       </div>
33 |     )}
34 |   </Separator>
35 | );
36 |
37 | export { ResizablePanelGroup, ResizablePanel, ResizableHandle };
```

src/components/ui/scroll-area.tsx

```
1 | import * as React from "react";
2 | import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area";
3 |
4 | import { cn } from "@lib/utlis";
5 |
6 | const ScrollArea = React.forwardRef<
7 |   React.ElementRef<typeof ScrollAreaPrimitive.Root>,
8 |   React.ComponentPropsWithoutRef<typeof ScrollAreaPrimitive.Root>
9 | >(({ className, children, ...props }, ref) => (
10 |   <ScrollAreaPrimitive.Root
11 |     ref={ref}
12 |     className={cn("relative overflow-hidden", className)}
13 |     {...props}
14 |   >
15 |     <ScrollAreaPrimitive.Viewport className="h-full w-full rounded-[inherit]">
16 |       {children}
17 |     </ScrollAreaPrimitive.Viewport>
18 |     <ScrollBar />
19 |     <ScrollAreaPrimitive.Corner />
20 |   </ScrollAreaPrimitive.Root>
21 | ));
22 | ScrollArea.displayName = ScrollAreaPrimitive.Root.displayName;
23 |
24 | const ScrollBar = React.forwardRef<
25 |   React.ElementRef<typeof ScrollAreaPrimitive.ScrollAreaScrollbar>,
26 |   React.ComponentPropsWithoutRef<typeof ScrollAreaPrimitive.ScrollAreaScrollbar>
27 | >(({ className, orientation = "vertical", ...props }, ref) => (
28 |   <ScrollAreaPrimitive.ScrollAreaScrollbar
29 |     ref={ref}
30 |     orientation={orientation}
31 |     className={cn(
32 |       "flex touch-none select-none transition-colors",
33 |       orientation === "vertical" && "h-full w-2.5 border-l border-l-transparent p-[1px]",
34 |       orientation === "horizontal" && "h-2.5 flex-col border-t border-t-transparent p-[1px]",
35 |       className,
36 |     )}
37 |     {...props}
38 |   >
39 |     <ScrollAreaPrimitive.ScrollAreaThumb className="relative flex-1 rounded-full bg-border" />
40 |   </ScrollAreaPrimitive.ScrollAreaScrollbar>
41 | ));
42 | ScrollBar.displayName = ScrollAreaPrimitive.ScrollAreaScrollbar.displayName;
43 |
44 | export { ScrollArea, ScrollBar };
```

src/components/ui/select.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as SelectPrimitive from "@radix-ui/react-select";
5 | import { Check, ChevronDown, ChevronUp } from "lucide-react";
6 |
7 | import { cn } from "@lib/Utils";
8 |
9 | const Select = SelectPrimitive.Root;
10 |
11 | const SelectGroup = SelectPrimitive.Group;
12 |
13 | const SelectValue = SelectPrimitive.Value;
14 |
15 | const SelectTrigger = React.forwardRef<
16 |   React.ElementRef<typeof SelectPrimitive.Trigger>,
17 |   React.ComponentPropsWithoutRef<typeof SelectPrimitive.Trigger>
18 | >(({ className, children, ...props }, ref) => {
19 |   <SelectPrimitive.Trigger
20 |     ref={ref}
21 |     className={cn(
22 |       "flex h-9 w-full items-center justify-between whitespace-nowrap rounded-md border border-input bg-transparent px
23 |       -3 py-2 text-sm shadow-sm ring-offset-background cursor-pointer data-[placeholder]:text-muted-foreground focus:out
24 |       line-none focus:ring-1 focus:ring-ring disabled:cursor-not-allowed disabled:opacity-50 [>span]:line-clamp-1",
25 |       className,
26 |     )}
27 |     {...props}
28 |   >
29 |     {children}
30 |     <SelectPrimitive.Icon asChild>
31 |       <ChevronDown className="h-4 w-4 opacity-50" />
32 |     </SelectPrimitive.Icon>
33 |   </SelectPrimitive.Trigger>
34 | );
35 | SelectTrigger.displayName = SelectPrimitive.Trigger.displayName;
36 |
37 | const SelectScrollUpButton = React.forwardRef<
38 |   React.ElementRef<typeof SelectPrimitive.ScrollUpButton>,
39 |   React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollUpButton>
40 | >(({ className, ...props }, ref) => {
41 |   <SelectPrimitive.ScrollUpButton
42 |     ref={ref}
43 |     className={cn("flex cursor-default items-center justify-center py-1", className)}
44 |     {...props}
45 |   >
46 |     <ChevronUp className="h-4 w-4" />
47 |   </SelectPrimitive.ScrollUpButton>
48 | );
49 | SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName;
50 |
51 | const SelectScrollDownButton = React.forwardRef<
52 |   React.ElementRef<typeof SelectPrimitive.ScrollDownButton>,
53 |   React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollDownButton>
54 | >(({ className, ...props }, ref) => {
55 |   <SelectPrimitive.ScrollDownButton
56 |     ref={ref}
57 |     className={cn("flex cursor-default items-center justify-center py-1", className)}
58 |     {...props}
59 |   >
60 |     <ChevronDown className="h-4 w-4" />
61 |   </SelectPrimitive.ScrollDownButton>
62 | );
63 | SelectScrollDownButton.displayName = SelectPrimitive.ScrollDownButton.displayName;
64 |
65 | const SelectContent = React.forwardRef<
66 |   React.ElementRef<typeof SelectPrimitive.Content>,
67 |   React.ComponentPropsWithoutRef<typeof SelectPrimitive.Content>
68 | >(({ className, children, position = "popper", ...props }, ref) => {
69 |   <SelectPrimitive.Portal>
70 |     <SelectPrimitive.Content
71 |       ref={ref}
72 |       className={cn(
73 |         "relative z-50 max-h-(--radix-select-content-available-height) min-w-[8rem] overflow-y-auto overflow-x-hidden
74 |         rounded-md border bg-popover text-popover-foreground shadow-md data-[state=open]:animate-in data-[state=closed]:an
75 |         imate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=o
76 |         pen]:zoom-in-95 data-[state=bottom]:slide-in-from-top-2 data-[state=left]:slide-in-from-right-2 data-[state=right]:sl
77 |         ide-in-from-left-2 data-[state=top]:slide-in-from-bottom-2 origin-(--radix-select-content-transform-origin)",
78 |         position === "popper" &&
79 |         "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1 data-[side
80 |         =top]:-translate-y-1",
81 |         className,
82 |       )}
83 |       position={position}
84 |       {...props}
85 |     >
86 |       <SelectScrollUpButton />
87 |       <SelectPrimitive.Viewport
88 |         className={cn(
89 |           "p-1",
90 |           position === "popper" &&
```

```

84 |         "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-width)]",
85 |     )}
86 |     >
87 |     {children}
88 |     </SelectPrimitive.Viewport>
89 |     <SelectScrollDownButton />
90 |     </SelectPrimitive.Content>
91 | </SelectPrimitive.Portal>
92 | ));
93 | SelectContent.displayName = SelectPrimitive.Content.displayName;
94 |
95 | const SelectLabel = React.forwardRef<
96 |   React.ElementRef<typeof SelectPrimitive.Label>,
97 |   React.ComponentPropsWithoutRef<typeof SelectPrimitive.Label>
98 | >(({ className, ...props }, ref) => (
99 |   <SelectPrimitive.Label
100 |     ref={ref}
101 |     className={cn("px-2 py-1.5 text-sm font-semibold", className)}
102 |     {...props}
103 |   />
104 | ));
105 | SelectLabel.displayName = SelectPrimitive.Label.displayName;
106 |
107 | const SelectItem = React.forwardRef<
108 |   React.ElementRef<typeof SelectPrimitive.Item>,
109 |   React.ComponentPropsWithoutRef<typeof SelectPrimitive.Item>
110 | >(({ className, children, ...props }, ref) => (
111 |   <SelectPrimitive.Item
112 |     ref={ref}
113 |     className={cn(
114 |       "relative flex w-full cursor-default select-none items-center rounded-sm py-1.5 pl-2 pr-8 text-sm outline-none focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
115 |       className,
116 |     )}
117 |     {...props}
118 |   >
119 |     <span className="absolute right-2 flex h-3.5 w-3.5 items-center justify-center">
120 |       <SelectPrimitive.ItemIndicator>
121 |         <Check className="h-4 w-4" />
122 |       </SelectPrimitive.ItemIndicator>
123 |     </span>
124 |     <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
125 |   </SelectPrimitive.Item>
126 | ));
127 | SelectItem.displayName = SelectPrimitive.Item.displayName;
128 |
129 | const SelectSeparator = React.forwardRef<
130 |   React.ElementRef<typeof SelectPrimitive.Separator>,
131 |   React.ComponentPropsWithoutRef<typeof SelectPrimitive.Separator>
132 | >(({ className, ...props }, ref) => (
133 |   <SelectPrimitive.Separator
134 |     ref={ref}
135 |     className={cn("-mx-1 my-1 h-px bg-muted", className)}
136 |     {...props}
137 |   />
138 | ));
139 | SelectSeparator.displayName = SelectPrimitive.Separator.displayName;
140 |
141 | export {
142 |   Select,
143 |   SelectGroup,
144 |   SelectValue,
145 |   SelectTrigger,
146 |   SelectContent,
147 |   SelectLabel,
148 |   SelectItem,
149 |   SelectSeparator,
150 |   SelectScrollUpButton,
151 |   SelectScrollDownButton,
152 | };

```

src/components/ui/separator.tsx

```
1 | import * as React from "react";
2 | import * as SeparatorPrimitive from "@radix-ui/react-separator";
3 |
4 | import { cn } from "@lib/utils";
5 |
6 | const Separator = React.forwardRef<
7 |   React.ElementRef<typeof SeparatorPrimitive.Root>,
8 |   React.ComponentPropsWithoutRef<typeof SeparatorPrimitive.Root>
9 | >(({ className, orientation = "horizontal", decorative = true, ...props }, ref) => (
10 |   <SeparatorPrimitive.Root
11 |     ref={ref}
12 |     decorative={decorative}
13 |     orientation={orientation}
14 |     className={cn(
15 |       "shrink-0 bg-border",
16 |       orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",
17 |       className,
18 |     )}
19 |     {...props}
20 |   />
21 | ));
22 | Separator.displayName = SeparatorPrimitive.Root.displayName;
23 |
24 | export { Separator };
```

src/components/ui/sheet.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as SheetPrimitive from "@radix-ui/react-dialog";
5 | import { cva, type VariantProps } from "class-variance-authority";
6 | import { X } from "lucide-react";
7 |
8 | import { cn } from "@lib/Utils";
9 |
10 | const Sheet = SheetPrimitive.Root;
11 |
12 | const SheetTrigger = SheetPrimitive.Trigger;
13 |
14 | const SheetClose = SheetPrimitive.Close;
15 |
16 | const SheetPortal = SheetPrimitive.Portal;
17 |
18 | const SheetOverlay = React.forwardRef<
19 |   React.ElementRef<typeof SheetPrimitive.Overlay>,
20 |   React.ComponentPropsWithoutRef<typeof SheetPrimitive.Overlay>
21 | >(({ className, ...props }, ref) => {
22 |   <SheetPrimitive.Overlay
23 |     className={cn(
24 |       "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
25 |       className,
26 |     )}
27 |     {...props}
28 |     ref={ref}
29 |   />
30 | ));
31 | SheetOverlay.displayName = SheetPrimitive.Overlay.displayName;
32 |
33 | const sheetVariants = cva(
34 |   "fixed z-50 gap-4 bg-background p-6 shadow-lg transition ease-in-out data-[state=closed]:duration-300 data-[state=open]:duration-500 data-[state=open]:animate-in data-[state=closed]:animate-out",
35 |   {
36 |     variants: {
37 |       side: {
38 |         top: "inset-x-0 top-0 border-b data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from-top",
39 |         bottom: "inset-x-0 bottom-0 border-t data-[state=closed]:slide-out-to-bottom data-[state=open]:slide-in-from-bottom",
40 |         left: "inset-y-0 left-0 h-full w-3/4 border-r data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left sm:max-w-sm",
41 |         right: "inset-y-0 right-0 h-full w-3/4 border-l data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right sm:max-w-sm",
42 |       },
43 |       defaultVariants: {
44 |         side: "right",
45 |       },
46 |     },
47 |   },
48 | );
49 |
50 | interface SheetContentProps
51 |   extends
52 |     React.ComponentPropsWithoutRef<typeof SheetPrimitive.Content>,
53 |     VariantProps<typeof sheetVariants> {}
54 |
55 | const SheetContent = React.forwardRef<
56 |   React.ElementRef<typeof SheetPrimitive.Content>,
57 |   SheetContentProps
58 | >(({ side = "right", className, children, ...props }, ref) => {
59 |   <SheetPortal>
60 |     <SheetOverlay>
61 |       <SheetPrimitive.Content ref={ref} className={cn(sheetVariants({ side }), className)} {...props}>
62 |         <SheetPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background cursor-pointer transition-opacity hover:opacity-100 focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:pointer-events-none data-[state=open]:bg-secondary">
63 |           <X className="h-4 w-4" />
64 |           <span className="sr-only">Close</span>
65 |         </SheetPrimitive.Close>
66 |         {children}
67 |       </SheetPrimitive.Content>
68 |     </SheetPortal>
69 |   </SheetOverlay>
70 | );
71 | SheetContent.displayName = SheetPrimitive.Content.displayName;
72 |
73 | const SheetHeader = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => {
74 |   <div className={cn("flex flex-col space-y-2 text-center sm:text-left", className)} {...props} />
75 | };
76 | SheetHeader.displayName = "SheetHeader";
77 |
78 | const SheetFooter = ({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) => {
79 |   <div
80 |     className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
81 |     {...props}
82 |   />
83 | }
```

```

84 | });
85 | SheetFooter.displayName = "SheetFooter";
86 |
87 | const SheetTitle = React.forwardRef<
88 |   React.ElementRef<typeof SheetPrimitive.Title>,
89 |   React.ComponentPropsWithoutRef<typeof SheetPrimitive.Title>
90 | >(({ className, ...props }, ref) => (
91 |   <SheetPrimitive.Title
92 |     ref={ref}
93 |     className={cn("text-lg font-semibold text-foreground", className)}
94 |     {...props}
95 |   />
96 | ));
97 | SheetTitle.displayName = SheetPrimitive.Title.displayName;
98 |
99 | const SheetDescription = React.forwardRef<
100 |   React.ElementRef<typeof SheetPrimitive.Description>,
101 |   React.ComponentPropsWithoutRef<typeof SheetPrimitive.Description>
102 | >(({ className, ...props }, ref) => (
103 |   <SheetPrimitive.Description
104 |     ref={ref}
105 |     className={cn("text-sm text-muted-foreground", className)}
106 |     {...props}
107 |   />
108 | ));
109 | SheetDescription.displayName = SheetPrimitive.Description.displayName;
110 |
111 | export {
112 |   Sheet,
113 |   SheetPortal,
114 |   SheetOverlay,
115 |   SheetTrigger,
116 |   SheetClose,
117 |   SheetContent,
118 |   SheetHeader,
119 |   SheetFooter,
120 |   SheetTitle,
121 |   SheetDescription,
122 | };

```

src/components/ui/sidebar.tsx

```
1 | import * as React from "react";
2 | import { Slot } from "@radix-ui/react-slot";
3 | import { cva, type VariantProps } from "class-variance-authority";
4 | import { PanelLeft } from "lucide-react";
5 |
6 | import { useIsMobile } from "@hooks/use-mobile";
7 | import { cn } from "@lib/utils";
8 | import { Button } from "@components/ui/button";
9 | import { Input } from "@components/ui/input";
10 | import { Separator } from "@components/ui/separator";
11 | import {
12 |   Sheet,
13 |   SheetContent,
14 |   SheetDescription,
15 |   SheetHeader,
16 |   SheetTitle,
17 | } from "@components/ui/sheet";
18 | import { Skeleton } from "@components/ui/skeleton";
19 | import { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger } from "@components/ui/tooltip";
20 |
21 | const SIDEBAR_COOKIE_NAME = "sidebar_state";
22 | const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7;
23 | const SIDEBAR_WIDTH = "16rem";
24 | const SIDEBAR_WIDTH_MOBILE = "18rem";
25 | const SIDEBAR_WIDTH_ICON = "3rem";
26 | const SIDEBAR_KEYBOARD_SHORTCUT = "b";
27 |
28 | type SidebarContextProps = {
29 |   state: "expanded" | "collapsed";
30 |   open: boolean;
31 |   setOpen: (open: boolean) => void;
32 |   openMobile: boolean;
33 |   setOpenMobile: (open: boolean) => void;
34 |   isMobile: boolean;
35 |   toggleSidebar: () => void;
36 | };
37 |
38 | const SidebarContext = React.createContext<SidebarContextProps | null>(null);
39 |
40 | function useSidebar() {
41 |   const context = React.useContext(SidebarContext);
42 |   if (!context) {
43 |     throw new Error("useSidebar must be used within a SidebarProvider.");
44 |   }
45 |
46 |   return context;
47 | }
48 |
49 | const SidebarProvider = React.forwardRef<
50 |   HTMLDivElement,
51 |   React.ComponentProps<"div"> & {
52 |     defaultOpen?: boolean;
53 |     open?: boolean;
54 |     onOpenChange?: (open: boolean) => void;
55 |   }
56 | >((
57 |   {
58 |     {
59 |       defaultOpen = true,
60 |       open: openProp,
61 |       onOpenChange: setOpenProp,
62 |       className,
63 |       style,
64 |       children,
65 |       ...props
66 |     },
67 |     ref,
68 |   ) => {
69 |     const isMobile = useIsMobile();
70 |     const [openMobile, setOpenMobile] = React.useState(false);
71 |
72 |     // This is the internal state of the sidebar.
73 |     // We use openProp and setOpenProp for control from outside the component.
74 |     const [_open, _setOpen] = React.useState(defaultOpen);
75 |     const open = openProp ?? _open;
76 |     const setOpen = React.useCallback(
77 |       (value: boolean | ((value: boolean) => boolean)) => {
78 |         const openState = typeof value === "function" ? value(open) : value;
79 |         if (setOpenProp) {
80 |           setOpenProp(openState);
81 |         } else {
82 |           _setOpen(openState);
83 |         }
84 |       },
85 |       // This sets the cookie to keep the sidebar state.
86 |       document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/; max-age=${SIDEBAR_COOKIE_MAX_AGE}`;
87 |     ),
88 |     [setOpenProp, open],
89 |   );
90 |
```

```

91 | // Helper to toggle the sidebar.
92 | const toggleSidebar = React.useCallback(() => {
93 |   return isMobile ? setOpenMobile((open) => !open) : setOpen((open) => !open);
94 | }, [isMobile, setOpen, setOpenMobile]);
95 |
96 | // Adds a keyboard shortcut to toggle the sidebar.
97 | React.useEffect(() => {
98 |   const handleKeyDown = (event: KeyboardEvent) => {
99 |     if (event.key === SIDEBAR_KEYBOARD_SHORTCUT && (event.metaKey || event.ctrlKey)) {
100 |       event.preventDefault();
101 |       toggleSidebar();
102 |     }
103 |   };
104 |
105 |   window.addEventListener("keydown", handleKeyDown);
106 |   return () => window.removeEventListener("keydown", handleKeyDown);
107 | }, [toggleSidebar]);
108 |
109 | // We add a state so that we can do data-state="expanded" or "collapsed".
110 | // This makes it easier to style the sidebar with Tailwind classes.
111 | const state = open ? "expanded" : "collapsed";
112 |
113 | const contextValue = React.useMemo<SidebarContextProps>(
114 |   () => ({
115 |     state,
116 |     open,
117 |     setOpen,
118 |     isMobile,
119 |     openMobile,
120 |     setOpenMobile,
121 |     toggleSidebar,
122 |   }),
123 |   [state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar],
124 | );
125 |
126 | return (
127 |   <SidebarContext.Provider value={contextValue}>
128 |     <TooltipProvider delayDuration={0}>
129 |       <div
130 |         style={
131 |           {
132 |             "--sidebar-width": SIDEBAR_WIDTH,
133 |             "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
134 |             ...style,
135 |           } as React.CSSProperties
136 |         }
137 |         className={cn(
138 |           "group/sidebar-wrapper flex min-h-svh w-full has-[[data-variant=inset]]:bg-sidebar",
139 |           className,
140 |         )}
141 |         ref={ref}
142 |         {...props}
143 |       >
144 |         {children}
145 |       </div>
146 |     </TooltipProvider>
147 |   </SidebarContext.Provider>
148 | );
149 | },
150 | );
151 | SidebarProvider.displayName = "SidebarProvider";
152 |
153 | const Sidebar = React.forwardRef<
154 |   HTMLDivElement,
155 |   React.ComponentProps<"div"> & {
156 |     side?: "left" | "right";
157 |     variant?: "sidebar" | "floating" | "inset";
158 |     collapsible?: "offcanvas" | "icon" | "none";
159 |   }
160 | >((
161 |   {
162 |     {
163 |       side = "left",
164 |       variant = "sidebar",
165 |       collapsible = "offcanvas",
166 |       className,
167 |       children,
168 |       ...props
169 |     },
170 |     ref,
171 |   ) => {
172 |     const { isMobile, state, openMobile, setOpenMobile } = useSidebar();
173 |
174 |     if (collapsible === "none") {
175 |       return (
176 |         <div
177 |           className={cn(
178 |             "flex h-full w-(--sidebar-width) flex-col bg-sidebar text-sidebar-foreground",
179 |             className,
180 |           )}
181 |           ref={ref}
182 |           {...props}
183 |         >

```

```

184 |         {children}
185 |       </div>
186 |     );
187 |   }
188 |
189 |   if (isMobile) {
190 |     return (
191 |       <Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
192 |         <SheetContent
193 |           data-sidebar="sidebar"
194 |           data-mobile="true"
195 |           className="w-(--sidebar-width) bg-sidebar p-0 text-sidebar-foreground [&button]:hidden"
196 |           style={
197 |             {
198 |               "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
199 |             } as React.CSSProperties
200 |           >
201 |             side={side}
202 |           >
203 |             <SheetHeader className="sr-only">
204 |               <SheetTitle>Sidebar</SheetTitle>
205 |               <SheetDescription>Displays the mobile sidebar.</SheetDescription>
206 |             </SheetHeader>
207 |             <div className="flex h-full w-full flex-col">{children}</div>
208 |           </SheetContent>
209 |         </Sheet>
210 |       );
211 |     }
212 |
213 |     return (
214 |       <div
215 |         ref={ref}
216 |         className="group peer hidden text-sidebar-foreground md:block"
217 |         data-state={state}
218 |         data-collapsible={state === "collapsed" ? collapsible : ""}
219 |         data-variant={variant}
220 |         data-side={side}
221 |       >
222 |         {/* This is what handles the sidebar gap on desktop */}
223 |         <div
224 |           className={cn(
225 |             "relative w-(--sidebar-width) bg-transparent transition-[width] duration-200 ease-linear",
226 |             "group-data-[collapsible=offcanvas]:w-0",
227 |             "group-data-[side=right]:rotate-180",
228 |             variant === "floating" || variant === "inset"
229 |               ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_theme(spacing.4))]"
230 |               : "group-data-[collapsible=icon]:w-(--sidebar-width-icon)",
231 |           )}
232 |         />
233 |         <div
234 |           className={cn(
235 |             "fixed inset-y-0 z-10 hidden h-svh w-(--sidebar-width) transition-[left,right,width] duration-200 ease-linear md:flex",
236 |             side === "left"
237 |               ? "left-0 group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-1)]"
238 |               : "right-0 group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)*-1)]",
239 |             // Adjust the padding for floating and inset variants.
240 |             variant === "floating" || variant === "inset"
241 |               ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_theme(spacing.4)_+2px)]"
242 |               : "group-data-[collapsible=icon]:w-(--sidebar-width-icon) group-data-[side=left]:border-r group-data-[side=right]:border-l",
243 |             className,
244 |           )}
245 |           {...props}
246 |         >
247 |           <div
248 |             data-sidebar="sidebar"
249 |             className="flex h-full w-full flex-col bg-sidebar group-data-[variant=floating]:rounded-lg group-data-[variant=floating]:border group-data-[variant=floating]:border-sidebar-border group-data-[variant=floating]:shadow"
250 |           >
251 |             {children}
252 |           </div>
253 |         </div>
254 |       </div>
255 |     );
256 |   },
257 | );
258 | Sidebar.displayName = "Sidebar";
259 |
260 | const SidebarTrigger = React.forwardRef<
261 |   React.ElementRef<typeof Button>,
262 |   React.ComponentProps<typeof Button>
263 | >(({ className, onClick, ...props }, ref) => {
264 |   const { toggleSidebar } = useSidebar();
265 |
266 |   return (
267 |     <Button
268 |       ref={ref}
269 |       data-sidebar="trigger"
270 |       variant="ghost"
271 |       size="icon"
272 |       className={cn("h-7 w-7", className)}
273 |       onClick={onClick} => {

```

```

274 |         onClick?.(event);
275 |         toggleSidebar();
276 |     }}
277 |     {...props}
278 | >
279 |     <PanelLeft />
280 |     <span className="sr-only">Toggle Sidebar</span>
281 | </Button>
282 | );
283 | });
284 | SidebarTrigger.displayName = "SidebarTrigger";
285 |
286 | const SidebarRail = React.forwardRef<HTMLButtonElement, React.ComponentProps<"button">>(
287 |   ({ className, ...props }, ref) => {
288 |     const { toggleSidebar } = useSidebar();
289 |
290 |     return (
291 |       <button
292 |         ref={ref}
293 |         data-sidebar="rail"
294 |         aria-label="Toggle Sidebar"
295 |         tabIndex={-1}
296 |         onClick={toggleSidebar}
297 |         title="Toggle Sidebar"
298 |         className={cn(
299 |           "absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all ease-linear after:absolute after:inset-y
-0 after:left-1/2 after:w-[2px] hover:after:bg-sidebar-border group-data-[side=left]:-right-4 group-data-[side=rig
ht]:left-0 sm:flex",
300 |           "[[data-side=left]&]:cursor-w-resize [[data-side=right]&]:cursor-e-resize",
301 |           "[[data-side=left][data-state=collapsed]&]:cursor-e-resize [[data-side=right][data-state=collapsed]&]:curs
or-w-resize",
302 |           "group-data-[collapsible=offcanvas]:translate-x-0 group-data-[collapsible=offcanvas]:after:left-full group-d
ata-[collapsible=offcanvas]:hover:bg-sidebar",
303 |           "[[data-side=left][data-collapsible=offcanvas]&]:-right-2",
304 |           "[[data-side=right][data-collapsible=offcanvas]&]:-left-2",
305 |           className,
306 |         )}
307 |         {...props}
308 |       />
309 |     );
310 |   },
311 | );
312 | SidebarRail.displayName = "SidebarRail";
313 |
314 | const SidebarInset = React.forwardRef<HTMLDivElement, React.ComponentProps<"main">>(
315 |   ({ className, ...props }, ref) => {
316 |     return (
317 |       <main
318 |         ref={ref}
319 |         className={cn(
320 |           "relative flex w-full flex-1 flex-col bg-background",
321 |           "md:peer-data-[variant=inset]:m-2 md:peer-data-[state=collapsed]:peer-data-[variant=inset]:ml-2 md:peer-data
-[variant=inset]:ml-0 md:peer-data-[variant=inset]:rounded-xl md:peer-data-[variant=inset]:shadow",
322 |           className,
323 |         )}
324 |         {...props}
325 |       />
326 |     );
327 |   },
328 | );
329 | SidebarInset.displayName = "SidebarInset";
330 |
331 | const SidebarInput = React.forwardRef<
332 |   React.ElementRef<typeof Input>,
333 |   React.ComponentProps<typeof Input>
334 | >(({ className, ...props }, ref) => {
335 |   return (
336 |     <Input
337 |       ref={ref}
338 |       data-sidebar="input"
339 |       className={cn(
340 |         "h-8 w-full bg-background shadow-none focus-visible:ring-2 focus-visible:ring-sidebar-ring",
341 |         className,
342 |       )}
343 |       {...props}
344 |     />
345 |   );
346 | });
347 | SidebarInput.displayName = "SidebarInput";
348 |
349 | const SidebarHeader = React.forwardRef<HTMLDivElement, React.ComponentProps<"div">>(
350 |   ({ className, ...props }, ref) => {
351 |     return (
352 |       <div
353 |         ref={ref}
354 |         data-sidebar="header"
355 |         className={cn("flex flex-col gap-2 p-2", className)}
356 |         {...props}
357 |       />
358 |     );
359 |   },
360 | );
361 | SidebarHeader.displayName = "SidebarHeader";

```

```

362 |
363 | const SidebarFooter = React.forwardRef<HTMLDivElement, React.ComponentProps<"div">>(
364 |   ({ className, ...props }, ref) => {
365 |     return (
366 |       <div
367 |         ref={ref}
368 |         data-sidebar="footer"
369 |         className={cn("flex flex-col gap-2 p-2", className)}
370 |         {...props}
371 |       />
372 |     );
373 |   },
374 | );
375 | SidebarFooter.displayName = "SidebarFooter";
376 |
377 | const SidebarSeparator = React.forwardRef<
378 |   React.ElementRef<typeof Separator>,
379 |   React.ComponentProps<typeof Separator>
380 | >(({ className, ...props }, ref) => {
381 |   return (
382 |     <Separator
383 |       ref={ref}
384 |       data-sidebar="separator"
385 |       className={cn("mx-2 w-auto bg-sidebar-border", className)}
386 |       {...props}
387 |     />
388 |   );
389 | });
390 | SidebarSeparator.displayName = "SidebarSeparator";
391 |
392 | const SidebarContent = React.forwardRef<HTMLDivElement, React.ComponentProps<"div">>(
393 |   ({ className, ...props }, ref) => {
394 |     return (
395 |       <div
396 |         ref={ref}
397 |         data-sidebar="content"
398 |         className={cn(
399 |           "flex min-h-0 flex-1 flex-col gap-2 overflow-auto group-data-[collapsible=icon]:overflow-hidden",
400 |           className,
401 |         )}
402 |         {...props}
403 |       />
404 |     );
405 |   },
406 | );
407 | SidebarContent.displayName = "SidebarContent";
408 |
409 | const SidebarGroup = React.forwardRef<HTMLDivElement, React.ComponentProps<"div">>(
410 |   ({ className, ...props }, ref) => {
411 |     return (
412 |       <div
413 |         ref={ref}
414 |         data-sidebar="group"
415 |         className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
416 |         {...props}
417 |       />
418 |     );
419 |   },
420 | );
421 | SidebarGroup.displayName = "SidebarGroup";
422 |
423 | const SidebarGroupLabel = React.forwardRef<
424 |   HTMLDivElement,
425 |   React.ComponentProps<"div"> & { asChild?: boolean }
426 | >(({ className, asChild = false, ...props }, ref) => {
427 |   const Comp = asChild ? Slot : "div";
428 |
429 |   return (
430 |     <Comp
431 |       ref={ref}
432 |       data-sidebar="group-label"
433 |       className={cn(
434 |         "flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-medium text-sidebar-foreground/70 outline-none ring-sidebar-ring transition-[margin,opacity] duration-200 ease-linear focus-visible:ring-2 [&svg]:size-4 [&svg]:shrink-0",
435 |         "group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0",
436 |         className,
437 |       )}
438 |       {...props}
439 |     />
440 |   );
441 | });
442 | SidebarGroupLabel.displayName = "SidebarGroupLabel";
443 |
444 | const SidebarGroupAction = React.forwardRef<
445 |   HTMLButtonElement,
446 |   React.ComponentProps<"button"> & { asChild?: boolean }
447 | >(({ className, asChild = false, ...props }, ref) => {
448 |   const Comp = asChild ? Slot : "button";
449 |
450 |   return (
451 |     <Comp
452 |       ref={ref}

```

```

453 |     data-sidebar="group-action"
454 |     className={cn(
455 |       "absolute right-3 top-3.5 flex aspect-square w-5 items-center justify-center rounded-md p-0 text-sidebar-foreground
round outline-none ring-sidebar-ring cursor-pointer transition-transform hover:bg-sidebar-accent hover:text-sideba
r-accent-foreground focus-visible:ring-2 [>svg]:size-4 [>svg]:shrink-0",
456 |       // Increases the hit area of the button on mobile.
457 |       "after:absolute after:-inset-2 after:md:hidden",
458 |       "group-data-[collapsible=icon]:hidden",
459 |       className,
460 |     )}
461 |     {...props}
462 |   />
463 | );
464 | });
465 | SidebarGroupAction.displayName = "SidebarGroupAction";
466 |
467 | const SidebarGroupContent = React.forwardRef<HTMLDivElement, React.ComponentProps<"div">>(
468 |   ({ className, ...props }, ref) => (
469 |     <div
470 |       ref={ref}
471 |       data-sidebar="group-content"
472 |       className={cn("w-full text-sm", className)}
473 |       {...props}
474 |     />
475 |   ),
476 | );
477 | SidebarGroupContent.displayName = "SidebarGroupContent";
478 |
479 | const SidebarMenu = React.forwardRef<HTMLUListElement, React.ComponentProps<"ul">>(
480 |   ({ className, ...props }, ref) => (
481 |     <ul
482 |       ref={ref}
483 |       data-sidebar="menu"
484 |       className={cn("flex w-full min-w-0 flex-col gap-1", className)}
485 |       {...props}
486 |     />
487 |   ),
488 | );
489 | SidebarMenu.displayName = "SidebarMenu";
490 |
491 | const SidebarMenuItem = React.forwardRef<HTMLLIElement, React.ComponentProps<"li">>(
492 |   ({ className, ...props }, ref) => (
493 |     <li
494 |       ref={ref}
495 |       data-sidebar="menu-item"
496 |       className={cn("group/menu-item relative", className)}
497 |       {...props}
498 |     />
499 |   ),
500 | );
501 | SidebarMenuItem.displayName = "SidebarMenuItem";
502 |
503 | const sidebarMenuButtonVariants = cva(
504 |   "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2 text-left text-sm outline-none ring-
sidebar-ring cursor-pointer transition-[width,height,padding] hover:bg-sidebar-accent hover:text-sidebar-accent-fo
reground focus-visible:ring-2 active:bg-sidebar-accent active:text-sidebar-accent-foreground disabled:pointer-even
ts-none disabled:opacity-50 disabled:cursor-not-allowed group-has-[[data-sidebar=menu-action]]/menu-item:pr-8 aria
-disabled:pointer-events-none aria-disabled:opacity-50 data-[active=true]:bg-sidebar-accent data-[active=true]:fon
t-medium data-[active=true]:text-sidebar-accent-foreground data-[state=open]:hover:bg-sidebar-accent data-[state=ope
n]:hover:text-sidebar-accent-foreground group-data-[collapsible=icon]:!size-8 group-data-[collapsible=icon]:!p-2
 [>span]:last-child:truncate [>svg]:size-4 [>svg]:shrink-0",
505 |   {
506 |     variants: {
507 |       variant: {
508 |         default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
509 |         outline:
510 |           "bg-background shadow-[0_0_1px_var(--sidebar-border)] hover:bg-sidebar-accent hover:text-sidebar-accent-fo
reground hover:shadow-[0_0_1px_var(--sidebar-accent)]",
511 |       },
512 |       size: {
513 |         default: "h-8 text-sm",
514 |         sm: "h-7 text-xs",
515 |         lg: "h-12 text-sm group-data-[collapsible=icon]:!p-0",
516 |       },
517 |     },
518 |     defaultVariants: {
519 |       variant: "default",
520 |       size: "default",
521 |     },
522 |   },
523 | );
524 |
525 | const SidebarMenuButton = React.forwardRef<
526 |   HTMLButtonElement,
527 |   React.ComponentProps<"button"> & {
528 |     asChild?: boolean;
529 |     isActive?: boolean;
530 |     tooltip?: string | React.ComponentProps<typeof TooltipContent>;
531 |   } & VariantProps<typeof sidebarMenuButtonVariants>
532 | >(
533 |   (
534 |     {
535 |       asChild = false,

```

```

536 |         isActive = false,
537 |         variant = "default",
538 |         size = "default",
539 |         tooltip,
540 |         className,
541 |         ...props
542 |     },
543 |     ref,
544 | ) => {
545 |     const Comp = asChild ? Slot : "button";
546 |     const { isMobile, state } = useSidebar();
547 |
548 |     const button = (
549 |         <Comp
550 |             ref={ref}
551 |             data-sidebar="menu-button"
552 |             data-size={size}
553 |             data-active={isActive}
554 |             className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
555 |             {...props}
556 |         />
557 |     );
558 |
559 |     if (!tooltip) {
560 |         return button;
561 |     }
562 |
563 |     if (typeof tooltip === "string") {
564 |         tooltip = {
565 |             children: tooltip,
566 |         };
567 |     }
568 |
569 |     return (
570 |         <Tooltip>
571 |             <TooltipTrigger asChild>{button}</TooltipTrigger>
572 |             <TooltipContent
573 |                 side="right"
574 |                 align="center"
575 |                 hidden={state !== "collapsed" || isMobile}
576 |                 {...tooltip}
577 |             />
578 |         </Tooltip>
579 |     );
580 | },
581 | );
582 | SidebarMenuButton.displayName = "SidebarMenuButton";
583 |
584 | const SidebarMenuAction = React.forwardRef<
585 |     HTMLButtonElement,
586 |     React.ComponentProps<"button"> & {
587 |         asChild?: boolean;
588 |         showOnHover?: boolean;
589 |     }
590 | >(({ className, asChild = false, showOnHover = false, ...props }, ref) => {
591 |     const Comp = asChild ? Slot : "button";
592 |
593 |     return (
594 |         <Comp
595 |             ref={ref}
596 |             data-sidebar="menu-action"
597 |             className={cn(
598 |                 "absolute right-1 top-1.5 flex aspect-square w-5 items-center justify-center rounded-md p-0 text-sidebar-foreg
round outline-none ring-sidebar-ring cursor-pointer transition-transform hover:bg-sidebar-accent hover:text-sideba
r-accent-foreground focus-visible:ring-2 peer-hover/menu-button:text-sidebar-accent-foreground [&svg]:size-4 [&g
vg]:shrink-0",
599 |                 // Increases the hit area of the button on mobile.
600 |                 "after:absolute after:-inset-2 after:md:hidden",
601 |                 "peer-data-[size=sm]/menu-button:top-1",
602 |                 "peer-data-[size=default]/menu-button:top-1.5",
603 |                 "peer-data-[size=lg]/menu-button:top-2.5",
604 |                 "group-data-[collapsible=icon]:hidden",
605 |                 showOnHover &&
606 |                 "group-focus-within/menu-item:opacity-100 group-hover/menu-item:opacity-100 data-[state=open]:opacity-100 pe
er-data-[active=true]/menu-button:text-sidebar-accent-foreground md:opacity-0",
607 |                 className,
608 |             )}
609 |             {...props}
610 |         />
611 |     );
612 | });
613 | SidebarMenuAction.displayName = "SidebarMenuAction";
614 |
615 | const SidebarMenuBadge = React.forwardRef<HTMLDivElement, React.ComponentProps<"div">>(>
616 |     ({ className, ...props }, ref) => (
617 |         <div
618 |             ref={ref}
619 |             data-sidebar="menu-badge"
620 |             className={cn(
621 |                 "pointer-events-none absolute right-1 flex h-5 min-w-5 select-none items-center justify-center rounded-md px-1
text-xs font-medium tabular-nums text-sidebar-foreground",
622 |                 "peer-hover/menu-button:text-sidebar-accent-foreground peer-data-[active=true]/menu-button:text-sidebar-accent
-foreground",

```

```

623 |         "peer-data-[size=sm]/menu-button:top-1",
624 |         "peer-data-[size=default]/menu-button:top-1.5",
625 |         "peer-data-[size=lg]/menu-button:top-2.5",
626 |         "group-data-[collapsible=icon]:hidden",
627 |         className,
628 |     )}
629 |     {...props}
630 | />
631 | ),
632 | );
633 | SidebarMenuBadge.displayName = "SidebarMenuBadge";
634 |
635 | const SidebarMenuSkeleton = React.forwardRef<
636 |     HTMLDivElement,
637 |     React.ComponentProps<"div"> & {
638 |         showIcon?: boolean;
639 |     }
640 | >(({ className, showIcon = false, ...props }, ref) => {
641 |     // Random width between 50 to 90%.
642 |     const width = React.useMemo(() => {
643 |         return `${Math.floor(Math.random() * 40) + 50}%`;
644 |     }, []);
645 |
646 |     return (
647 |         <div
648 |             ref={ref}
649 |             data-sidebar="menu-skeleton"
650 |             className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
651 |             {...props}
652 |         >
653 |             {showIcon && <Skeleton className="size-4 rounded-md" data-sidebar="menu-skeleton-icon" />}
654 |             <Skeleton
655 |                 className="h-4 max-w-(--skeleton-width) flex-1"
656 |                 data-sidebar="menu-skeleton-text"
657 |                 style={
658 |                     {
659 |                         "--skeleton-width": width,
660 |                     } as React.CSSProperties
661 |                 }
662 |             />
663 |         </div>
664 |     );
665 | });
666 | SidebarMenuSkeleton.displayName = "SidebarMenuSkeleton";
667 |
668 | const SidebarMenuSub = React.forwardRef<HTMLUListElement, React.ComponentProps<"ul">>(
669 |     ({ className, ...props }, ref) => (
670 |         <ul
671 |             ref={ref}
672 |             data-sidebar="menu-sub"
673 |             className={cn(
674 |                 "mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l border-sidebar-border px-2.5 py-0.5",
675 |                 "group-data-[collapsible=icon]:hidden",
676 |                 className,
677 |             )}
678 |             {...props}
679 |         />
680 |     ),
681 | );
682 | SidebarMenuSub.displayName = "SidebarMenuSub";
683 |
684 | const SidebarMenuSubItem = React.forwardRef<HTMLLIElement, React.ComponentProps<"li">>(
685 |     ({ ...props }, ref) => <li ref={ref} {...props} />,
686 | );
687 | SidebarMenuSubItem.displayName = "SidebarMenuSubItem";
688 |
689 | const SidebarMenuSubButton = React.forwardRef<
690 |     HTMLAnchorElement,
691 |     React.ComponentProps<"a"> & {
692 |         asChild?: boolean;
693 |         size?: "sm" | "md";
694 |         isActive?: boolean;
695 |     }
696 | >(({ asChild = false, size = "md", isActive, className, ...props }, ref) => {
697 |     const Comp = asChild ? Slot : "a";
698 |
699 |     return (
700 |         <Comp
701 |             ref={ref}
702 |             data-sidebar="menu-sub-button"
703 |             data-size={size}
704 |             data-active={isActive}
705 |             className={cn(
706 |                 "flex h-7 min-w-0 -translate-x-px items-center gap-2 overflow-hidden rounded-md px-2 text-sidebar-foreground o
utline-none ring-sidebar-ring cursor-pointer hover:bg-sidebar-accent hover:text-sidebar-accent-foreground focus-vi
sible:ring-2 active:bg-sidebar-accent active:text-sidebar-foreground disabled:pointer-events-none disabled:
opacity-50 disabled:cursor-not-allowed aria-disabled:pointer-events-none aria-disabled:opacity-50 [&gt;span]:last-chi
ld:truncate [&gt;svg]:size-4 [&gt;svg]:shrink-0 [&gt;svg]:text-sidebar-accent-foreground",
707 |                 "data-[active=true]:bg-sidebar-accent data-[active=true]:text-sidebar-accent-foreground",
708 |                 size === "sm" && "text-xs",
709 |                 size === "md" && "text-sm",
710 |                 "group-data-[collapsible=icon]:hidden",
711 |                 className,

```

```

712 |         })}
713 |         {...props}
714 |     />
715 |     );
716 | });
717 | SidebarMenuSubButton.displayName = "SidebarMenuSubButton";
718 |
719 | export {
720 |     Sidebar,
721 |     SidebarContent,
722 |     SidebarFooter,
723 |     SidebarGroup,
724 |     SidebarGroupAction,
725 |     SidebarGroupContent,
726 |     SidebarGroupLabel,
727 |     SidebarHeader,
728 |     SidebarInput,
729 |     SidebarInset,
730 |     SidebarMenu,
731 |     SidebarMenuAction,
732 |     SidebarMenuBadge,
733 |     SidebarMenuButton,
734 |     SidebarMenuItem,
735 |     SidebarMenuSkeleton,
736 |     SidebarMenuSub,
737 |     SidebarMenuSubButton,
738 |     SidebarMenuSubItem,
739 |     SidebarProvider,
740 |     SidebarRail,
741 |     SidebarSeparator,
742 |     SidebarTrigger,
743 |     useSidebar,
744 | };

```

src/components/ui/skeleton.tsx

```
1 | import { cn } from "@lib/utils";
2 |
3 | function Skeleton({ className, ...props }: React.HTMLAttributes<HTMLDivElement>) {
4 |   return <div className={cn("animate-pulse rounded-md bg-primary/10", className)} {...props} />;
5 | }
6 |
7 | export { Skeleton };
```

src/components/ui/slider.tsx

```
1 | import * as React from "react";
2 | import * as SliderPrimitive from "@radix-ui/react-slider";
3 |
4 | import { cn } from "@lib/utls";
5 |
6 | const Slider = React.forwardRef<
7 |   React.ElementRef<typeof SliderPrimitive.Root>,
8 |   React.ComponentPropsWithoutRef<typeof SliderPrimitive.Root>
9 | >(({ className, ...props }, ref) => (
10 |   <SliderPrimitive.Root
11 |     ref={ref}
12 |     className={cn("relative flex w-full touch-none select-none items-center", className)}
13 |     {...props}
14 |   >
15 |     <SliderPrimitive.Track className="relative h-1.5 w-full grow overflow-hidden rounded-full bg-primary/20">
16 |       <SliderPrimitive.Range className="absolute h-full bg-primary" />
17 |     </SliderPrimitive.Track>
18 |     <SliderPrimitive.Thumb className="block h-4 w-4 rounded-full border border-primary/50 bg-background shadow transit
19 |       ion-colors focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none di
20 |       sabled:opacity-50" />
21 |   </SliderPrimitive.Root>
22 | ));
23 | Slider.displayName = SliderPrimitive.Root.displayName;
24 |
25 | export { Slider };
```

src/components/ui/sonner.tsx

```
1 | import { Toaster as Sonner } from "sonner";
2 |
3 | type ToasterProps = React.ComponentProps<typeof Sonner>;
4 |
5 | const Toaster = ({ ...props }: ToasterProps) => {
6 |   return (
7 |     <Sonner
8 |       className="toaster group"
9 |       toastOptions={{
10 |         classNames: {
11 |           toast:
12 |             "group toast group-[.toaster]:bg-background group-[.toaster]:text-foreground group-[.toaster]:border-border group-[.toaster]:shadow-lg",
13 |           description: "group-[.toast]:text-muted-foreground",
14 |           actionButton: "group-[.toast]:bg-primary group-[.toast]:text-primary-foreground",
15 |           cancelButton: "group-[.toast]:bg-muted group-[.toast]:text-muted-foreground",
16 |         },
17 |       }}
18 |     >{...props}</>
19 |   );
20 | };
21 |
22 | export { Toaster };
```

src/components/ui/switch.tsx

```
1 | import * as React from "react";
2 | import * as SwitchPrimitives from "@radix-ui/react-switch";
3 |
4 | import { cn } from "@lib/utls";
5 |
6 | const Switch = React.forwardRef<
7 |   React.ElementRef<typeof SwitchPrimitives.Root>,
8 |   React.ComponentPropsWithoutRef<typeof SwitchPrimitives.Root>
9 | >(({ className, ...props }, ref) => (
10 |   <SwitchPrimitives.Root
11 |     className={cn(
12 |       "peer inline-flex h-5 w-9 shrink-0 cursor-pointer items-center rounded-full border-2 border-transparent shadow-s
13 | m transition-colors focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-off
14 | set-2 focus-visible:ring-offset-background disabled:cursor-not-allowed disabled:opacity-50 data-[state=checked]:bg
15 | -primary data-[state=unchecked]:bg-input",
16 |       className,
17 |     )}
18 |     {...props}
19 |     ref={ref}
20 |   >
21 |     <SwitchPrimitives.Thumb
22 |       className={cn(
23 |         "pointer-events-none block h-4 w-4 rounded-full bg-background shadow-lg ring-0 transition-transform data-[stat
24 | e=checked]:translate-x-4 data-[state=unchecked]:translate-x-0",
25 |       )}
26 |     />
27 |   </SwitchPrimitives.Root>
28 | ));
29 | Switch.displayName = SwitchPrimitives.Root.displayName;
30 |
31 | export { Switch };
```

src/components/ui/table.tsx

```
1 | import * as React from "react";
2 |
3 | import { cn } from "@lib/utils";
4 |
5 | const Table = React.forwardRef<HTMLTableElement, React.HTMLAttributes<HTMLTableElement>>((
6 |   ({ className, ...props }, ref) => (
7 |     <div className="relative w-full overflow-auto">
8 |       <table ref={ref} className={cn("w-full caption-bottom text-sm", className)} {...props} />
9 |     </div>
10 |   ),
11 | );
12 | Table.displayName = "Table";
13 |
14 | const TableHeader = React.forwardRef<
15 |   HTMLTableSectionElement,
16 |   React.HTMLAttributes<HTMLTableSectionElement>
17 | >(({ className, ...props }, ref) => (
18 |   <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
19 | ));
20 | TableHeader.displayName = "TableHeader";
21 |
22 | const TableBody = React.forwardRef<
23 |   HTMLTableSectionElement,
24 |   React.HTMLAttributes<HTMLTableSectionElement>
25 | >(({ className, ...props }, ref) => (
26 |   <tbody ref={ref} className={cn("[&_tr:last-child]:border-0", className)} {...props} />
27 | ));
28 | TableBody.displayName = "TableBody";
29 |
30 | const TableFooter = React.forwardRef<
31 |   HTMLTableSectionElement,
32 |   React.HTMLAttributes<HTMLTableSectionElement>
33 | >(({ className, ...props }, ref) => (
34 |   <tfoot
35 |     ref={ref}
36 |     className={cn("border-t bg-muted/50 font-medium [&_tr]:last:border-b-0", className)}
37 |     {...props}
38 |   />
39 | ));
40 | TableFooter.displayName = "TableFooter";
41 |
42 | const TableRow = React.forwardRef<HTMLTableRowElement, React.HTMLAttributes<HTMLTableRowElement>>((
43 |   ({ className, ...props }, ref) => (
44 |     <tr
45 |       ref={ref}
46 |       className={cn(
47 |         "border-b transition-colors hover:bg-muted/50 data-[state=selected]:bg-muted",
48 |         className,
49 |       )}
50 |       {...props}
51 |     />
52 |   ),
53 | );
54 | TableRow.displayName = "TableRow";
55 |
56 | const TableHead = React.forwardRef<
57 |   HTMLTableCellElement,
58 |   React.ThHTMLAttributes<HTMLTableCellElement>
59 | >(({ className, ...props }, ref) => (
60 |   <th
61 |     ref={ref}
62 |     className={cn(
63 |       "h-10 px-2 text-left align-middle font-medium text-muted-foreground [&:has([role=checkbox])]:pr-0 [&[role=checkbox]]:translate-y-[2px]",
64 |       className,
65 |     )}
66 |     {...props}
67 |   />
68 | ));
69 | TableHead.displayName = "TableHead";
70 |
71 | const TableCell = React.forwardRef<
72 |   HTMLTableCellElement,
73 |   React.TdHTMLAttributes<HTMLTableCellElement>
74 | >(({ className, ...props }, ref) => (
75 |   <td
76 |     ref={ref}
77 |     className={cn(
78 |       "p-2 align-middle [&:has([role=checkbox])]:pr-0 [&[role=checkbox]]:translate-y-[2px]",
79 |       className,
80 |     )}
81 |     {...props}
82 |   />
83 | ));
84 | TableCell.displayName = "TableCell";
85 |
86 | const TableCaption = React.forwardRef<
87 |   HTMLTableCaptionElement,
88 |   React.HTMLAttributes<HTMLTableCaptionElement>
89 | >(({ className, ...props }, ref) => (
```

```
90 |   <caption ref={ref} className={cn("mt-4 text-sm text-muted-foreground", className)} {...props} />
91 | ));
92 | TableCaption.displayName = "TableCaption";
93 |
94 | export { Table, TableHeader, TableBody, TableFooter, TableHead, TableRow, TableCell, TableCaption };
```

src/components/ui/tabs.tsx

```
1 | import * as React from "react";
2 | import * as TabsPrimitive from "@radix-ui/react-tabs";
3 |
4 | import { cn } from "@lib/utls";
5 |
6 | const Tabs = TabsPrimitive.Root;
7 |
8 | const TabsList = React.forwardRef<
9 |   React.ElementRef<typeof TabsPrimitive.List>,
10 |   React.ComponentPropsWithoutRef<typeof TabsPrimitive.List>
11 | >(({ className, ...props }, ref) => (
12 |   <TabsPrimitive.List
13 |     ref={ref}
14 |     className={cn(
15 |       "inline-flex h-9 items-center justify-center rounded-lg bg-muted p-1 text-muted-foreground",
16 |       className,
17 |     )}
18 |     {...props}
19 |   />
20 | ));
21 | TabsList.displayName = TabsPrimitive.List.displayName;
22 |
23 | const TabsTrigger = React.forwardRef<
24 |   React.ElementRef<typeof TabsPrimitive.Trigger>,
25 |   React.ComponentPropsWithoutRef<typeof TabsPrimitive.Trigger>
26 | >(({ className, ...props }, ref) => (
27 |   <TabsPrimitive.Trigger
28 |     ref={ref}
29 |     className={cn(
30 |       "inline-flex items-center justify-center whitespace-nowrap rounded-md px-3 py-1 text-sm font-medium ring-offset-
31 |       background cursor-pointer transition-all focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring f
32 |       ocus-visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed data-[stat
33 |       e=active]:bg-background data-[state=active]:text-foreground data-[state=active]:shadow",
34 |       className,
35 |     )}
36 |     {...props}
37 |   />
38 | ));
39 | TabsTrigger.displayName = TabsPrimitive.Trigger.displayName;
40 |
41 | const TabsContent = React.forwardRef<
42 |   React.ElementRef<typeof TabsPrimitive.Content>,
43 |   React.ComponentPropsWithoutRef<typeof TabsPrimitive.Content>
44 | >(({ className, ...props }, ref) => (
45 |   <TabsPrimitive.Content
46 |     ref={ref}
47 |     className={cn(
48 |       "mt-2 ring-offset-background focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visib
49 |       le:ring-offset-2",
50 |       className,
51 |     )}
52 |     {...props}
53 |   />
54 | ));
55 | TabsContent.displayName = TabsPrimitive.Content.displayName;
56 |
57 | export { Tabs, TabsList, TabsTrigger, TabsContent };
```

src/components/ui/textarea.tsx

```
1 | import * as React from "react";
2 |
3 | import { cn } from "@lib/utils";
4 |
5 | const Textarea = React.forwardRef<HTMLTextAreaElement, React.ComponentProps<"textarea">>({
6 |   className, ...props }, ref) => {
7 |   return (
8 |     <textarea
9 |       className={cn(
10 |        "flex min-h-[60px] w-full rounded-md border border-input bg-transparent px-3 py-2 text-base shadow-sm placeh
11 |        older:text-muted-foreground focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:curso
12 |        r-not-allowed disabled:opacity-50 md:text-sm",
13 |        className,
14 |      )}
15 |       ref={ref}
16 |       {...props}
17 |     />
18 |   );
19 |   Textarea.displayName = "Textarea";
20 |
21 | export { Textarea };
```

src/components/ui/toggle-group.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group";
5 | import { type VariantProps } from "class-variance-authority";
6 |
7 | import { cn } from "@lib/utils";
8 | import { toggleVariants } from "@components/ui/toggle";
9 |
10 | const ToggleGroupContext = React.createContext<VariantProps<typeof toggleVariants>>({
11 |   size: "default",
12 |   variant: "default",
13 | });
14 |
15 | const ToggleGroup = React.forwardRef<
16 |   React.ElementRef<typeof ToggleGroupPrimitive.Root>,
17 |   React.ComponentPropsWithoutRef<typeof ToggleGroupPrimitive.Root> &
18 |     VariantProps<typeof toggleVariants>
19 | >(({ className, variant, size, children, ...props }, ref) => {
20 |   <ToggleGroupPrimitive.Root
21 |     ref={ref}
22 |     className={cn("flex items-center justify-center gap-1", className)}
23 |     {...props}
24 |   >
25 |     <ToggleGroupContext.Provider value={{ variant, size }}>{children}</ToggleGroupContext.Provider>
26 |   </ToggleGroupPrimitive.Root>
27 | );
28 |
29 | ToggleGroup.displayName = ToggleGroupPrimitive.Root.displayName;
30 |
31 | const ToggleGroupItem = React.forwardRef<
32 |   React.ElementRef<typeof ToggleGroupPrimitive.Item>,
33 |   React.ComponentPropsWithoutRef<typeof ToggleGroupPrimitive.Item> &
34 |     VariantProps<typeof toggleVariants>
35 | >(({ className, children, variant, size, ...props }, ref) => {
36 |   const context = React.useContext(ToggleGroupContext);
37 |
38 |   return (
39 |     <ToggleGroupPrimitive.Item
40 |       ref={ref}
41 |       className={cn(
42 |         toggleVariants({
43 |           variant: context.variant || variant,
44 |           size: context.size || size,
45 |         }),
46 |         className,
47 |       )}
48 |       {...props}
49 |     >
50 |       {children}
51 |     </ToggleGroupPrimitive.Item>
52 |   );
53 | });
54 |
55 | ToggleGroupItem.displayName = ToggleGroupPrimitive.Item.displayName;
56 |
57 | export { ToggleGroup, ToggleGroupItem };
```

src/components/ui/toggle.tsx

```
1 | import * as React from "react";
2 | import * as TogglePrimitive from "@radix-ui/react-toggle";
3 | import { cva, type VariantProps } from "class-variance-authority";
4 |
5 | import { cn } from "@lib/utils";
6 |
7 | const toggleVariants = cva(
8 |   "inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium cursor-pointer transition-colors hover
   |   :bg-muted hover:text-muted-foreground focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disa
   |   bled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed data-[state=on]:bg-accent data-[state=on]
   |   :text-accent-foreground [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
9 |   {
10 |     variants: {
11 |       variant: {
12 |         default: "bg-transparent",
13 |         outline:
14 |           "border border-input bg-transparent shadow-sm hover:bg-accent hover:text-accent-foreground",
15 |       },
16 |       size: {
17 |         default: "h-9 px-2 min-w-9",
18 |         sm: "h-8 px-1.5 min-w-8",
19 |         lg: "h-10 px-2.5 min-w-10",
20 |       },
21 |     },
22 |     defaultVariants: {
23 |       variant: "default",
24 |       size: "default",
25 |     },
26 |   },
27 | );
28 |
29 | const Toggle = React.forwardRef<
30 |   React.ElementRef<typeof TogglePrimitive.Root>,
31 |   React.ComponentPropsWithoutRef<typeof TogglePrimitive.Root> & VariantProps<typeof toggleVariants>
32 | >(({ className, variant, size, ...props }, ref) => (
33 |   <TogglePrimitive.Root
34 |     ref={ref}
35 |     className={cn(toggleVariants({ variant, size, className }))}
36 |     {...props}
37 |   />
38 | ));
39 |
40 | Toggle.displayName = TogglePrimitive.Root.displayName;
41 |
42 | export { Toggle, toggleVariants };
```

src/components/ui/tooltip.tsx

```
1 | "use client";
2 |
3 | import * as React from "react";
4 | import * as TooltipPrimitive from "@radix-ui/react-tooltip";
5 |
6 | import { cn } from "@lib/utls";
7 |
8 | const TooltipProvider = TooltipPrimitive.Provider;
9 |
10 | const Tooltip = TooltipPrimitive.Root;
11 |
12 | const TooltipTrigger = TooltipPrimitive.Trigger;
13 |
14 | const TooltipContent = React.forwardRef<
15 |   React.ElementRef<typeof TooltipPrimitive.Content>,
16 |   React.ComponentPropsWithoutRef<typeof TooltipPrimitive.Content>
17 | >(({ className, sideOffset = 4, ...props }, ref) => (
18 |   <TooltipPrimitive.Portal>
19 |     <TooltipPrimitive.Content
20 |       ref={ref}
21 |       sideOffset={sideOffset}
22 |       className={cn(
23 |         "z-50 overflow-hidden rounded-md bg-primary px-3 py-1.5 text-xs text-primary-foreground animate-in fade-in-0 zoom-in-95 data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=closed]:zoom-out-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-(--radix-tooltip-content-transform-origin)",
24 |         className,
25 |       )}
26 |       {...props}
27 |     />
28 |   </TooltipPrimitive.Portal>
29 | ));
30 | TooltipContent.displayName = TooltipPrimitive.Content.displayName;
31 |
32 | export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider };
```

src/hooks/use-mobile.tsx

```
1 | import * as React from "react";
2 |
3 | const MOBILE_BREAKPOINT = 768;
4 |
5 | export function useIsMobile() {
6 |   const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined);
7 |
8 |   React.useEffect(() => {
9 |     const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
10 |     const onChange = () => {
11 |       setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
12 |     };
13 |     mql.addEventListener("change", onChange);
14 |     setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
15 |     return () => mql.removeEventListener("change", onChange);
16 |   }, []);
17 |
18 |   return !!isMobile;
19 | }
```

src/lib/api/example.functions.ts

```
1 | import { createServerFn } from "@tanstack/react-start";
2 | import { z } from "zod";
3 |
4 | import { getServerConfig } from "../config.server";
5 |
6 | // Example createServerFn. Server-side handler invoked from the client:
7 | //   const result = await getGreeting({ data: { name: "Ada" } })
8 | // The .handler body runs server-only – imports used only inside it (like
9 | // .server.ts modules) are tree-shaken from the client bundle. Module-level
10 | // code here still ships to the client; for truly server-only helpers, put
11 | // them in a .server.ts file. Use this pattern instead of Supabase Edge
12 | // Functions for server logic.
13 |
14 | export const getGreeting = createServerFn({ method: "POST" })
15 |   .inputValidator(z.object({ name: z.string().min(1) }))
16 |   .handler(async ({ data }) => {
17 |     const config = getServerConfig();
18 |     return {
19 |       greeting: `Hello, ${data.name}!`,
20 |       mode: config.nodeEnv ?? "unknown",
21 |     };
22 |   });
```

src/lib/camera-error.ts

```
1 | // Turn a getUserMedia failure into an actionable message for the user.
2 | export function describeCameraError(e: unknown): string {
3 |   const err = e as { name?: string; message?: string } | null;
4 |   const name = err?.name ?? "";
5 |   if (name === "NotAllowedError" || name === "SecurityError") {
6 |     return "Camera permission denied. Click the 📷 / camera icon in your browser's address bar, set Camera to Allow, then reload this page.";
7 |   }
8 |   if (name === "NotFoundError" || name === "OverconstrainedError") {
9 |     return "No camera found on this device.";
10 |   }
11 |   if (name === "NotReadableError") {
12 |     return "Camera is already in use by another app. Close it (Zoom, Meet, other tabs) and try again.";
13 |   }
14 |   if (name === "AbortError") {
15 |     return "Camera start was interrupted. Try again.";
16 |   }
17 |   if (typeof window !== "undefined" && !window.isSecureContext) {
18 |     return "Camera needs a secure (https://) page. Open the published/preview URL over HTTPS.";
19 |   }
20 |   return err?.message || "Could not access camera";
21 | }
```

src/lib/config.server.ts

```
1 | import process from "node:process";
2 |
3 | // Server-only config. The .server.ts suffix prevents Vite from bundling
4 | // this file into the client – values here never reach the browser.
5 | //
6 | // On Cloudflare Workers, env binds at REQUEST time. Module-scope reads
7 | // (e.g. `const x = process.env.X`) resolve to undefined – always read
8 | // process.env INSIDE a function or handler.
9 | //
10 | // When to use which env-access pattern:
11 | //   - .server.ts module (this file): server-only helpers reused across
12 | //     handlers. Wrap reads in a function so they run per-request.
13 | //   - inline process.env inside a createServerFn handler: one-off reads
14 | //     not reused elsewhere.
15 | //   - import.meta.env.VITE_FOO: PUBLIC config readable from both client
16 | //     and server (analytics IDs, public URLs). Define in .env with the
17 | //     VITE_ prefix. Never put secrets here – they ship to the browser.
18 |
19 | export function getServerConfig() {
20 |   return {
21 |     nodeEnv: process.env.NODE_ENV,
22 |     // Add server-only values here, e.g.:
23 |     //   databaseUrl: process.env.DATABASE_URL,
24 |     //   stripeSecretKey: process.env.STRIPE_SECRET_KEY,
25 |   };
26 | }
```

src/lib/detector.ts

```
1 | import * as cocoSsd from "@tensorflow-models/coco-ssd";
2 | import "@tensorflow/tfjs";
3 |
4 | let modelPromise: Promise<cocoSsd.ObjectDetection> | null = null;
5 |
6 | export function loadModel() {
7 |   if (!modelPromise) {
8 |     modelPromise = cocoSsd.load({ base: "lite_mobilenet_v2" });
9 |   }
10 |   return modelPromise;
11 | }
12 |
13 | export type Detection = cocoSsd.DetectedObject;
14 |
15 | export async function detectPeople(
16 |   input: HTMLVideoElement | HTMLImageElement | HTMLCanvasElement,
17 | ): Promise<Detection[]> {
18 |   const model = await loadModel();
19 |   const predictions = await model.detect(input);
20 |   return predictions.filter((p) => p.class === "person");
21 | }
```

src/lib/error-capture.ts

```
1 | // Captures the original Error out-of-band so server.ts can recover the stack
2 | // when h3 has already swallowed the throw into a generic 500 Response.
3 |
4 | let lastCapturedError: { error: unknown; at: number } | undefined;
5 | const TTL_MS = 5_000;
6 |
7 | function record(error: unknown) {
8 |   lastCapturedError = { error, at: Date.now() };
9 | }
10 |
11 | if (typeof globalThis.addEventListener === "function") {
12 |   globalThis.addEventListener("error", (event) => record((event as ErrorEvent).error ?? event));
13 |   globalThis.addEventListener("unhandledrejection", (event) =>
14 |     record((event as PromiseRejectionEvent).reason),
15 |   );
16 | }
17 |
18 | export function consumeLastCapturedError(): unknown {
19 |   if (!lastCapturedError) return undefined;
20 |   if (Date.now() - lastCapturedError.at > TTL_MS) {
21 |     lastCapturedError = undefined;
22 |     return undefined;
23 |   }
24 |   const { error } = lastCapturedError;
25 |   lastCapturedError = undefined;
26 |   return error;
27 | }
```

src/lib/error-page.ts

```
1 | export function renderErrorPage(): string {
2 |   return `<!doctype html>
3 | <html lang="en">
4 |   <head>
5 |     <meta charset="utf-8" />
6 |     <title>This page didn't load</title>
7 |     <meta name="viewport" content="width=device-width, initial-scale=1" />
8 |     <style>
9 |       body { font: 15px/1.5 system-ui, -apple-system, sans-serif; background: #fafafa; color: #111; display: grid; pla
10 | ce-items: center; min-height: 100vh; margin: 0; padding: 1.5rem; }
11 |       .card { max-width: 28rem; width: 100%; text-align: center; padding: 2rem; }
12 |       h1 { font-size: 1.25rem; margin: 0 0 0.5rem; }
13 |       p { color: #4b5563; margin: 0 0 1.5rem; }
14 |       .actions { display: flex; gap: 0.5rem; justify-content: center; flex-wrap: wrap; }
15 |       a, button { padding: 0.5rem 1rem; border-radius: 0.375rem; font: inherit; cursor: pointer; text-decoration: none
16 | ; border: 1px solid transparent; }
17 |       .primary { background: #111; color: #fff; }
18 |       .secondary { background: #fff; color: #111; border-color: #d1d5db; }
19 |     </style>
20 |   </head>
21 |   <body>
22 |     <div class="card">
23 |       <h1>This page didn't load</h1>
24 |       <p>Something went wrong on our end. You can try refreshing or head back home.</p>
25 |       <div class="actions">
26 |         <button class="primary" onclick="location.reload()">Try again</button>
27 |         <a class="secondary" href="/">Go home</a>
28 |       </div>
29 |     </div>
30 |   </body>
31 | </html>`;
32 | }
```

src/lib/lovable-error-reporting.ts

```
1 | type LovableErrorOptions = {
2 |   mechanism?: "manual" | "onerror" | "unhandledrejection" | "react_error_boundary";
3 |   handled?: boolean;
4 |   severity?: "error" | "warning" | "info";
5 | };
6 |
7 | type LovableEvents = {
8 |   captureException?: (
9 |     error: unknown,
10 |     context?: Record<string, unknown>,
11 |     options?: LovableErrorOptions,
12 |   ) => void;
13 | };
14 |
15 | declare global {
16 |   interface Window {
17 |     __lovableEvents?: LovableEvents;
18 |   }
19 | }
20 |
21 | export function reportLovableError(error: unknown, context: Record<string, unknown> = {}) {
22 |   if (typeof window === "undefined") return;
23 |   window.__lovableEvents?.captureException?.(
24 |     error,
25 |     {
26 |       source: "react_error_boundary",
27 |       route: window.location.pathname,
28 |       ...context,
29 |     },
30 |     {
31 |       mechanism: "react_error_boundary",
32 |       handled: false,
33 |       severity: "error",
34 |     },
35 |   );
36 | }
```

src/lib/pratisha-content.ts

```
1 | // Translated content for the PRATISHA reference sections.
2 | // Languages without an entry fall back to English.
3 |
4 | export type Content = {
5 |   about: { eyebrow: string; title: string; p1: string; p2: string; p3: string };
6 |   components: {
7 |     eyebrow: string;
8 |     title: string;
9 |     intro: string;
10 |    cards: { title: string; body: string }[]; // 6
11 |  };
12 |   features: {
13 |     eyebrow: string;
14 |     title: string;
15 |     items: { label: string; body: string }[]; // 5
16 |  };
17 |   workflow: {
18 |     eyebrow: string;
19 |     title: string;
20 |     cards: { title: string; body: string }[]; // 4
21 |  };
22 |   applications: {
23 |     eyebrow: string;
24 |     title: string;
25 |     items: { title: string; body: string }[]; // 3
26 |  };
27 |   materials: {
28 |     eyebrow: string;
29 |     title: string;
30 |     intro: string;
31 |     cards: { title: string; body: string }[]; // 4
32 |  };
33 |   cost: {
34 |     eyebrow: string;
35 |     title: string;
36 |     prototypeHeading: string;
37 |     rows: string[]; // 7 labels
38 |     totallabel: string;
39 |     productionHeading: string;
40 |     productionPara: string;
41 |     lowEnd: string;
42 |     highEnd: string;
43 |  };
44 |   future: {
45 |     eyebrow: string;
46 |     title: string;
47 |     cards: { title: string; body: string }[]; // 6
48 |  };
49 |   crab: { eyebrow: string; title: string; p1: string; p2: string };
50 | };
51 |
52 | const en: Content = {
53 |   about: {
54 |     eyebrow: "About the Project",
55 |     title: "PRATISHA – Prompt Recovery And Assistance For Trapped Individuals in Structural Hazards and Accidents",
56 |     p1: "The PRATISHA Snake Robot is an advanced prototype tailored for Collapsed Structure Search and Rescue (CSSR) missions. Designed to navigate treacherous and confined environments, this innovative robotic system excels in detecting obstacles, providing real-time situational surveillance, and facilitating precise remote-controlled operations.",
57 |     p2: "Its snake-like structure enables unparalleled flexibility and maneuverability, allowing it to access areas that are hazardous or inaccessible for humans and conventional robotic systems. Equipped with cutting-edge sensors and communication technologies, the robot delivers critical data in real-time, ensuring timely and informed decision-making during rescue operations.",
58 |     p3: "By redefining the capabilities of emergency response tools, the PRATISHA Snake Robot stands as a vital asset for saving lives in high-risk, disaster-stricken zones.",
59 |   },
60 |   components: {
61 |     eyebrow: "Hardware",
62 |     title: "Components Overview",
63 |     intro: "The Pratisha Snake Robot incorporates cutting-edge components, each crucial to its exceptional performance in CSSR missions:",
64 |     cards: [
65 |       { title: "ESP32 Microcontroller", body: "The robot's central nervous system. Manages movement control, processes data, and facilitates seamless communication with external devices." },
66 |       { title: "ESP32-CAM Module", body: "High-resolution camera for live-streaming. Provides real-time reconnaissance and critical situational awareness during rescue efforts." },
67 |       { title: "HC-05 Bluetooth Module", body: "Enables wireless communication between operator and robot. Supports remote operation via smartphone or computer." },
68 |       { title: "IR Sensors", body: "Detect obstacles autonomously using infrared technology. Allow safe navigation by preventing collisions." },
69 |       { title: "Gas Hazard Detection (MQ-3)", body: "Continuously monitors hazardous gases in real-time. Alerts operators to dangerous conditions for informed decisions." },
70 |       { title: "PLA Body (3D Printed)", body: "Lightweight, biodegradable PLA segments joined by connective hinges enable serpentine mobility and rapid prototyping." },
71 |     ],
72 |   },
73 |   features: {
74 |     eyebrow: "Capabilities",
75 |     title: "Key Features",
76 |     items: [
```

```

77 |     { label: "Compact and Agile Design", body: "Highly maneuverable in tight spaces, crucial in collapsed buildings
    or debris." },
78 |     { label: "Surveillance and Monitoring", body: "Cameras and sensors (with possible thermal imaging) provide real-
    time visual and environmental data." },
79 |     { label: "Environmental Sensors", body: "Detect hazardous gases, temperature changes, and structural stability f
    or safer operations." },
80 |     { label: "Remote Navigation", body: "Operators can guide the robot into areas too dangerous for human responders
    ." },
81 |     { label: "Autonomous Features", body: "Optional autonomous navigation to detect and avoid obstacles without huma
    n input." },
82 |   ],
83 | },
84 | workflow: {
85 |   eyebrow: "How It Works",
86 |   title: "Workflow and Operation",
87 |   cards: [
88 |     { title: "Initialization", body: "On power-up, the robot establishes a Bluetooth connection with the operator's
    device and powers on the surveillance camera, preparing the system for real-time operations." },
89 |     { title: "Movement", body: "The operator sends directional commands via Bluetooth. The robot propels forward, re
    treats, or turns – demonstrating flexible, serpentine mobility." },
90 |     { title: "Obstacle Avoidance", body: "IR sensors continuously monitor surroundings. On detecting an obstacle, th
    e robot adjusts its trajectory to avoid collision." },
91 |     { title: "Surveillance", body: "The ESP32-CAM streams live video to the operator for constant reconnaissance and
    situational assessment in hazardous areas." },
92 |   ],
93 | },
94 | applications: {
95 |   eyebrow: "Use Cases",
96 |   title: "Applications",
97 |   items: [
98 |     { title: "CSSR Operations", body: "In post-disaster situations such as earthquakes or building collapses, the ro
    bot slithers through rubble to locate survivors where humans cannot go." },
99 |     { title: "Search and Surveillance", body: "In environments hazardous to humans – chemical plants, nuclear facili
    ties – the robot provides real-time data and surveillance without exposing personnel to danger." },
100 |    { title: "Exploration of Confined Spaces", body: "Its serpentine form navigates constrained areas unreachable by
    conventional wheeled or tracked robots – ideal for industrial, military, and emergency applications." },
101 |   ],
102 | },
103 | materials: {
104 |   eyebrow: "Construction",
105 |   title: "Materials & Design",
106 |   intro: "The prototype is fabricated using PLA (Polylactic Acid), a biodegradable, plant-derived bioplastic. The en
    tire construction is realized through 3D printing – enabling precise, customizable, and rapid prototyping.",
107 |   cards: [
108 |     { title: "Serpentine Mobility", body: "Flexible body segments linked by connective hinges enable elegant snake-l
    ike movement through rubble and confined spaces." },
109 |     { title: "Lightweight Durability", body: "PLA fiber keeps the robot lightweight yet exceptionally durable for de
    manding search-and-rescue operations." },
110 |     { title: "Precision & Flexibility", body: "3D printing enables intricate, customized parts and seamless articula
    tion between segments." },
111 |     { title: "Real-Time Video Surveillance", body: "The integrated ESP32-CAM streams live visuals so operators can n
    avigate hazardous areas with confidence." },
112 |   ],
113 | },
114 | cost: {
115 |   eyebrow: "Economics",
116 |   title: "Cost & Scalability",
117 |   prototypeHeading: "Prototype Cost Analysis",
118 |   rows: [
119 |     "Materials (PLA Fiber)",
120 |     "ESP32 Microcontroller",
121 |     "ESP32-CAM Module",
122 |     "IR Sensors (per sensor)",
123 |     "HC-05 Bluetooth Module",
124 |     "Battery Pack",
125 |     "Miscellaneous (wiring, connectors)",
126 |   ],
127 |   totalLabel: "Total Prototype",
128 |   productionHeading: "Real-World Production Estimate",
129 |   productionPara: "The modular design and 3D printing make mass production feasible. To develop a real-world version
    of PRATISHA, the costs would range between approximately ₹12.45 Lakh and ₹49.80 Lakh (USD 150K–600K), depending o
    n complexity, components, and design choices.",
130 |   lowEnd: "Low-End Estimate",
131 |   highEnd: "High-End Estimate",
132 | },
133 | future: {
134 |   eyebrow: "Roadmap",
135 |   title: "Future Enhancements",
136 |   cards: [
137 |     { title: "AI for Autonomous Navigation", body: "AI algorithms enabling the robot to detect survivors and hazards
    without human input." },
138 |     { title: "Acoustic & Seismic Sensors", body: "Detect faint distress sounds and vibrations from survivors to impr
    ove search capabilities." },
139 |     { title: "LoRaWAN Connectivity", body: "Long-range, low-power communication to operate in remote areas without t
    raditional networks." },
140 |     { title: "LIDAR & Radar Systems", body: "Create 3D maps for precise navigation and obstacle avoidance in collaps
    ed structures." },
141 |     { title: "Human Tracking & Communication", body: "Voice/signal detection with AI-based recognition to identify s
    urvivors' calls and enhance interaction." },
142 |     { title: "Battery Swapping & Wireless Charging", body: "Automated battery swap and wireless charging to extend o
    perational time for continuous missions." },
143 |   ],
144 | },

```



```

275 | features: {
276 |   eyebrow: "സ്മാർട്ട്",
277 |   title: "സ്മാർട്ട് സെൻസർ",
278 |   items: [
279 |     { label: "സെൻസർ ഓൺ ഓഫ് സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ, സ്മാർട്ട് സ്മാർട്ട് സ്മാർട്ട് സെൻസർ സെൻസർ" },
280 |     { label: "സെൻസർ ഓൺ സെൻസർ", body: "സെൻസർ സെൻസർ (സെൻസർ സെൻസർ സെൻസർ) സെൻസർ-സെൻസർ സെൻസർ സെൻസർ" },
281 |     { label: "സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ, സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
282 |     { label: "സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
283 |     { label: "സെൻസർ സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
284 |   ],
285 | },
286 | workflow: {
287 |   eyebrow: "സ്മാർട്ട് സെൻസർ സെൻസർ",
288 |   title: "സെൻസർ സെൻസർ സെൻസർ",
289 |   cards: [
290 |     { title: "സെൻസർ സെൻസർ", body: "സെൻസർ-സെൻസർ, സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
291 |     { title: "സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
292 |     { title: "സെൻസർ സെൻസർ", body: "IR സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
293 |     { title: "സെൻസർ", body: "ESP32-CAM സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
294 |   ],
295 | },
296 | applications: {
297 |   eyebrow: "സെൻസർ സെൻസർ",
298 |   title: "സെൻസർ",
299 |   items: [
300 |     { title: "CSSR സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
301 |     { title: "സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
302 |     { title: "സെൻസർ സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
303 |   ],
304 | },
305 | materials: {
306 |   eyebrow: "സെൻസർ",
307 |   title: "സെൻസർ സെൻസർ",
308 |   intro: "സെൻസർ സെൻസർ PLA (സെൻസർ സെൻസർ) സെൻസർ സെൻസർ, സെൻസർ സെൻസർ സെൻസർ, സെൻസർ-സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ 3D സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ - സെൻസർ, സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ",
309 |   cards: [
310 |     { title: "സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
311 |     { title: "സെൻസർ സെൻസർ സെൻസർ", body: "PLA സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
312 |     { title: "സെൻസർ സെൻസർ", body: "3D സെൻസർ സെൻസർ, സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
313 |     { title: "സെൻസർ-സെൻസർ സെൻസർ സെൻസർ", body: "സെൻസർ ESP32-CAM സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
314 |   ],
315 | },
316 | cost: {
317 |   eyebrow: "സെൻസർ",
318 |   title: "സെൻസർ സെൻസർ സെൻസർ",
319 |   prototypeHeading: "സെൻസർ സെൻസർ സെൻസർ",
320 |   rows: [
321 |     "സെൻസർ (PLA സെൻസർ)",
322 |     "ESP32 സെൻസർ സെൻസർ",
323 |     "ESP32-CAM സെൻസർ",
324 |     "IR സെൻസർ (സെൻസർ സെൻസർ)",
325 |     "HC-05 സെൻസർ സെൻസർ",
326 |     "സെൻസർ സെൻസർ",
327 |     "സെൻസർ (സെൻസർ, സെൻസർ)",
328 |   ],
329 |   totalLabel: "സെൻസർ സെൻസർ",
330 |   productionHeading: "സെൻസർ-സെൻസർ സെൻസർ സെൻസർ",
331 |   productionPara: "സെൻസർ സെൻസർ 3D സെൻസർ സെൻസർ സെൻസർ സെൻസർ PRATISHA-സെൻസർ-സെൻസർ സെൻസർ സെൻസർ, സെൻസർ, സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ ₹12.45 സെൻസർ ₹49.80 സെൻസർ (USD 150K-600K) സെൻസർ സെൻസർ",
332 |   lowEnd: "സെൻസർ-സെൻസർ സെൻസർ",
333 |   highEnd: "സെൻസർ-സെൻസർ സെൻസർ",
334 | },
335 | future: {
336 |   eyebrow: "സെൻസർ",
337 |   title: "സെൻസർ സെൻസർ",
338 |   cards: [
339 |     { title: "സെൻസർ സെൻസർ സെൻസർ AI", body: "AI സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
340 |     { title: "സെൻസർ സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
341 |     { title: "LoRaWAN സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
342 |     { title: "LIDAR സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" },
343 |     { title: "സെൻസർ സെൻസർ സെൻസർ", body: "സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ സെൻസർ" }

```

[illegible]

src/lib/settings.tsx

```
1 | import { createContext, useContext, useEffect, useState, type ReactNode } from "react";
2 |
3 | export type ThemeColor = {
4 |   id: string;
5 |   label: string;
6 |   primary: string; // oklch
7 |   ring: string;
8 |   swatch: string; // hex/rgb for preview dot
9 |   hue: number; // base hue for background tinting
10 | };
11 |
12 | export const THEME_COLORS: ThemeColor[] = [
13 |   { id: "indigo", label: "Indigo", primary: "oklch(0.45 0.18 270)", ring: "oklch(0.6 0.18 270)", swatch: "#4f46e5", hue: 270 },
14 |   { id: "emerald", label: "Emerald", primary: "oklch(0.55 0.16 155)", ring: "oklch(0.65 0.16 155)", swatch: "#10b981", hue: 155 },
15 |   { id: "rose", label: "Rose", primary: "oklch(0.6 0.2 15)", ring: "oklch(0.7 0.2 15)", swatch: "#e11d48", hue: 15 },
16 |   { id: "amber", label: "Amber", primary: "oklch(0.7 0.17 70)", ring: "oklch(0.78 0.17 70)", swatch: "#f59e0b", hue: 70 },
17 |   { id: "violet", label: "Violet", primary: "oklch(0.5 0.22 300)", ring: "oklch(0.62 0.22 300)", swatch: "#8b5cf6", hue: 300 },
18 |   { id: "sky", label: "Sky", primary: "oklch(0.6 0.15 235)", ring: "oklch(0.7 0.15 235)", swatch: "#0ea5e9", hue: 235 },
19 |   { id: "teal", label: "Teal", primary: "oklch(0.58 0.13 195)", ring: "oklch(0.68 0.13 195)", swatch: "#14b8a6", hue: 195 },
20 |   { id: "crimson", label: "Crimson", primary: "oklch(0.5 0.22 25)", ring: "oklch(0.62 0.22 25)", swatch: "#dc2626", hue: 25 },
21 |   { id: "saffron", label: "Saffron", primary: "oklch(0.68 0.18 55)", ring: "oklch(0.76 0.18 55)", swatch: "#ff9933", hue: 55 },
22 | ];
23 |
24 | export type Gradient = {
25 |   id: string;
26 |   label: string;
27 |   css: string; // full CSS gradient value
28 |   preview: string; // same, used for swatch
29 | };
30 |
31 | export const GRADIENTS: Gradient[] = [
32 |   { id: "none", label: "Solid", css: "", preview: "linear-gradient(135deg, var(--primary), var(--primary))" },
33 |   { id: "sunset", label: "Sunset", css: "linear-gradient(135deg, oklch(0.7 0.2 30), oklch(0.65 0.22 350))", preview: "linear-gradient(135deg, #ff8a3d, #e0398a)" },
34 |   { id: "ocean", label: "Ocean", css: "linear-gradient(135deg, oklch(0.6 0.15 235), oklch(0.6 0.16 175))", preview: "linear-gradient(135deg, #0ea5e9, #14b8a6)" },
35 |   { id: "aurora", label: "Aurora", css: "linear-gradient(135deg, oklch(0.55 0.2 290), oklch(0.65 0.18 180))", preview: "linear-gradient(135deg, #8b5cf6, #14b8a6)" },
36 |   { id: "ember", label: "Ember", css: "linear-gradient(135deg, oklch(0.6 0.22 25), oklch(0.7 0.18 60))", preview: "linear-gradient(135deg, #dc2626, #f59e0b)" },
37 |   { id: "tricolor", label: "Tiranga", css: "linear-gradient(135deg, oklch(0.7 0.18 55), oklch(0.55 0.16 155))", preview: "linear-gradient(135deg, #ff9933, #138808)" },
38 |   { id: "forest", label: "Forest", css: "linear-gradient(135deg, oklch(0.55 0.16 155), oklch(0.45 0.12 200))", preview: "linear-gradient(135deg, #10b981, #0369a1)" },
39 |   { id: "candy", label: "Candy", css: "linear-gradient(135deg, oklch(0.75 0.18 340), oklch(0.7 0.18 260))", preview: "linear-gradient(135deg, #f472b6, #818cf8)" },
40 |   { id: "midnight", label: "Midnight", css: "linear-gradient(135deg, oklch(0.35 0.18 270), oklch(0.5 0.2 300))", preview: "linear-gradient(135deg, #312e81, #8b5cf6)" },
41 |   { id: "custom", label: "Custom", css: "", preview: "linear-gradient(135deg, #6366f1, #ec4899)" },
42 | ];
43 |
44 | type Dict = Record<string, string>;
45 | type Lang = { code: string; label: string; native: string; dict: Dict };
46 |
47 | const en: Dict = {
48 |   tagline: "Search & Rescue · Human Detection",
49 |   navDetect: "Detect",
50 |   navAbout: "About",
51 |   navComponents: "Components",
52 |   navFeatures: "Features",
53 |   navWorkflow: "Workflow",
54 |   navApplications: "Applications",
55 |   navMaterials: "Materials",
56 |   navCost: "Cost",
57 |   navFuture: "Future",
58 |   navCrab: "C.R.A.B",
59 |   liveModule: "Live Module",
60 |   humanDetection: "Human Detection",
61 |   detectIntro:
62 |     "Simulating PRATISHA's on-board vision – detect survivors in real time via webcam feed or by analyzing uploaded imagery. Runs entirely on-device.",
63 |   webcam: "Webcam",
64 |   upload: "Upload",
65 |   poweredBy: "Powered by TensorFlow.js · COCO-SSD · Your camera and images never leave your device.",
66 |   footer: "PRATISHA – Prompt Recovery And Assistance For Trapped Individuals in Structural Hazards and Accidents",
67 |   settings: "Settings",
68 |   theme: "Theme color",
69 |   gradient: "Gradient",
70 |   language: "Language",
71 |   confidence: "Confidence",
72 |   person: "Person",
73 | };
```


[illegible]

```

240 |
241 | const RTL = new Set(["ur", "ks", "sd"]);
242 |
243 | export type GradientStop = { color: string; pos: number };
244 | export type CustomGradient = { angle: number; stops: GradientStop[] };
245 |
246 | const DEFAULT_CUSTOM: CustomGradient = {
247 |   angle: 135,
248 |   stops: [
249 |     { color: "#6366f1", pos: 0 },
250 |     { color: "#ec4899", pos: 100 },
251 |   ],
252 | };
253 |
254 | export function buildGradientCss(c: CustomGradient): string {
255 |   const stops = [...c.stops].sort((a, b) => a.pos - b.pos);
256 |   const parts = stops.map((s) => `${s.color} ${s.pos}%`).join(", ");
257 |   return `linear-gradient(${c.angle}deg, ${parts})`;
258 | }
259 |
260 | export type MotionMode = "full" | "calm" | "off";
261 |
262 | type Ctx = {
263 |   color: ThemeColor;
264 |   setColor: (id: string) => void;
265 |   gradient: Gradient;
266 |   setGradient: (id: string) => void;
267 |   custom: CustomGradient;
268 |   setCustom: (c: CustomGradient) => void;
269 |   lang: Lang;
270 |   setLang: (code: string) => void;
271 |   motion: MotionMode;
272 |   setMotion: (m: MotionMode) => void;
273 |   t: (key: keyof typeof en) => string;
274 | };
275 |
276 | const SettingsCtx = createContext<Ctx | null>(null);
277 |
278 | export function SettingsProvider({ children }: { children: ReactNode }) {
279 |   const [colorId, setColorId] = useState<string>("indigo");
280 |   const [gradientId, setGradientId] = useState<string>("aurora");
281 |   const [custom, setCustom] = useState<CustomGradient>(DEFAULT_CUSTOM);
282 |   const [langCode, setLangCode] = useState<string>("en");
283 |   const [motion, setMotion] = useState<MotionMode>("full");
284 |
285 |   // hydrate from storage
286 |   useEffect(() => {
287 |     try {
288 |       const c = localStorage.getItem("pratisha.color");
289 |       const g = localStorage.getItem("pratisha.gradient");
290 |       const cu = localStorage.getItem("pratisha.custom");
291 |       const l = localStorage.getItem("pratisha.lang");
292 |       const m = localStorage.getItem("pratisha.motion") as MotionMode | null;
293 |       if (c) setColorId(c);
294 |       if (g) setGradientId(g);
295 |       if (cu) {
296 |         try {
297 |           const parsed = JSON.parse(cu);
298 |           if (parsed?.stops?.length >= 2) setCustom(parsed);
299 |         } catch {}
300 |       }
301 |       if (l) setLangCode(l);
302 |       if (m === "full" || m === "calm" || m === "off") {
303 |         setMotion(m);
304 |       } else if (window.matchMedia?.("(prefers-reduced-motion: reduce)").matches) {
305 |         setMotion("calm");
306 |       }
307 |     } catch {}
308 |   }, []);
309 |
310 |   const color = THEME_COLORS.find((c) => c.id === colorId) ?? THEME_COLORS[0];
311 |   const gradient = GRADIENTS.find((g) => g.id === gradientId) ?? GRADIENTS[0];
312 |   const lang = LANGS.find((l) => l.code === langCode) ?? LANGS[0];
313 |
314 |   useEffect(() => {
315 |     const root = document.documentElement;
316 |     const h = color.hue;
317 |     root.style.setProperty("--primary", color.primary);
318 |     root.style.setProperty("--ring", color.ring);
319 |     root.style.setProperty("--sidebar-primary", color.primary);
320 |     // Tint the whole surface palette based on hue
321 |     root.style.setProperty("--background", `oklch(0.985 0.012 ${h})`);
322 |     root.style.setProperty("--foreground", `oklch(0.18 0.03 ${h})`);
323 |     root.style.setProperty("--card", `oklch(1 0.008 ${h})`);
324 |     root.style.setProperty("--card-foreground", `oklch(0.18 0.03 ${h})`);
325 |     root.style.setProperty("--popover", `oklch(1 0.008 ${h})`);
326 |     root.style.setProperty("--popover-foreground", `oklch(0.18 0.03 ${h})`);
327 |     root.style.setProperty("--muted", `oklch(0.955 0.018 ${h})`);
328 |     root.style.setProperty("--muted-foreground", `oklch(0.48 0.04 ${h})`);
329 |     root.style.setProperty("--accent", `oklch(0.94 0.03 ${h})`);
330 |     root.style.setProperty("--accent-foreground", `oklch(0.22 0.05 ${h})`);
331 |     root.style.setProperty("--secondary", `oklch(0.94 0.025 ${h})`);
332 |     root.style.setProperty("--secondary-foreground", `oklch(0.22 0.05 ${h})`);

```

```

333 |     root.style.setProperty("--border", `oklch(0.9 0.025 ${h})`);
334 |     root.style.setProperty("--input", `oklch(0.9 0.025 ${h})`);
335 |     root.style.setProperty("--sidebar", `oklch(0.97 0.02 ${h})`);
336 |     root.style.setProperty("--sidebar-accent", `oklch(0.94 0.03 ${h})`);
337 |     root.style.setProperty("--sidebar-border", `oklch(0.9 0.025 ${h})`);
338 |     try { localStorage.setItem("pratisha.color", color.id); } catch {}
339 |   }, [color]);
340 |
341 |   useEffect(() => {
342 |     const root = document.documentElement;
343 |     let css: string;
344 |     if (gradient.id === "custom") css = buildGradientCss(custom);
345 |     else css = gradient.css || `linear-gradient(135deg, ${color.primary}, ${color.primary})`;
346 |     root.style.setProperty("--gradient-primary", css);
347 |     try { localStorage.setItem("pratisha.gradient", gradient.id); } catch {}
348 |     try { localStorage.setItem("pratisha.custom", JSON.stringify(custom)); } catch {}
349 |   }, [gradient, color, custom]);
350 |
351 |   useEffect(() => {
352 |     document.documentElement.lang = lang.code;
353 |     document.documentElement.dir = RTL.has(lang.code) ? "rtl" : "ltr";
354 |     try { localStorage.setItem("pratisha.lang", lang.code); } catch {}
355 |   }, [lang]);
356 |
357 |   useEffect(() => {
358 |     document.documentElement.setAttribute("data-motion", motion);
359 |     try { localStorage.setItem("pratisha.motion", motion); } catch {}
360 |   }, [motion]);
361 |
362 |   const t = (key: keyof typeof en) => lang.dict[key] ?? en[key];
363 |
364 |   return (
365 |     <SettingsCtx.Provider value={{ color, setColor: setColorId, gradient, setGradient: setGradientId, custom, setCustom, lang, setLang: setLangCode, motion, setMotion, t }}>
366 |       {children}
367 |     </SettingsCtx.Provider>
368 |   );
369 | }
370 |
371 |
372 |
373 | export function useSettings() {
374 |   const ctx = useContext(SettingsCtx);
375 |   if (!ctx) throw new Error("useSettings must be used inside SettingsProvider");
376 |   return ctx;
377 | }

```

src/lib/utils.ts

```
1 | import { clsx, type ClassValue } from "clsx";
2 | import { twMerge } from "tailwind-merge";
3 |
4 | export function cn(...inputs: ClassValue[]) {
5 |   return twMerge(clsx(inputs));
6 | }
```

src/router.tsx

```
1 | import { QueryClient } from "@tanstack/react-query";
2 | import { createRouter } from "@tanstack/react-router";
3 | import { routeTree } from "../routeTree.gen";
4 |
5 | export const getRouter = () => {
6 |   const queryClient = new QueryClient();
7 |
8 |   const router = createRouter({
9 |     routeTree,
10 |    context: { queryClient },
11 |    scrollRestoration: true,
12 |    defaultPreloadStaleTime: 0,
13 |  });
14 |
15 |   return router;
16 | };
```

src/routes/_root.tsx

```
1 | import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
2 | import {
3 |   Outlet,
4 |   Link,
5 |   createRootRouteWithContext,
6 |   useRouter,
7 |   HeadContent,
8 |   Scripts,
9 | } from "@tanstack/react-router";
10 | import { useEffect, type ReactNode } from "react";
11 |
12 | import appCss from "../styles.css?url";
13 | import { reportLovableError } from "../lib/lovable-error-reporting";
14 | import { SettingsProvider } from "../lib/settings";
15 |
16 | function NotFoundComponent() {
17 |   return (
18 |     <div className="flex min-h-screen items-center justify-center bg-background px-4">
19 |       <div className="max-w-md text-center">
20 |         <h1 className="text-7xl font-bold text-foreground">404</h1>
21 |         <h2 className="mt-4 text-xl font-semibold text-foreground">Page not found</h2>
22 |         <p className="mt-2 text-sm text-muted-foreground">
23 |           The page you're looking for doesn't exist or has been moved.
24 |         </p>
25 |         <div className="mt-6">
26 |           <Link
27 |             to="/"
28 |             className="inline-flex items-center justify-center rounded-md bg-primary px-4 py-2 text-sm font-medium tex
29 |             t-primary-foreground transition-colors hover:bg-primary/90"
30 |           >
31 |             Go home
32 |           </Link>
33 |         </div>
34 |       </div>
35 |     </div>
36 |   );
37 | }
38 |
39 | function ErrorComponent({ error, reset }: { error: Error; reset: () => void }) {
40 |   console.error(error);
41 |   const router = useRouter();
42 |   useEffect(() => {
43 |     reportLovableError(error, { boundary: "tanstack_root_error_component" });
44 |   }, [error]);
45 |
46 |   return (
47 |     <div className="flex min-h-screen items-center justify-center bg-background px-4">
48 |       <div className="max-w-md text-center">
49 |         <h1 className="text-xl font-semibold tracking-tight text-foreground">
50 |           This page didn't load
51 |         </h1>
52 |         <p className="mt-2 text-sm text-muted-foreground">
53 |           Something went wrong on our end. You can try refreshing or head back home.
54 |         </p>
55 |         <div className="mt-6 flex flex-wrap justify-center gap-2">
56 |           <button
57 |             onClick={() => {
58 |               router.invalidate();
59 |               reset();
60 |             }}
61 |             className="inline-flex items-center justify-center rounded-md bg-primary px-4 py-2 text-sm font-medium tex
62 |             t-primary-foreground transition-colors hover:bg-primary/90"
63 |           >
64 |             Try again
65 |           </button>
66 |           <a
67 |             href="/"
68 |             className="inline-flex items-center justify-center rounded-md border border-input bg-background px-4 py-2
69 |             text-sm font-medium text-foreground transition-colors hover:bg-accent"
70 |           >
71 |             Go home
72 |           </a>
73 |         </div>
74 |       </div>
75 |     </div>
76 |   );
77 | }
78 |
79 | export const Route = createRootRouteWithContext<{ queryClient: QueryClient }>(() => ({
80 |   head: () => ({
81 |     meta: [
82 |       { charSet: "utf-8" },
83 |       { name: "viewport", content: "width=device-width, initial-scale=1" },
84 |       { title: "PRATISHA – Human Detection for Search & Rescue" },
85 |       { name: "description", content: "PRATISHA Snake Robot: real-time human detection for Collapsed Structure Search
and Rescue (CSSR) missions. Detect survivors via webcam or uploaded images." },
86 |       { name: "author", content: "Sarthak Kumar Panda" },
87 |       { property: "og:title", content: "PRATISHA – Human Detection for Search & Rescue" },
88 |       { property: "og:description", content: "PRATISHA Snake Robot: real-time human detection for Collapsed Structure
Search and Rescue (CSSR) missions. Detect survivors via webcam or uploaded images." },
```

```

86 |     { property: "og:type", content: "website" },
87 |     { name: "twitter:card", content: "summary" },
88 |     { name: "twitter:site", content: "@pratisha" },
89 |     { name: "twitter:title", content: "PRATISHA – Human Detection for Search & Rescue" },
90 |     { name: "twitter:description", content: "PRATISHA Snake Robot: real-time human detection for Collapsed Structure
    | Search and Rescue (CSSR) missions. Detect survivors via webcam or uploaded images." },
91 |     { property: "og:image", content: "https://pub-bb2e103a32db4e198524a2e9ed8f35b4.r2.dev/9050a627-0d5b-484d-b980-b4
    | 10b5f89a47/id-preview-dbb128a8--ffeaeelc-78e3-4558-bde2-90cb9631ade2.lovable.app-1783442799709.png" },
92 |     { name: "twitter:image", content: "https://pub-bb2e103a32db4e198524a2e9ed8f35b4.r2.dev/9050a627-0d5b-484d-b980-b
    | 410b5f89a47/id-preview-dbb128a8--ffeaeelc-78e3-4558-bde2-90cb9631ade2.lovable.app-1783442799709.png" },
93 |   ],
94 |   links: [
95 |     { rel: "stylesheet", href: appCss },
96 |     { rel: "icon", type: "image/png", href: "/logo.png" },
97 |     { rel: "apple-touch-icon", href: "/logo.png" },
98 |   ],
99 | }),
100 | shellComponent: RootShell,
101 | component: RootComponent,
102 | notFoundComponent: NotFoundComponent,
103 | errorComponent: ErrorComponent,
104 | });
105 |
106 | function RootShell({ children }: { children: ReactNode }) {
107 |   return (
108 |     <html lang="en">
109 |       <head>
110 |         <HeadContent />
111 |       </head>
112 |       <body>
113 |         {children}
114 |         <Scripts />
115 |       </body>
116 |     </html>
117 |   );
118 | }
119 |
120 | function RootComponent() {
121 |   const { queryClient } = Route.useRouteContext();
122 |
123 |   return (
124 |     <QueryClientProvider client={queryClient}>
125 |       <SettingsProvider>
126 |         /* Required: nested routes render here. Removing <Outlet /> breaks all child routes. */
127 |         <Outlet />
128 |       </SettingsProvider>
129 |     </QueryClientProvider>
130 |   );
131 | }

```

src/routes/index.tsx

```
1 | import { createFileRoute } from "@tanstack/react-router";
2 | import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
3 | import { WebcamDetector } from "@components/WebcamDetector";
4 | import { ThermalDetector } from "@components/ThermalDetector";
5 | import { CooledThermalDetector } from "@components/CooledThermalDetector";
6 | import { ImageDetector } from "@components/ImageDetector";
7 | import { XRayDetector } from "@components/XRayDetector";
8 | import { GasDetector } from "@components/GasDetector";
9 | import { HudOverlay } from "@components/HudOverlay";
10 | import { SystemTicker } from "@components/SystemTicker";
11 | import { HyperspectralDetector } from "@components/HyperspectralDetector";
12 | import { PratishaInfo } from "@components/PratishaInfo";
13 | import { SettingsBar } from "@components/SettingsBar";
14 | import { useSettings } from "@lib/settings";
15 | import { Camera, Image as ImageIcon, Thermometer, Snowflake, ScanLine, Wind, Layers } from "lucide-react";
16 |
17 | import heroImg from "@assets/hero-snake-robot.jpg";
18 | import thermalImg from "@assets/thermal-survivor.jpg";
19 | import closeupImg from "@assets/snake-robot-closeup.jpg";
20 |
21 | export const Route = createFileRoute("/")({
22 |   head: () => ({
23 |     meta: [
24 |       { title: "PRATISHA — Human Detection for Search & Rescue" },
25 |       {
26 |         name: "description",
27 |         content:
28 |           "PRATISHA Snake Robot: real-time human detection for Collapsed Structure Search and Rescue (CSSR) missions. Detect survivors via webcam or uploaded images.",
29 |       },
30 |     ],
31 |   }),
32 |   component: Index,
33 | });
34 |
35 | const NAV_KEYS = [
36 |   { id: "detect", key: "navDetect" },
37 |   { id: "about", key: "navAbout" },
38 |   { id: "components", key: "navComponents" },
39 |   { id: "features", key: "navFeatures" },
40 |   { id: "workflow", key: "navWorkflow" },
41 |   { id: "applications", key: "navApplications" },
42 |   { id: "materials", key: "navMaterials" },
43 |   { id: "cost", key: "navCost" },
44 |   { id: "future", key: "navFuture" },
45 |   { id: "crab", key: "navCrab" },
46 | ] as const;
47 |
48 | function Index() {
49 |   const { t } = useSettings();
50 |   return (
51 |     <div className="min-h-screen bg-background">
52 |       <header className="border-b sticky top-0 z-30 bg-background/80 backdrop-blur">
53 |         <div className="max-w-5xl mx-auto px-6 py-4 flex items-center gap-3 flex-wrap">
54 |           <div className="flex items-center gap-3 mr-auto">
55 |             
62 |             <div>
63 |               <h1
64 |                 className="text-lg font-semibold tracking-tight bg-clip-text text-transparent"
65 |                 style={{ backgroundImage: "var(--gradient-primary)" }}
66 |               >
67 |                 PRATISHA
68 |               </h1>
69 |               <p className="text-xs text-muted-foreground">{t("tagline")}</p>
70 |             </div>
71 |           </div>
72 |           <nav className="flex items-center gap-1 flex-wrap text-xs">
73 |             {NAV_KEYS.map((n) => (
74 |               <a
75 |                 key={n.id}
76 |                 href={`#${n.id}`}
77 |                 className="px-2.5 py-1 rounded-md hover:bg-muted text-muted-foreground hover:text-foreground transition
78 |                 n-colors"
79 |                 >
80 |                   {t(n.key)}
81 |                 </a>
82 |             ))}
83 |           </nav>
84 |           <SettingsBar />
85 |         </div>
86 |       </header>
87 |       <main className="max-w-5xl mx-auto px-6 py-10">
88 |         <section className="relative mb-6 overflow-hidden rounded-2xl border border-primary/30 shadow-[0_0_60px_-15px_
```

```

var(--primary)] hud-corners">
89   <img
90     src={heroImg}
91     alt="PRATISHA snake robot navigating disaster rubble"
92     width={1600}
93     height={900}
94     className="w-full h-[280px] sm:h-[360px] md:h-[440px] object-cover"
95   />
96   <div className="absolute inset-0 bg-gradient-to-t from-background via-background/50 to-background/20" />
97   <HudOverlay />
98   <div className="absolute bottom-0 left-0 right-0 p-6 sm:p-10 pointer-events-none">
99     <p className="text-[10px] font-mono uppercase tracking-[0.3em] text-primary mb-2">
100       [ CSSR · SNAKE ROBOTICS · v3.14 ]
101     </p>
102     <h2 className="text-2xl sm:text-3xl md:text-4xl font-semibold tracking-tight max-w-2xl text-glow">
103       Reaching survivors where humans can't.
104     </h2>
105     <p className="text-sm text-muted-foreground mt-2 max-w-xl font-mono">
106       &gt; Slip through gaps · See · Hear · Sense life – in real time.
107     </p>
108   </div>
109 </section>
110
111 <div className="mb-12">
112   <SystemTicker />
113 </div>
114
115
116
117 <section id="detect" className="scroll-mt-24">
118   <div className="text-center mb-8">
119     <p className="text-xs uppercase tracking-widest text-primary mb-2">
120       {t("liveModule")}
121     </p>
122     <h2
123       className="text-3xl font-semibold tracking-tight bg-clip-text text-transparent inline-block"
124       style={{ backgroundImage: "var(--gradient-primary)" }}
125     >
126       {t("humanDetection")}
127     </h2>
128     <p className="text-sm text-muted-foreground mt-2 max-w-xl mx-auto">
129       {t("detectIntro")}
130     </p>
131   </div>
132   <Tabs defaultValue="webcam" className="space-y-6">
133     <TabsList className="grid grid-cols-2 sm:grid-cols-7 w-full max-w-5xl mx-auto h-auto">
134       <TabsTrigger value="webcam" className="gap-2">
135         <Camera className="w-4 h-4" /> {t("webcam")}
136       </TabsTrigger>
137       <TabsTrigger value="thermal" className="gap-2">
138         <Thermometer className="w-4 h-4" /> Thermal
139       </TabsTrigger>
140       <TabsTrigger value="cooled" className="gap-2">
141         <Snowflake className="w-4 h-4" /> Cooled
142       </TabsTrigger>
143       <TabsTrigger value="hyper" className="gap-2">
144         <Layers className="w-4 h-4" /> Hyper
145       </TabsTrigger>
146       <TabsTrigger value="xray" className="gap-2">
147         <ScanLine className="w-4 h-4" /> X-ray
148       </TabsTrigger>
149       <TabsTrigger value="gas" className="gap-2">
150         <Wind className="w-4 h-4" /> Gas
151       </TabsTrigger>
152       <TabsTrigger value="image" className="gap-2">
153         <ImageIcon className="w-4 h-4" /> {t("upload")}
154       </TabsTrigger>
155     </TabsList>
156     <TabsContent value="webcam">
157       <WebcamDetector />
158     </TabsContent>
159     <TabsContent value="thermal">
160       <ThermalDetector />
161     </TabsContent>
162     <TabsContent value="cooled">
163       <CooledThermalDetector />
164     </TabsContent>
165     <TabsContent value="hyper">
166       <HyperspectralDetector />
167     </TabsContent>
168     <TabsContent value="xray">
169       <XRyDetector />
170     </TabsContent>
171     <TabsContent value="gas">
172       <GasDetector />
173     </TabsContent>
174     <TabsContent value="image">
175       <ImageDetector />
176     </TabsContent>
177   </Tabs>
178   <p className="text-center text-xs text-muted-foreground mt-6">
179     {t("poweredBy")}
180   </p>

```

```

181 |     </section>
182 |
183 |     <section className="mt-16 grid gap-6 md:grid-cols-2">
184 |       <figure className="overflow-hidden rounded-xl border shadow-sm group">
185 |         <img
186 |           src={thermalImg}
187 |           alt="Thermal infrared view of a survivor's heat signature through rubble"
188 |           loading="lazy"
189 |           width={1280}
190 |           height={800}
191 |           className="w-full h-56 object-cover transition-transform duration-500 group-hover:scale-105"
192 |         />
193 |         <figcaption className="p-4">
194 |           <p className="text-xs uppercase tracking-widest text-primary mb-1">
195 |             Thermal vision
196 |           </p>
197 |           <p className="text-sm text-muted-foreground">
198 |             Body-heat signatures pierce dust and darkness – survivors stay visible
199 |             even when buried.
200 |           </p>
201 |         </figcaption>
202 |       </figure>
203 |       <figure className="overflow-hidden rounded-xl border shadow-sm group">
204 |         <img
205 |           src={closeupImg}
206 |           alt="Close-up of the snake robot's modular sensor head"
207 |           loading="lazy"
208 |           width={1280}
209 |           height={800}
210 |           className="w-full h-56 object-cover transition-transform duration-500 group-hover:scale-105"
211 |         />
212 |         <figcaption className="p-4">
213 |           <p className="text-xs uppercase tracking-widest text-primary mb-1">
214 |             Modular sensor head
215 |           </p>
216 |           <p className="text-sm text-muted-foreground">
217 |             Stereo cameras, thermal sensor, and microphone in a 30 mm articulated
218 |             tip – the eyes and ears of the rescue.
219 |           </p>
220 |         </figcaption>
221 |       </figure>
222 |     </section>
223 |
224 |     <PratishaInfo />
225 |
226 |     <footer className="mt-20 pt-8 border-t text-center text-xs text-muted-foreground">
227 |       {t("footer")}
228 |       <div className="mt-2 text-[10px] uppercase tracking-widest text-primary/80">
229 |         Powered by Sarthak Kumar Panda – a ATL Student of DAV School Pokhariput
230 |       </div>
231 |     </footer>
232 |   </main>
233 | </div>
234 | );
235 | }

```

src/server.ts

```
1 | import './lib/error-capture';
2 |
3 | import { consumeLastCapturedError } from './lib/error-capture';
4 | import { renderErrorPage } from './lib/error-page';
5 |
6 | type ServerEntry = {
7 |   fetch: (request: Request, env: unknown, ctx: unknown) => Promise<Response> | Response;
8 | };
9 |
10 | let serverEntryPromise: Promise<ServerEntry> | undefined;
11 |
12 | async function getServerEntry(): Promise<ServerEntry> {
13 |   if (!serverEntryPromise) {
14 |     serverEntryPromise = import('@tanstack/react-start/server-entry').then(
15 |       (m) => (m.default ?? m) as ServerEntry,
16 |     );
17 |   }
18 |   return serverEntryPromise;
19 | }
20 |
21 | // h3 swallows in-handler throws into a normal 500 Response with body
22 | // {"unhandled":true,"message":"HTTPError"} – try/catch alone never fires for those.
23 | async function normalizeCatastrophicSsrResponse(response: Response): Promise<Response> {
24 |   if (response.status < 500) return response;
25 |   const contentType = response.headers.get("content-type") ?? "";
26 |   if (!contentType.includes("application/json")) return response;
27 |
28 |   const body = await response.clone().text();
29 |   if (!body.includes("unhandled:true") || !body.includes("message":"HTTPError")) {
30 |     return response;
31 |   }
32 |
33 |   console.error(consumeLastCapturedError() ?? new Error(`h3 swallowed SSR error: ${body}`));
34 |   return new Response(renderErrorPage(), {
35 |     status: 500,
36 |     headers: { "content-type": "text/html; charset=utf-8" },
37 |   });
38 | }
39 |
40 | export default {
41 |   async fetch(request: Request, env: unknown, ctx: unknown) {
42 |     try {
43 |       const handler = await getServerEntry();
44 |       const response = await handler.fetch(request, env, ctx);
45 |       return await normalizeCatastrophicSsrResponse(response);
46 |     } catch (error) {
47 |       console.error(error);
48 |       return new Response(renderErrorPage(), {
49 |         status: 500,
50 |         headers: { "content-type": "text/html; charset=utf-8" },
51 |       });
52 |     }
53 |   },
54 | };

```

src/start.ts

```
1 | import { createStart, createMiddleware } from "@tanstack/react-start";
2 |
3 | import { renderErrorPage } from "../lib/error-page";
4 |
5 | const errorMiddleware = createMiddleware().server(async ({ next }) => {
6 |   try {
7 |     return await next();
8 |   } catch (error) {
9 |     if (error != null && typeof error === "object" && "statusCode" in error) {
10 |       throw error;
11 |     }
12 |     console.error(error);
13 |     return new Response(renderErrorPage(), {
14 |       status: 500,
15 |       headers: { "content-type": "text/html; charset=utf-8" },
16 |     });
17 |   }
18 | });
19 |
20 | export const startInstance = createStart(() => ({
21 |   requestMiddleware: [errorMiddleware],
22 | }));
```

src/styles.css

```
1 | @import "tailwindcss" source(none);
2 | @source "../src";
3 | @import "tw-animate-css";
4 |
5 | @custom-variant dark (&:is(.dark *));
6 |
7 | /*
8 |  * Design system definition.
9 |  *
10 |  * The @theme inline block maps CSS custom properties to Tailwind utility
11 |  * classes (e.g. --color-primary -> bg-primary, text-primary).
12 |  *
13 |  * The :root and .dark blocks define the actual color values using oklch.
14 |  * All colors MUST use oklch format.
15 |  *
16 |  * To add a new semantic color:
17 |  * 1. Add the variable to :root (light value) and .dark (dark value)
18 |  * 2. Register it in @theme inline as --color-<name>: var(--<name>)
19 |  */
20 |
21 | @theme inline {
22 |   --radius-sm: calc(var(--radius) - 4px);
23 |   --radius-md: calc(var(--radius) - 2px);
24 |   --radius-lg: var(--radius);
25 |   --radius-xl: calc(var(--radius) + 4px);
26 |   --radius-2xl: calc(var(--radius) + 8px);
27 |   --radius-3xl: calc(var(--radius) + 12px);
28 |   --radius-4xl: calc(var(--radius) + 16px);
29 |   --color-background: var(--background);
30 |   --color-foreground: var(--foreground);
31 |   --color-card: var(--card);
32 |   --color-card-foreground: var(--card-foreground);
33 |   --color-popover: var(--popover);
34 |   --color-popover-foreground: var(--popover-foreground);
35 |   --color-primary: var(--primary);
36 |   --color-primary-foreground: var(--primary-foreground);
37 |   --color-secondary: var(--secondary);
38 |   --color-secondary-foreground: var(--secondary-foreground);
39 |   --color-muted: var(--muted);
40 |   --color-muted-foreground: var(--muted-foreground);
41 |   --color-accent: var(--accent);
42 |   --color-accent-foreground: var(--accent-foreground);
43 |   --color-destructive: var(--destructive);
44 |   --color-destructive-foreground: var(--destructive-foreground);
45 |   --color-border: var(--border);
46 |   --color-input: var(--input);
47 |   --color-ring: var(--ring);
48 |   --color-ring-offset-background: var(--background);
49 |   --color-chart-1: var(--chart-1);
50 |   --color-chart-2: var(--chart-2);
51 |   --color-chart-3: var(--chart-3);
52 |   --color-chart-4: var(--chart-4);
53 |   --color-chart-5: var(--chart-5);
54 |   --color-sidebar: var(--sidebar);
55 |   --color-sidebar-foreground: var(--sidebar-foreground);
56 |   --color-sidebar-primary: var(--sidebar-primary);
57 |   --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
58 |   --color-sidebar-accent: var(--sidebar-accent);
59 |   --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
60 |   --color-sidebar-border: var(--sidebar-border);
61 |   --color-sidebar-ring: var(--sidebar-ring);
62 | }
63 |
64 | :root {
65 |   --radius: 0.625rem;
66 |   --background: oklch(1 0 0);
67 |   --foreground: oklch(0.129 0.042 264.695);
68 |   --card: oklch(1 0 0);
69 |   --card-foreground: oklch(0.129 0.042 264.695);
70 |   --popover: oklch(1 0 0);
71 |   --popover-foreground: oklch(0.129 0.042 264.695);
72 |   --primary: oklch(0.208 0.042 265.755);
73 |   --primary-foreground: oklch(0.984 0.003 247.858);
74 |   --secondary: oklch(0.968 0.007 247.896);
75 |   --secondary-foreground: oklch(0.208 0.042 265.755);
76 |   --muted: oklch(0.968 0.007 247.896);
77 |   --muted-foreground: oklch(0.554 0.046 257.417);
78 |   --accent: oklch(0.968 0.007 247.896);
79 |   --accent-foreground: oklch(0.208 0.042 265.755);
80 |   --destructive: oklch(0.577 0.245 27.325);
81 |   --destructive-foreground: oklch(0.984 0.003 247.858);
82 |   --border: oklch(0.929 0.013 255.508);
83 |   --input: oklch(0.929 0.013 255.508);
84 |   --ring: oklch(0.704 0.04 256.788);
85 |   --chart-1: oklch(0.646 0.222 41.116);
86 |   --chart-2: oklch(0.6 0.118 184.704);
87 |   --chart-3: oklch(0.398 0.07 227.392);
88 |   --chart-4: oklch(0.828 0.189 84.429);
89 |   --chart-5: oklch(0.769 0.188 70.08);
90 |   --sidebar: oklch(0.984 0.003 247.858);
```

```

91 | --sidebar-foreground: oklch(0.129 0.042 264.695);
92 | --sidebar-primary: oklch(0.208 0.042 265.755);
93 | --sidebar-primary-foreground: oklch(0.984 0.003 247.858);
94 | --sidebar-accent: oklch(0.968 0.007 247.896);
95 | --sidebar-accent-foreground: oklch(0.208 0.042 265.755);
96 | --sidebar-border: oklch(0.929 0.013 255.508);
97 | --sidebar-ring: oklch(0.704 0.04 256.788);
98 | }
99 |
100 | .dark {
101 |   --background: oklch(0.129 0.042 264.695);
102 |   --foreground: oklch(0.984 0.003 247.858);
103 |   --card: oklch(0.208 0.042 265.755);
104 |   --card-foreground: oklch(0.984 0.003 247.858);
105 |   --popover: oklch(0.208 0.042 265.755);
106 |   --popover-foreground: oklch(0.984 0.003 247.858);
107 |   --primary: oklch(0.929 0.013 255.508);
108 |   --primary-foreground: oklch(0.208 0.042 265.755);
109 |   --secondary: oklch(0.279 0.041 260.031);
110 |   --secondary-foreground: oklch(0.984 0.003 247.858);
111 |   --muted: oklch(0.279 0.041 260.031);
112 |   --muted-foreground: oklch(0.704 0.04 256.788);
113 |   --accent: oklch(0.279 0.041 260.031);
114 |   --accent-foreground: oklch(0.984 0.003 247.858);
115 |   --destructive: oklch(0.704 0.191 22.216);
116 |   --destructive-foreground: oklch(0.984 0.003 247.858);
117 |   --border: oklch(1 0 0 / 10%);
118 |   --input: oklch(1 0 0 / 15%);
119 |   --ring: oklch(0.551 0.027 264.364);
120 |   --chart-1: oklch(0.488 0.243 264.376);
121 |   --chart-2: oklch(0.696 0.17 162.48);
122 |   --chart-3: oklch(0.769 0.188 70.08);
123 |   --chart-4: oklch(0.627 0.265 303.9);
124 |   --chart-5: oklch(0.645 0.246 16.439);
125 |   --sidebar: oklch(0.208 0.042 265.755);
126 |   --sidebar-foreground: oklch(0.984 0.003 247.858);
127 |   --sidebar-primary: oklch(0.488 0.243 264.376);
128 |   --sidebar-primary-foreground: oklch(0.984 0.003 247.858);
129 |   --sidebar-accent: oklch(0.279 0.041 260.031);
130 |   --sidebar-accent-foreground: oklch(0.984 0.003 247.858);
131 |   --sidebar-border: oklch(1 0 0 / 10%);
132 |   --sidebar-ring: oklch(0.551 0.027 264.364);
133 | }
134 |
135 | @layer base {
136 |   * {
137 |     border-color: var(--color-border);
138 |   }
139 |
140 |   body {
141 |     background-color: var(--color-background);
142 |     color: var(--color-foreground);
143 |   }
144 | }
145 |
146 | /* ===== Futuristic HUD utilities ===== */
147 |
148 | @utility cyber-grid {
149 |   background-image:
150 |     linear-gradient(to right, color-mix(in oklab, var(--primary) 14%, transparent) 1px, transparent 1px),
151 |     linear-gradient(to bottom, color-mix(in oklab, var(--primary) 14%, transparent) 1px, transparent 1px);
152 |   background-size: 44px 44px;
153 |   mask-image: radial-gradient(ellipse at center, black 40%, transparent 85%);
154 | }
155 |
156 | @utility scanlines {
157 |   background-image: repeating-linear-gradient(
158 |     to bottom,
159 |     transparent 0px,
160 |     transparent 2px,
161 |     color-mix(in oklab, var(--primary) 10%, transparent) 3px,
162 |     transparent 4px
163 |   );
164 |   pointer-events: none;
165 | }
166 |
167 | @utility text-glow {
168 |   text-shadow:
169 |     0 0 12px color-mix(in oklab, var(--primary) 55%, transparent),
170 |     0 0 32px color-mix(in oklab, var(--primary) 25%, transparent);
171 | }
172 |
173 | .hud-corners { position: relative; }
174 | .hud-corners::before,
175 | .hud-corners::after {
176 |   content: "";
177 |   position: absolute;
178 |   width: 24px;
179 |   height: 24px;
180 |   border: 1.5px solid color-mix(in oklab, var(--primary) 80%, transparent);
181 |   pointer-events: none;
182 |   z-index: 10;
183 | }

```

```

184 | .hud-corners::before { top: 10px; left: 10px; border-right: none; border-bottom: none; }
185 | .hud-corners::after { bottom: 10px; right: 10px; border-left: none; border-top: none; }
186 |
187 | @keyframes hud-sweep {
188 |   0% { transform: translateY(-100%); opacity: 0; }
189 |   10% { opacity: 1; }
190 |   90% { opacity: 1; }
191 |   100% { transform: translateY(100%); opacity: 0; }
192 | }
193 | .animate-hud-sweep { animation: hud-sweep 4s linear infinite; }
194 |
195 | @keyframes hud-pulse {
196 |   0%, 100% { opacity: 1; }
197 |   50% { opacity: 0.35; }
198 | }
199 | .animate-hud-pulse { animation: hud-pulse 1.6s ease-in-out infinite; }
200 |
201 | @keyframes ticker {
202 |   0% { transform: translateX(0); }
203 |   100% { transform: translateX(-50%); }
204 | }
205 | .animate-ticker { animation: ticker 45s linear infinite; }
206 |
207 | /* ---- Motion modes -----
208 | [data-motion="calm"] → slower, subtler animation
209 | [data-motion="off"] → no animation, static fallback
210 | The OS `prefers-reduced-motion` setting also triggers calm as a floor. */
211 |
212 | html[data-motion="calm"] .animate-hud-sweep { animation-duration: 12s; opacity: 0.5; }
213 | html[data-motion="calm"] .animate-hud-pulse { animation-duration: 3.2s; }
214 | html[data-motion="calm"] .animate-ticker { animation-duration: 120s; }
215 | html[data-motion="calm"] .scanlines { opacity: 0.4; }
216 |
217 | html[data-motion="off"] .animate-hud-sweep,
218 | html[data-motion="off"] .animate-hud-pulse,
219 | html[data-motion="off"] .animate-ticker {
220 |   animation: none !important;
221 | }
222 | html[data-motion="off"] .animate-hud-sweep { opacity: 0.35; }
223 | html[data-motion="off"] .scanlines { display: none; }
224 |
225 | @media (prefers-reduced-motion: reduce) {
226 |   html:not([data-motion="full"]) .animate-hud-sweep,
227 |   html:not([data-motion="full"]) .animate-hud-pulse,
228 |   html:not([data-motion="full"]) .animate-ticker {
229 |     animation: none !important;
230 |   }
231 |   html:not([data-motion="full"]) .animate-hud-sweep { opacity: 0.35; }
232 |   html:not([data-motion="full"]) .scanlines { display: none; }
233 | }

```

tsconfig.json

```
1 | {
2 |   "include": ["src/**/*.ts", "src/**/*.tsx", "vite.config.ts", "eslint.config.js"],
3 |   "compilerOptions": {
4 |     "target": "ES2022",
5 |     "jsx": "react-jsx",
6 |     "module": "ESNext",
7 |     "lib": ["ES2022", "DOM", "DOM.Iterable"],
8 |     "types": ["vite/client"],
9 |
10 |    /* Bundler mode */
11 |    "moduleResolution": "Bundler",
12 |    "allowImportingTsExtensions": true,
13 |    "verbatimModuleSyntax": false,
14 |    "noEmit": true,
15 |
16 |    /* Linting */
17 |    "skipLibCheck": true,
18 |    "strict": true,
19 |    "noUnusedLocals": false,
20 |    "noUnusedParameters": false,
21 |    "noFallthroughCasesInSwitch": true,
22 |    "noUncheckedSideEffectImports": true,
23 |    "paths": {
24 |      "@/*": [".src/*"]
25 |    }
26 |   }
27 | }
```

vite.config.ts

```
1 | // @lovable.dev/vite-tanstack-config already includes the following – do NOT add them manually
2 | // or the app will break with duplicate plugins:
3 | //   - tanstackStart, viteReact, tailwindcss, tsConfigPaths, nitro (build-only using cloudflare as a default target),
4 | //     componentTagger (dev-only), VITE_* env injection, @ path alias, React/TanStack dedupe,
5 | //     error logger plugins, and sandbox detection (port/host/strictPort).
6 | // You can pass additional config via defineConfig({ vite: { ... }, etc... }) if needed.
7 | import { defineConfig } from "@lovable.dev/vite-tanstack-config";
8 |
9 | export default defineConfig({
10 |   tanstackStart: {
11 |     // Redirect TanStack Start's bundled server entry to src/server.ts (our SSR error wrapper).
12 |     // nitro/vite builds from this
13 |     server: { entry: "server" },
14 |   },
15 | });
```